

Datenbereinigung und Analyse – Produktsalienz

Gruppe Salienz

Contents

1	Setup und Grundkonfiguration	2
2	Datenaufbereitung und Bereinigung	4
2.1	Headerbereinigung und Entfernung der Testteilnehmenden	4
2.2	Demografischer Überblick vor Ausschluss mobiler Geräte	4
2.3	Gerätefilter und Antwortgeschwindigkeit	11
2.4	Suspicion Probe, Variablenaufbereitung und auffällige Antwortmuster	19
2.5	Demografie nach automatischen Filtern und manuelle Prüfung	22
2.6	Finale Stichprobe, Reshaping und Umpolung der Inkonsistenzskala	29
3	Reliabilität und Skalenbildung	31
4	Explorative Bewertung der fünf Outfits	38
5	Hauptanalyse: Wahrgenommene stilistische Inkonsistenz (WSI)	41
5.1	Hilfsfunktionen für die Hauptanalysen	41
5.2	Produktgröße: GDS vs. KDS	43
5.3	Position: GZS vs. GDS	47
5.4	Stärke (2 Stufen): GDS vs. GDL	50
5.5	Stärke (3 Stufen): M vs. GDL vs. GDS	54
5.6	Schuh-Stärke (2 Stufen): M vs. KDS	59
6	Power-Analyse und Fehlerwahrscheinlichkeiten	62
7	Unterschiede nach Geschlecht	65
7.1	Hilfsfunktionen für die Mixed-ANOVAs	65
7.2	Position $\times$ Geschlecht	68
7.3	Größe $\times$ Geschlecht	72
7.4	Stärke (2 Stufen) $\times$ Geschlecht	76
7.5	Schuh-Stärke (2 Stufen) $\times$ Geschlecht	80
7.6	Stärke (3 Stufen) $\times$ Geschlecht	85
8	Unterschiede nach CVPA	91
8.1	CVPA-Gruppen und Hilfsfunktionen	91
8.2	Position $\times$ CVPA	94
8.3	Größe $\times$ CVPA	99
8.4	Stärke (2 Stufen) $\times$ CVPA	105
8.5	Stärke (3 Stufen) $\times$ CVPA	110
8.6	Schuh-Stärke (2 Stufen) $\times$ CVPA	115

9 Ermüdungseffekt und Reihenfolge	120
10 Mediationsanalyse	133
11 Ausblick: Wahrgenommene Stilzuordnung	138

1 Setup und Grundkonfiguration

Pakete, Bedingungslabels, Grafik-Theme und die Funktionen zum Speichern der Grafiken werden vorbereitet.

```
#MaFo Analyse Code - Gruppe Salienz

#Pakete laden, bei Bedarf vorher noch installieren
library(ggbreak)
library(WRS2)
library(dplyr)
library(tidyr)
library(ggplot2)
library(psych)
library(car)
library(afex)
library(emmeans)
library(rstatix)
library(pwr)
library(tidyverse)
library(lme4)
library(lmerTest)
library(mediation)

# Laden der Daten
salienz <- read.table("raw_data.csv", header = FALSE, sep = ";")

# Interne Condition-Namen bleiben unverändert, damit Filter, Tests und Kontraste exakt
# gleich funktionieren.
# Für sichtbare Tabellen- und Diagrammbeschriftungen werden ausschließlich die
# Abkürzungen verwendet.
bedingung_labels <- c(
  "Jacke-stark-dezentral" = "GDS",
  "Jacke-leicht-dezentral" = "GDL",
  "Schuh-stark-dezentral" = "KDS",
  "Jacke-stark-zentral" = "GZS",
  "Min-Baseline" = "M"
)

kurz_bedingung <- function(x) {
  x <- as.character(x)
  treffer <- x %in% names(bedingung_labels)
  x[treffer] <- unname(bedingung_labels[x[treffer]])
  x
}

# Seitengerechtes Theme für Boxplots und Dichteplots.
# Es verändert ausschließlich die Darstellung, nicht die zugrunde liegenden Daten.
theme_box_density_gross <- theme_minimal(base_size = 11) +
  theme(
    plot.title = element_text(size = 13, face = "bold", hjust = 0.5),
    plot.subtitle = element_text(size = 11, hjust = 0.5),
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 9),
```

```

    legend.title = element_text(size = 9),
    legend.text = element_text(size = 9),
    strip.text = element_text(size = 9, face = "bold"),
    plot.margin = margin(8, 8, 8, 8)
  )

# Alle im Skript erzeugten Grafiken nummeriert als PNG speichern
grafik_ordner <- "Grafiken"
dir.create(grafik_ordner, showWarnings = FALSE, recursive = TRUE)

grafik_counter <- 0

speichere_grafik <- function(plot, name = "Grafik") {
  grafik_counter <- grafik_counter + 1
  sicherer_name <- gsub(
    "[^A-Za-z0-9ÄÖÜäöüß_-]+", "_", name)
  dateiname <- file.path(
    grafik_ordner,
    sprintf(
      "%03d_%s.png",
      grafik_counter,
      sicherer_name))

  # Einheitliche, normale Grafikgröße für gespeicherte PNG-Dateien.
  # Die Analyse selbst bleibt unverändert.
  breite <- 8
  hoehe <- 5.5

  if (grepl("Dichteplots_der_WSI", sicherer_name)) {
    breite <- 8
    hoehe <- 7
  }

  ggsave(
    filename = dateiname,
    plot = plot,
    width = breite,
    height = hoehe,
    dpi = 300)
  invisible(plot)}

# Für Grafiken, die im ursprünglichen Code direkt erzeugt und angezeigt werden,
# wird die Grafik gespeichert und anschließend wie bisher als Ergebnis zurückgegeben.
zeige_und_speichere_grafik <- function(plot, name = "Grafik") {
  speichere_grafik(plot, name)
  plot
}

# Es sind 255 obs., davon sind 3 Header
nrow(salienz)

```

Table 1: Kontrolle des eingelesenen Rohdatensatzes

Kennwert	Wert
Zeilen im Rohdatensatz vor Headerbereinigung	255

2 Datenaufbereitung und Bereinigung

2.1 Headerbereinigung und Entfernung der Testteilnehmenden

```
# Data Preperation & Cleaning

# 1. Extrahieren der zweiten Zeile als Header
new_header <- as.character(salienz[2, ])
print(new_header)

# 2. Entfernen der ersten beiden Zeilen (alte Header und neue Headerzeile)
salienz <- salienz[-c(1, 2), ]

# 3. der neuen Headerzeilen als Spaltennamen
colnames(salienz) <- new_header

# 4. Entfernen erster Zeile
salienz <- salienz[-c(1), ]

# 5. Zuweisung von unique col names
colnames(salienz) <- make.names(colnames(salienz), unique = TRUE)

# Es sind 252 Teilnehmer
nrow(salienz)

# 6. Testteilnehmer entfernen
# diese haben Matheaufgaben nicht beantwortet -> dort steht aber kein N/A sondern -77
# wegen unipark
class(salienz$Plus.Rechnung.2)
salienz <- salienz %>%
  filter(Plus.Rechnung.2 != "-77")

# Es bleiben 247 Teilnehmer
nrow(salienz)
```

Table 2: Stichprobengröße nach erstem Ausschluss

Kennwert	Wert
Teilnehmende nach Entfernung der Testteilnehmenden	247

2.2 Demografischer Überblick vor Ausschluss mobiler Geräte

```
# Beginn Analyse
# 1. Ziel: Stichproben Demografie aller Teilnehmer - inklusive Handy User - zum
# Überblick verschaffen

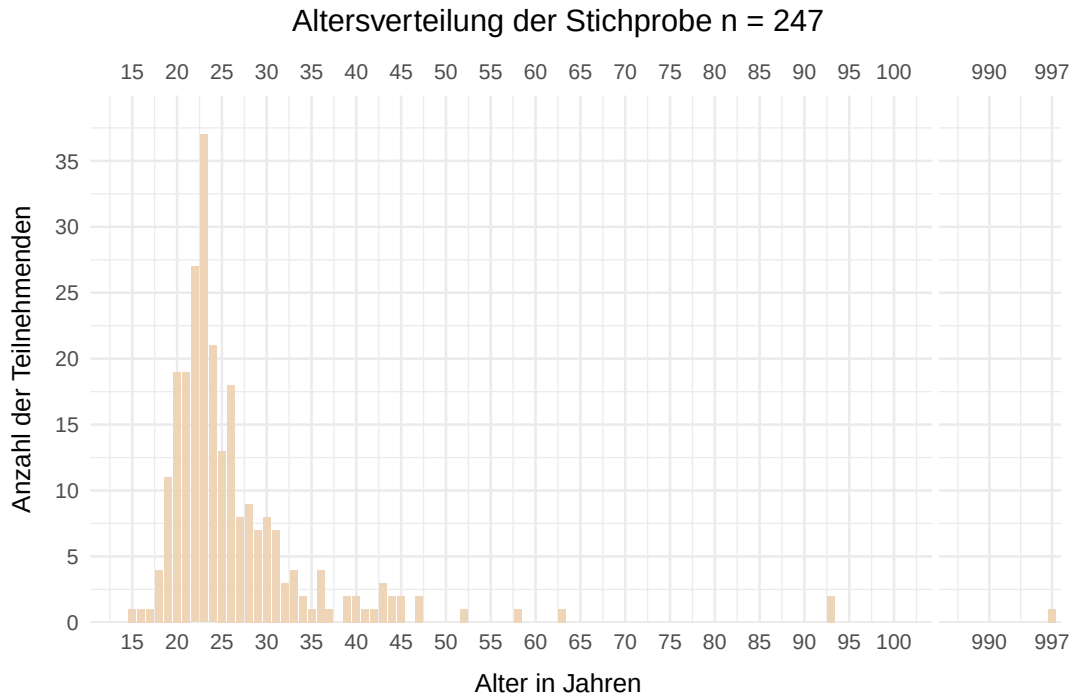
# Altersverteilung
# Alter in numerische Variable umwandeln
salienz <- salienz %>%
  mutate(Alter = as.numeric(Alter))

# Häufigkeiten pro Alter berechnen
alter_plot <- salienz %>%
  filter(!is.na(Alter)) %>%
  count(Alter)
```

```

# Balkendiagramm
zeige_und_speichere_grafik(
  ggplot(alter_plot, aes(x = Alter, y = n)) +
    geom_col(fill = "#EED5B7", width = 0.8) +
    scale_x_continuous(
      breaks = c(seq(15, 100, by = 5), 990, 997)) +
    scale_x_break(c(100, 985)) +
    scale_y_continuous(
      breaks = scales::pretty_breaks(n = 10),
      expand = expansion(mult = c(0, 0.08))) +
    labs(
      title = "Altersverteilung der Stichprobe n = 247",
      x = "Alter in Jahren",
      y = "Anzahl der Teilnehmenden") +
    theme_minimal() +
    theme(
      plot.title = element_text(
        hjust = 0.5)),
  "Altersverteilung der Stichprobe"
)

```



```

# Geschlechtsverteilung
# Häufigkeiten und Prozentwerte berechnen
geschlecht_plot <- salienz %>%
  filter(!is.na(Geschlecht)) %>%
  count(Geschlecht) %>%
  mutate(
    Prozent = n / sum(n) * 100,
    Geschlecht_Label = factor(
      as.character(Geschlecht),
      levels = as.character(1:4),
      labels = c(
        "weiblich",
        "männlich",
        "divers",

```

```

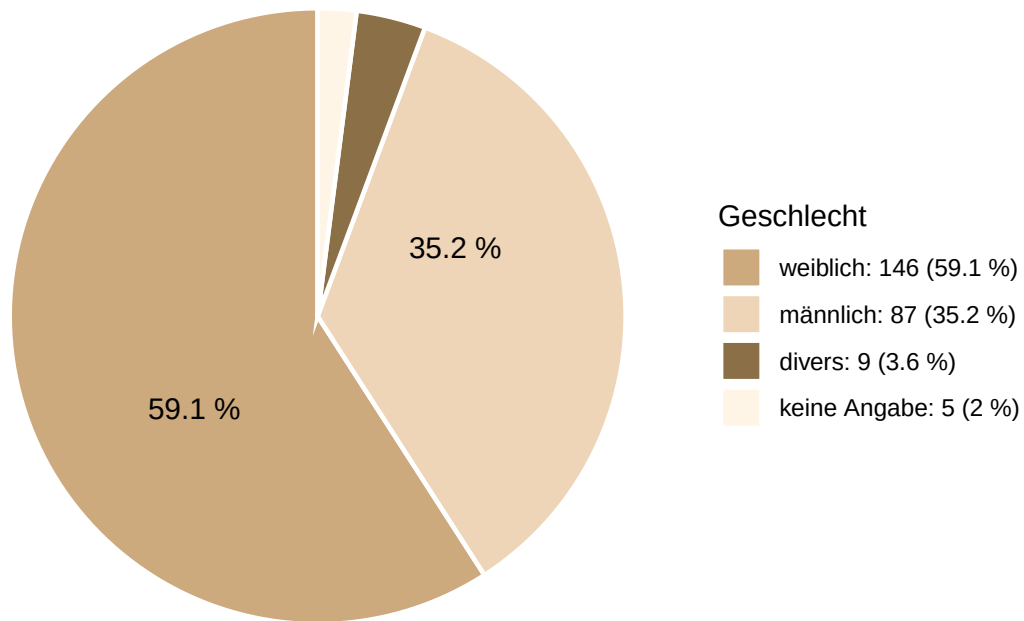
    "keine Angabe"))))

geschlecht_legende <- setNames(
  paste0(
    as.character(geschlecht_plot$Geschlecht_Label),
    ": ",
    geschlecht_plot$n,
    " (",
    round(geschlecht_plot$Prozent, 1),
    "%)" ),
  as.character(geschlecht_plot$Geschlecht_Label)
)

# Kreisdiagramm
# Kleine Segmente werden nicht im Kreis beschriftet, damit sich Texte nicht überlagern.
# Die vollständigen Angaben stehen in der Legende.
zeige_und_speichere_grafik(
  ggplot(geschlecht_plot, aes(x = "", y = n, fill = Geschlecht_Label)) +
    geom_col(width = 1, color = "white", linewidth = 0.8) +
    coord_polar(theta = "y") +
    geom_text(
      data = geschlecht_plot %>% filter(Prozent >= 5),
      aes(label = paste0(round(Prozent, 1), "%")),
      position = position_stack(vjust = 0.5),
      size = 4) +
    scale_fill_manual(
      values = c(
        "weiblich" = "#CDAA7D",
        "männlich" = "#EED5B7",
        "divers" = "#8B6F47",
        "keine Angabe" = "#FFF5E6"),
      labels = geschlecht_legende) +
    labs(
      title = "Geschlechterverteilung der Stichprobe n = 247",
      fill = "Geschlecht") +
    theme_void() +
    theme(
      plot.title = element_text(hjust = 0.5),
      legend.position = "right") +
    guides(fill = guide_legend(title.position = "top")),
  "Geschlechterverteilung der Stichprobe"
)

```

Geschlechterverteilung der Stichprobe n = 247



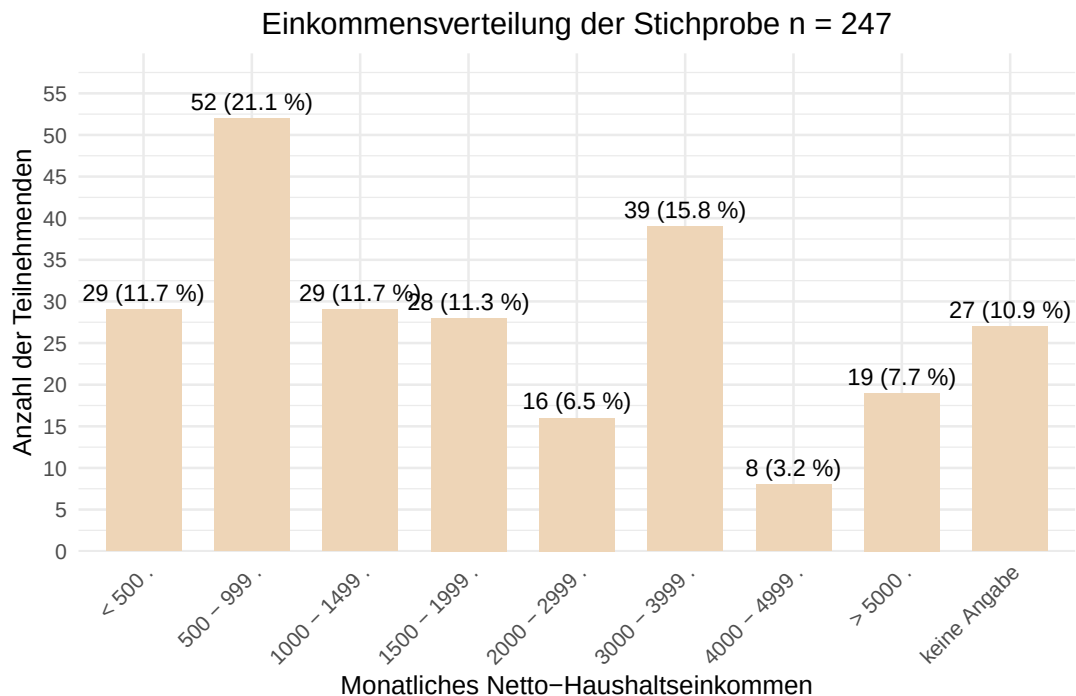
```
#Einkommensverteilung
# Reihenfolge der Einkommenskategorien festlegen
einkommen_plot <- salienz %>%
  filter(!is.na(Einkommen)) %>%
  count(Einkommen) %>%
  mutate(
    Prozent = n / sum(n) * 100,
    Einkommen_Label = factor(
      as.character(Einkommen),
      levels = as.character(1:9),
      labels = c(
        "< 500 €",
        "500 - 999 €",
        "1000 - 1499 €",
        "1500 - 1999 €",
        "2000 - 2999 €",
        "3000 - 3999 €",
        "4000 - 4999 €",
        "> 5000 €",
        "keine Angabe"
      )))

# Balkendiagramm
zeige_und_speichere_grafik(
  ggplot(einkommen_plot, aes(x = Einkommen_Label, y = n)) +
  geom_col(fill = "#EED5B7", width = 0.7) +
  geom_text(
    aes(label = paste0(n, " (", round(Prozent, 1), " %)")),
    vjust = -0.5,
    size = 3.5) +
  scale_y_continuous(
    breaks = scales::pretty_breaks(n = 10),
    expand = expansion(
      mult = c(0, 0.15))) +
  labs(
    title = "Einkommensverteilung der Stichprobe n = 247",
```

```

x = "Monatliches Netto-Haushaltseinkommen",
y = "Anzahl der Teilnehmenden") +
theme_minimal() +
theme(
  plot.title = element_text(
    hjust = 0.5),
  axis.text.x = element_text(
    angle = 45,
    hjust = 1)),
"Einkommensverteilung der Stichprobe"
)

```



```

# Tätigkeitsverteilung
# Häufigkeiten und Prozentwerte berechnen
taetigkeit_plot <- salienz %>%
  filter(!is.na(Beruf)) %>%
  count(Beruf) %>%
  mutate(
    Prozent = n / sum(n) * 100,
    Beruf_Label = factor(
      as.character(Beruf),
      levels = as.character(1:3),
      labels = c(
        "Student/in",
        "Berufstätig",
        "Sonstiges")))

# Kreisdiagramm
zeige_und_speichere_grafik(
  ggplot(taetigkeit_plot, aes(x = "", y = n, fill = Beruf_Label)) +
  geom_col(width = 1, color = "white") +
  coord_polar(theta = "y") +
  geom_text(
    aes(label = paste0(Beruf_Label, "\n", n, " (", round(Prozent, 1), " %)")),
    position = position_stack(vjust = 0.5),
    size = 3.8) +

```



```

scale_fill_manual(values = c(
  "Student/in" = "#CDAA7D",
  "Berufstätig" = "#EED5B7",
  "Sonstiges" = "#FFF5E6")) +
labs(
  title = "Tätigkeitsverteilung der Stichprobe n = 247",
  fill = "Aktuelle Haupttätigkeit") +
theme_void() +
theme(
  plot.title = element_text(hjust = 0.5),
  legend.position = "none"),
"Tätigkeitsverteilung der Stichprobe"
)

```

Tätigkeitsverteilung der Stichprobe n = 247

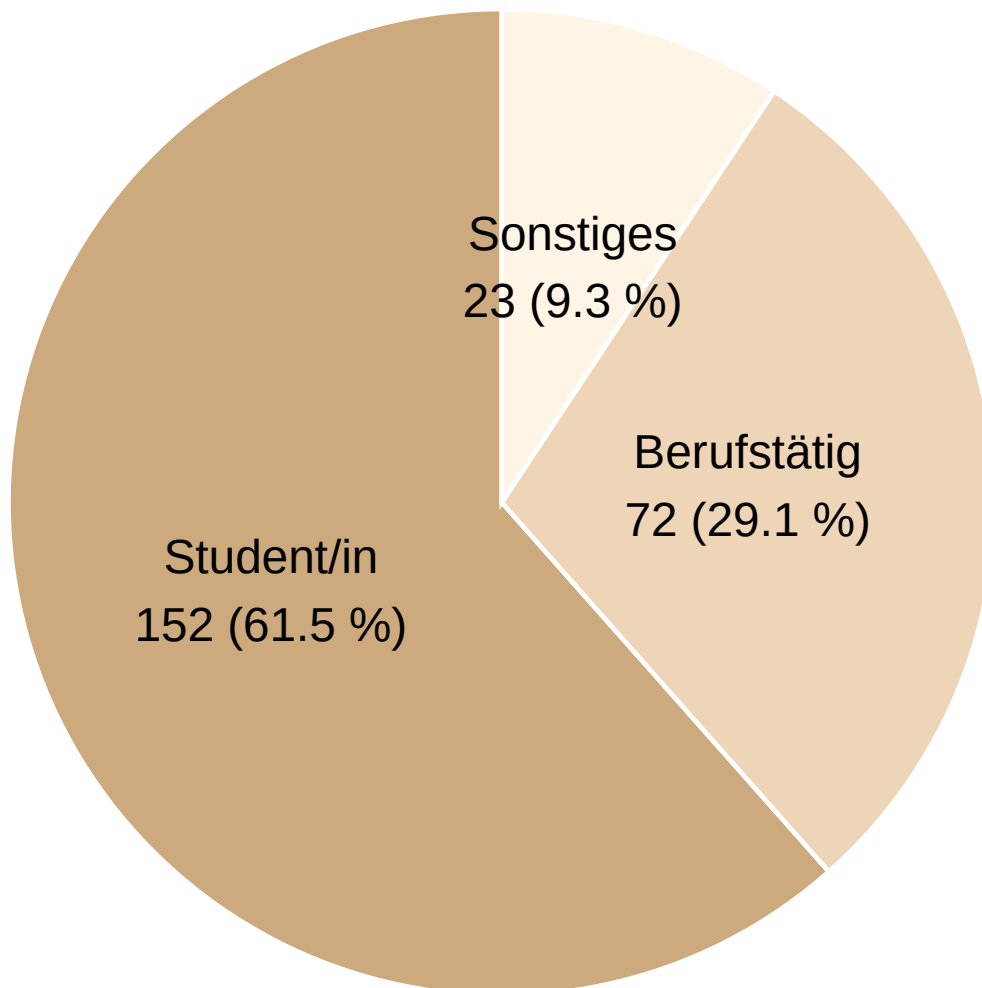


Table 3: Altersverteilung vor Gerätefilter

Alter	n
15	1
16	1
17	1
18	4
19	11
20	19
21	19
22	27
23	37
24	21
25	13
26	18
27	8
28	9
29	7
30	8
31	7
32	3
33	4
34	2
35	1
36	4
37	1
39	2
40	2
41	1
42	1
43	3
44	2
45	2
47	2
52	1
58	1
63	1
93	2
997	1

Table 4: Geschlechtsverteilung vor Gerätefilter

Geschlecht	n	Prozent	Geschlecht_Label
1	146	59.109	weiblich
2	87	35.223	männlich
3	9	3.644	divers
4	5	2.024	keine Angabe

Table 5: Einkommensverteilung vor Gerätefilter

Einkommen	n	Prozent	Einkommen_Label
1	29	11.741	< 500 €
2	52	21.053	500 - 999 €
3	29	11.741	1000 - 1499 €
4	28	11.336	1500 - 1999 €
5	16	6.478	2000 - 2999 €
6	39	15.789	3000 - 3999 €
7	8	3.239	4000 - 4999 €
8	19	7.692	> 5000 €
9	27	10.931	keine Angabe

Table 6: Tätigkeitsverteilung vor Gerätefilter

Beruf	n	Prozent	Beruf_Label
1	152	61.538	Student/in
2	72	29.150	Berufstätig
3	23	9.312	Sonstiges

2.3 Gerätefilter und Antwortgeschwindigkeit

```

# Fortsetzung Data Preperation & Cleaning
# Ausschlusskriterien umsetzen
# 1. Mobile Geräte ausschließen

# Mit Hilfe von browser.id bestimmen, ob PC/Laptop oder mobiles Gerät genutzt wurde
salienz <- salienz %>%
  mutate( Geraetekategorie = case_when(
    # Mobiles Endgerät
    grepl(
      "Mobile|Android|iPhone|iPad|iPod|Windows Phone|Tablet",
      browser.id,
      ignore.case = TRUE) ~ "Mobilgerät",
    # PC/Laptop Betriebssysteme
    grepl(
      "Windows NT|Macintosh|X11|Linux|CrOS",
      browser.id,
      ignore.case = TRUE) ~ "PC / Laptop",
    # Extra-Kategorie falls ein browser.id nicht zugeordnet werden kann und dann
    # manuell prüfen
    TRUE ~ "Unklar"))

# Die Geräte auswählen und Anteile berechnen
geraete_plot <- salienz %>%
  filter(Geraetekategorie %in% c("PC / Laptop", "Mobilgerät")) %>%
  count(Geraetekategorie) %>%
  mutate(Prozent = n / sum(n) * 100)

# Kreisdiagramm für Verteilung der Endgeräte
zeige_und_speichere_grafik(
  ggplot(

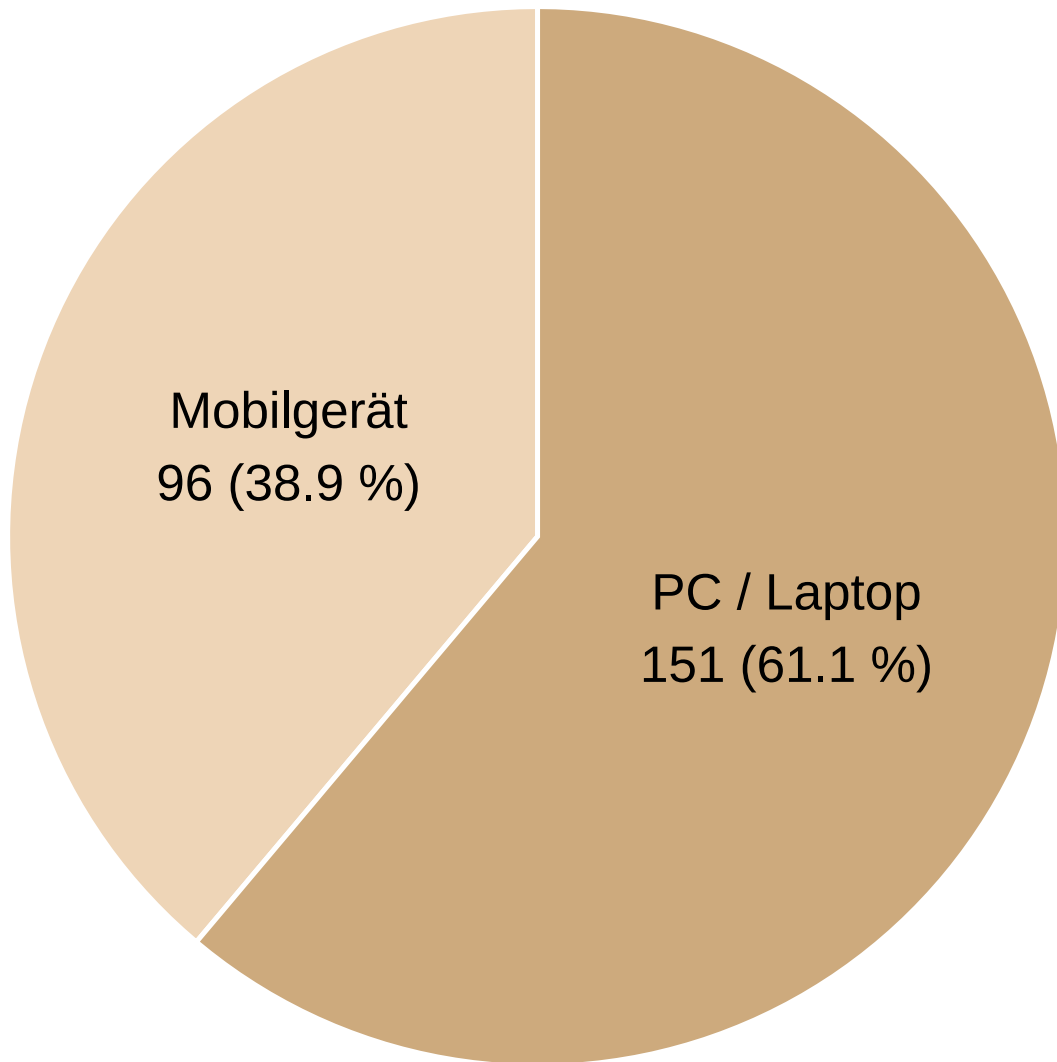
```

```

geraete_plot,
aes(x = "", y = n, fill = Geraetekategorie)) +
geom_col(width = 1, color = "white") +
coord_polar(theta = "y") +
geom_text(
  aes(label = paste0(Geraetekategorie, "\n", n, " (", round(Prozent, 1), " %)"),
    position = position_stack(vjust = 0.5),
    size = 3.8) +
scale_fill_manual(values = c(
  "PC / Laptop" = "#CDAA7D",
  "Mobilgerät" = "#EED5B7")) +
labs(
  title = "Verteilung der verwendeten Endgeräte",
  fill = "Gerätekategorie") +
theme_void() +
theme(
  legend.position = "none",
  plot.title = element_text(hjust = 0.5)),
"Verteilung der verwendeten Endgeräte"
)

```

Verteilung der verwendeten Endgeräte



```
# mobile Endgeräte aus dem Datensatz salienz entfernen
salienz <- salienz %>%
  filter(Geraetekategorie == "PC / Laptop")

# Es bleiben 151 Teilnehmer
nrow(salienz)

# 2. Gesamtbearbeitungszeit filtern

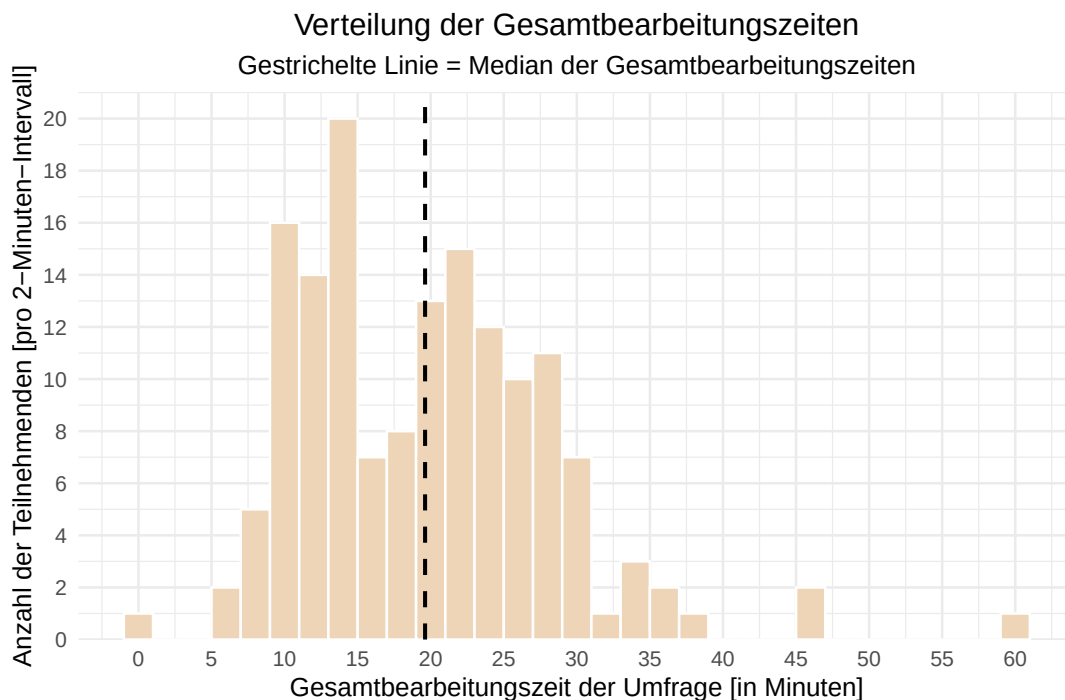
# Gesamtbearbeitungszeit von Sekunden in Minuten
salienz <- salienz %>%
  mutate(Bearbeitungszeit_Minuten = as.numeric(time.to.complete.survey) / 60)

# Balkendia der Gesamtbearbeitungszeiten erstellen
zeige_und_speichere_grafik(
  ggplot(salienz, aes(x = Bearbeitungszeit_Minuten)) +
```

```

geom_histogram(binwidth = 2, fill = "#EED5B7", color = "white") +
geom_vline(
  xintercept = median(
    salienz$Bearbeitungszeit_Minuten,
    na.rm = TRUE
  ),
  linetype = "dashed",
  linewidth = 0.8
) +
scale_x_continuous(breaks = scales::pretty_breaks(n = 15)) +
scale_y_continuous(breaks = scales::pretty_breaks(n = 10), expand = expansion(mult = c(0, 0.05))) +
labs(
  title = "Verteilung der Gesamtbearbeitungszeiten",
  subtitle = "Gestrichelte Linie = Median der Gesamtbearbeitungszeiten",
  x = "Gesamtbearbeitungszeit der Umfrage [in Minuten]",
  y = "Anzahl der Teilnehmenden [pro 2-Minuten-Intervall]") +
theme_minimal() +
theme(
  legend.position = "none",
  plot.title = element_text(
    hjust = 0.5),
  plot.subtitle = element_text(
    hjust = 0.5)),
"Verteilung der Gesamtbearbeitungszeiten"
)

```



```

# Berechnen der Bearbeitungszeiten der fünf Outfits, die relative timestamps sind
# Zeitstempel
# -> also muss immer der der vorhergehenden Seite abgezogen werden
salienz <- salienz %>%
  mutate(
    Zeit_7698435 =
      as.numeric(relative.timestamp.for.page.7698435) -
      as.numeric(relative.timestamp.for.page.7698434),
    Zeit_7698431 =
      as.numeric(relative.timestamp.for.page.7698431) -

```

```

    as.numeric(relative.timestamp.for.page.7698430),
    Zeit_7698433 =
      as.numeric(relative.timestamp.for.page.7698433) -
      as.numeric(relative.timestamp.for.page.7698432),
    Zeit_7698438 =
      as.numeric(relative.timestamp.for.page.7698438) -
      as.numeric(relative.timestamp.for.page.7698437),
    Zeit_7698427 =
      as.numeric(relative.timestamp.for.page.7698427) -
      as.numeric(relative.timestamp.for.page.7698426)
  )

# Speichern der Namen der fünf Bearbeitungszeitvariablen
zeit_spalten <- c("Zeit_7698435", "Zeit_7698431", "Zeit_7698433", "Zeit_7698438", "Zeit_7698427")

# Bestimmen des Medians der Bearbeitungszeit für jedes Outfit
seiten_mediane <- sapply(
  salienz[, zeit_spalten],
  median,
  na.rm = TRUE)
print(seiten_mediane)

# Berechnen der relativen Speed Faktoren nach Leiner (2019) mit der Formel:
# Speed = Median der Seite / individuelle Bearbeitungszeit und auf maximal 3 festlegen
salienz <- salienz %>%
  mutate(
    Speed_7698435 = pmin(seiten_mediane["Zeit_7698435"] / Zeit_7698435, 3),
    Speed_7698431 = pmin(seiten_mediane["Zeit_7698431"] / Zeit_7698431, 3),
    Speed_7698433 = pmin(seiten_mediane["Zeit_7698433"] / Zeit_7698433, 3),
    Speed_7698438 = pmin(seiten_mediane["Zeit_7698438"] / Zeit_7698438, 3),
    Speed_7698427 = pmin(seiten_mediane["Zeit_7698427"] / Zeit_7698427, 3))

# Bilden der durchschnittlichen Relative Completion Speed Indize
salienz <- salienz %>%
  mutate(
    Relative_Speed_Index = rowMeans(
      across(
        c(Speed_7698435, Speed_7698431, Speed_7698433, Speed_7698438, Speed_7698427)),
      na.rm = FALSE))

# Identifizieren von auffällig schnellen Teilnehmern
salienz <- salienz %>%
  mutate(Antwortzeit_auffaellig = Relative_Speed_Index > 2)

# Anzahl anzeigen
table(salienz$Antwortzeit_auffaellig)

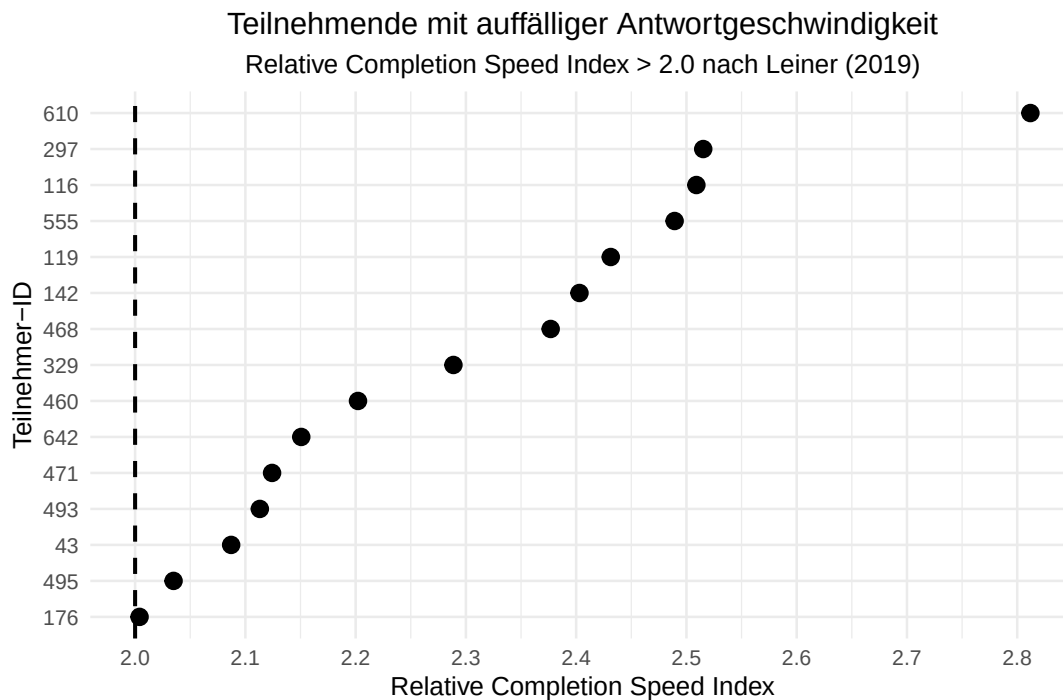
# Betrachten der Verteilung der mit Relative Speed Index > 2.0
zeige_und_speichere_grafik(
  salienz %>%
    filter(Relative_Speed_Index > 2.0) %>%
    ggplot(aes(x = Relative_Speed_Index, y = reorder(as.factor(number), Relative_Speed_Index))) +
    geom_point(size = 3) +
    geom_vline(xintercept = 2, linetype = "dashed", linewidth = 0.8) +
    scale_x_continuous(
      breaks = seq(2, ceiling(max(salienz$Relative_Speed_Index, na.rm = TRUE) * 10) / 10, by = 0.1)) +
    labs(
      title = "Teilnehmende mit auffälliger Antwortgeschwindigkeit",
      subtitle = "Relative Completion Speed Index > 2.0 nach Leiner (2019)",
      x = "Relative Completion Speed Index",
      y = "Teilnehmer-ID") +

```

```

theme_minimal() +
theme(
  legend.position = "none",
  plot.title = element_text(hjust = 0.5),
  plot.subtitle = element_text(hjust = 0.5)),
"Teilnehmende mit auffälliger Antwortgeschwindigkeit"
)

```



```

# Betroffene Teilnehmer anzeigen
salienz %>%
  filter(Antwortzeit_auffaellig) %>%
  dplyr::select(
    number,
    Bearbeitungszeit_Minuten,
    Relative_Speed_Index,
    all_of(zeit_spalten)
  ) %>%
  arrange(desc(Relative_Speed_Index))

# Betroffene Teilnehmer in der Gesamtbearbeitungsverteilung farbig markieren
zeige_und_speichere_grafik(
  ggplot(salienz, aes(x = Bearbeitungszeit_Minuten, fill = Antwortzeit_auffaellig)) +
  geom_histogram(binwidth = 2, color = "white") +
  # Median der Gesamtbearbeitungszeit
  geom_vline(
    xintercept = median(
      salienz$Bearbeitungszeit_Minuten,
      na.rm = TRUE
    ),
    linetype = "dashed",
    linewidth = 0.8
  ) +
  # Farben für unauffällige und auffällige Personen
  scale_fill_manual(
    values = c("FALSE" = "#EED5B7", "TRUE" = "grey50"),
    labels = c("FALSE" = "Nicht auffällig", "TRUE" = "Auffällig, da Relative Speed Index > 2.0")) +

```



```

scale_x_continuous(
  breaks = scales::pretty_breaks(n = 15)) +
scale_y_continuous(
  breaks = scales::pretty_breaks(n = 10),
  expand = expansion(mult = c(0, 0.05))) +
labs(
  title = "Auffällige Bearbeitungszeiten nach dem Relative Speed Index",
  x = "Gesamtbearbeitungszeit der Umfrage [in Minuten]",
  y = "Anzahl der Teilnehmenden [pro 2-Minuten-Intervall]",
  fill = "Legende") +
guides(
  fill = guide_legend(
    nrow = 1,
    byrow = TRUE)) +
theme_minimal() +
theme(
  legend.position = "bottom",
  plot.title = element_text(hjust = 0.5)),
"Auffällige Gesamtbearbeitungszeiten"
)

```

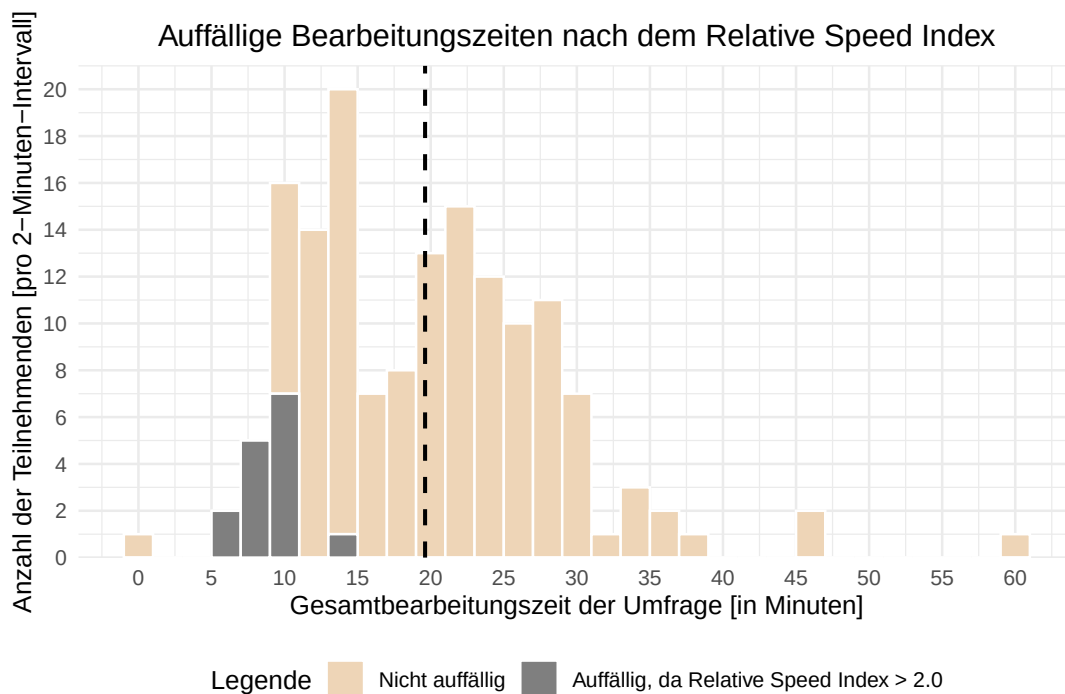


Table 7: Verteilung der verwendeten Endgeräte

Geraetekategorie	n	Prozent
Mobilgerät	96	38.866
PC / Laptop	151	61.134

Table 8: Mediane Bearbeitungszeiten der fünf Outfit-Seiten

Kennwert	Wert
Zeit_7698435	138

Kennwert	Wert
Zeit_7698431	155
Zeit_7698433	135
Zeit_7698438	142
Zeit_7698427	122

Table 9: Auffällige Antwortgeschwindigkeit nach Relative Speed Index

Var1	Freq
FALSE	136
TRUE	15

Table 10: Teilnehmende mit auffälliger Antwortgeschwindigkeit - Teil 1 von 2

number	Bearbeitungszeit_Minuten	Relative_Speed_Index	Zeit_7698435	Zeit_7698431
610	6.500	2.812	67	22
297	6.933	2.515	80	54
116	7.917	2.509	29	59
555	7.417	2.489	53	54
119	9.283	2.431	131	35
142	7.017	2.403	129	19
468	10.100	2.377	47	76
329	7.633	2.289	56	52
460	10.850	2.202	55	162
642	10.333	2.151	92	63
471	14.117	2.124	59	56
493	10.300	2.113	78	55
43	10.317	2.087	103	51
495	9.000	2.035	60	127
176	10.683	2.004	84	73

Table 11: Teilnehmende mit auffälliger Antwortgeschwindigkeit - Teil 2 von 2

number	Bearbeitungszeit_Minuten	Zeit_7698433	Zeit_7698438	Zeit_7698427
610	6.500	25	32	22
297	6.933	60	52	36
116	7.917	100	52	43
555	7.417	73	52	51
119	9.283	32	43	58
142	7.017	21	73	34
468	10.100	45	113	46

number	Bearbeitungszeit_Minuten	Zeit_7698433	Zeit_7698438	Zeit_7698427
329	7.633	50	102	64
460	10.850	57	51	51
642	10.333	39	59	88
471	14.117	75	61	88
493	10.300	138	46	61
43	10.317	91	80	43
495	9.000	56	101	43
176	10.683	63	62	67

```
# Auffällige Teilnehmer aus dem Datensatz salienz entfernen
salienz <- salienz %>%
  filter(!Antwortzeit_auffaellig)

# Es bleiben 136 Teilnehmer
nrow(salienz)
```

Table 12: Stichprobengröße nach Antwortzeitfilter

Kennwert	Wert
Verbleibende Teilnehmende nach Antwortzeitfilter	136

2.4 Suspicion Probe, Variablenaufbereitung und auffällige Antwortmuster

```
#3. Suspicion Probe scannen
# Kommentare alle anzeigen und speichern um sie manuell zu prüfen
print(salienz$Suspicion.Probe)

suspicionprobe_antworten <- salienz %>%
  dplyr::select(
    number,
    Suspicion.Probe)
write.csv2(
  suspicionprobe_antworten,
  "SuspicionProbe_Antworten.csv",
  row.names = FALSE,
  fileEncoding = "UTF-8")

#Unnötige Spalten entfernen
# Die Spalten 237 bis 241 bleiben aber weil sie die Randomisierungsreihenfolge der
# fünf Outfits enthalten
salienz <- salienz %>%
  dplyr::select(
    -c(
      2:20,
      186:236,
      242:ncol(salienz)))

# Spalten übergreifend umbenennen

# Konsistenz
```

```

names(salienz)[grepl("uneinheitlich", names(salienz))] <- "Konsistenz1"
names(salienz)[grepl("unstimmig", names(salienz))] <- "Konsistenz2"
names(salienz)[grepl("unharmonisch", names(salienz))] <- "Konsistenz3"

# Interesse
names(salienz)[grepl("langweilig", names(salienz))] <- "Interesse1"
names(salienz)[grepl("uninteressant", names(salienz))] <- "Interesse2"

# Liking
names(salienz)[grepl("Das.Outfit.gefällt.mir.gut", names(salienz))] <- "Liking1"
names(salienz)[grepl("Ich.mag.das.Outfit", names(salienz))] <- "Liking2"
names(salienz)[grepl("Ich.finde.das.Outfit.ansprechend", names(salienz))] <- "Liking3"

# Fluency
names(salienz)[grepl("das.Outfit.zu.bewerten", names(salienz))] <- "Fluency1"
names(salienz)[grepl("das.Outfit.visuell.schnell.zu.erfassen", names(salienz))] <- "Fluency2"
names(salienz)[grepl("geschlossenen.Augen", names(salienz))] <- "Fluency3"
names(salienz)[grepl("einer.andere.Person.zu.beschreiben", names(salienz))] <- "Fluency4"

# Kreativität
names(salienz)[grepl("wirkt.kreativ", names(salienz))] <- "Kreativitaet1"
names(salienz)[grepl("wirkt.originell", names(salienz))] <- "Kreativitaet2"
names(salienz)[grepl("wirkt.einfallsreich", names(salienz))] <- "Kreativitaet3"
names(salienz)[grepl("hebt.sich.von.modischen.Normen.ab", names(salienz))] <- "Kreativitaet4"
names(salienz)[grepl("ist.unkonventionell", names(salienz))] <- "Kreativitaet5"

# Authentizität
names(salienz)[grepl("Zusammenstellung.glaubwürdig", names(salienz))] <- "Authentizitaet1"
names(salienz)[grepl("nicht.künstlich.erzwungen", names(salienz))] <- "Authentizitaet2"
names(salienz)[grepl("eigenständigen.Charakter", names(salienz))] <- "Authentizitaet3"
names(salienz)[grepl("nicht.aufgesetzt", names(salienz))] <- "Authentizitaet4"

# Sophisticated Taste
names(salienz)[grepl("gekonnt.zusammengestellt", names(salienz))] <- "SophisticatedTaste1"
names(salienz)[grepl("stilvoll.kleidet", names(salienz))] <- "SophisticatedTaste2"
names(salienz)[grepl("kleidet.sich.bewusst", names(salienz))] <- "SophisticatedTaste3"
names(salienz)[grepl("raffinierten.Geschmack", names(salienz))] <- "SophisticatedTaste4"

# Manipulations-Stilcheck
names(salienz)[grepl("Minimalismus", names(salienz))] <- "Minimalismus"
names(salienz)[grepl("Hip.Hop", names(salienz))] <- "HipHopActiveWear"
names(salienz)[grepl("Hippie", names(salienz))] <- "HippieBoho"

# Matheaufgabe
mathe_spalten <- grep("Plus.Rechnung", names(salienz))
# Kontrolle dass fünf Blöcke mit jeweils zwei Aufgaben
length(mathe_spalten)
names(salienz)[mathe_spalten] <- rep(
  c("Matheaufgabe1", "Matheaufgabe2"),
  length.out = length(mathe_spalten))

# CVPA
names(salienz)[grepl("hervorragendes.Design", names(salienz))] <- "CVPA1"
names(salienz)[grepl("Auslagen.von.Produkten", names(salienz))] <- "CVPA2"
names(salienz)[grepl("Quelle.der.Freude", names(salienz))] <- "CVPA3"
names(salienz)[grepl("Welt.lebenswerter", names(salienz))] <- "CVPA4"
names(salienz)[grepl("feine.Unterschiede.im.Produktdesign", names(salienz))] <- "CVPA5"
names(salienz)[grepl("anderen.meist.entgehen", names(salienz))] <- "CVPA6"
names(salienz)[grepl("zu.andere.Dinge.passt", names(salienz))] <- "CVPA7"
names(salienz)[grepl("besser.aussehen.lässt", names(salienz))] <- "CVPA8"
names(salienz)[grepl("in.seinen.Bann.zieht", names(salienz))] <- "CVPA9"

```

```

# Randomisierung
names(salienz)[167:171] <- c(
  "Randomisierung_1",
  "Randomisierung_2",
  "Randomisierung_3",
  "Randomisierung_4",
  "Randomisierung_5")
names(salienz)[167:171]

# Rename Columns by Block sodass man ablesen kann zu welchem Outfit es gehört
rename_columns_with_suffix <- function(df, start, end, suffix) {
  names(df)[start:end] <- paste0(names(df)[start:end], suffix)
  return(df)}

# Start at column 2 bis column 31 ist erstes Outfit
start <- 2

# Define block sizes (number of columns per block, wir haben 5 Blöcke mit je 30 Fragen)
block_sizes <- c(30, 30, 30, 30, 30)
suffixes <- c("_Jacke-stark-dezentral",
  "_Jacke-leicht-dezentral",
  "_Schuh-stark-dezentral",
  "_Jacke-stark-zentral",
  "_Min-Baseline")

for (i in seq_along(suffixes)) {
  end <- start + block_sizes[i] - 1
  # Check if end exceeds total columns
  if (end > ncol(salienz)) end <- ncol(salienz)
  # Rename the columns
  salienz <- rename_columns_with_suffix(
    salienz,
    start,
    end,
    suffixes[i])
  # Next block starts after this one
  start <- end + 1
  if (start > ncol(salienz)) break}

# Die fünf experimentellen Bedingungen
conditions <- c(
  "Jacke-stark-dezentral",
  "Jacke-leicht-dezentral",
  "Schuh-stark-dezentral",
  "Jacke-stark-zentral",
  "Min-Baseline")

# Regulärer Ausdruck für die fünf Suffixe
condition_regex <- paste(conditions, collapse = "|")

# Alle Spalten ermitteln, die zu einem Outfitblock gehören
outfit_columns <- grep(
  paste0("_(", condition_regex, ")$"),
  names(salienz),
  value = TRUE)
print(outfit_columns)
length(outfit_columns)

```

```

#4.Auffällige Antwortmuster
# Jene speichern, die innerhalb einer Outfit-Frageseite straightlinen
straightlining_pro_outfit <- sapply(conditions, function(condition) {
  # Alle Spalten der jeweiligen Outfit-Frageseite
  aktuelle_spalten <- grep(
    paste0("_", condition, "$"),
    names(salienz),
    value = TRUE)
  # Ohne Matheaufgaben
  aktuelle_spalten <- aktuelle_spalten[!grepl("^Matheaufgabe", aktuelle_spalten)]
  # Für jeden Teilnehmer prüfen ob innerhalb dieser Outfit-Seite alle Antworten gleich
  # sind
  apply(
    salienz[, aktuelle_spalten, drop = FALSE],
    1,
    function(antworten) {
      all(length(unique(antworten)) == 1)
    }
  )
})

# Prüfen, ob ein Teilnehmer auf ALLEN fünf Outfit-Seiten gestraightlined hat
straightlining_alle_outfits <- apply(straightlining_pro_outfit, 1, all)

# Anzahl der betroffenen Teilnehmer anzeigen
table(straightlining_alle_outfits)

# Betroffene Teilnehmer speichern zum Angucken seiner Antworten
straightliner_teilnehmer <- salienz[straightlining_alle_outfits, , drop = FALSE]

# Teilnehmer-IDs separat speichern
straightliner_ids <- salienz$number[straightlining_alle_outfits]

# Betroffene Teilnehmer-IDs anzeigen
print(straightliner_ids)

# Anzahl der betroffenen Teilnehmer anzeigen
length(straightliner_ids)

```

Table 13: Straightlining über alle fünf Outfit-Seiten

straightlining_alle_outfits	Freq
FALSE	135
TRUE	1

Table 14: IDs der identifizierten Straightliner

Wert
32

2.5 Demografie nach automatischen Filtern und manuelle Prüfung

```

# Beginn der Datenanalyse mit bereinigtem Datensatz
# Altersverteilung

```

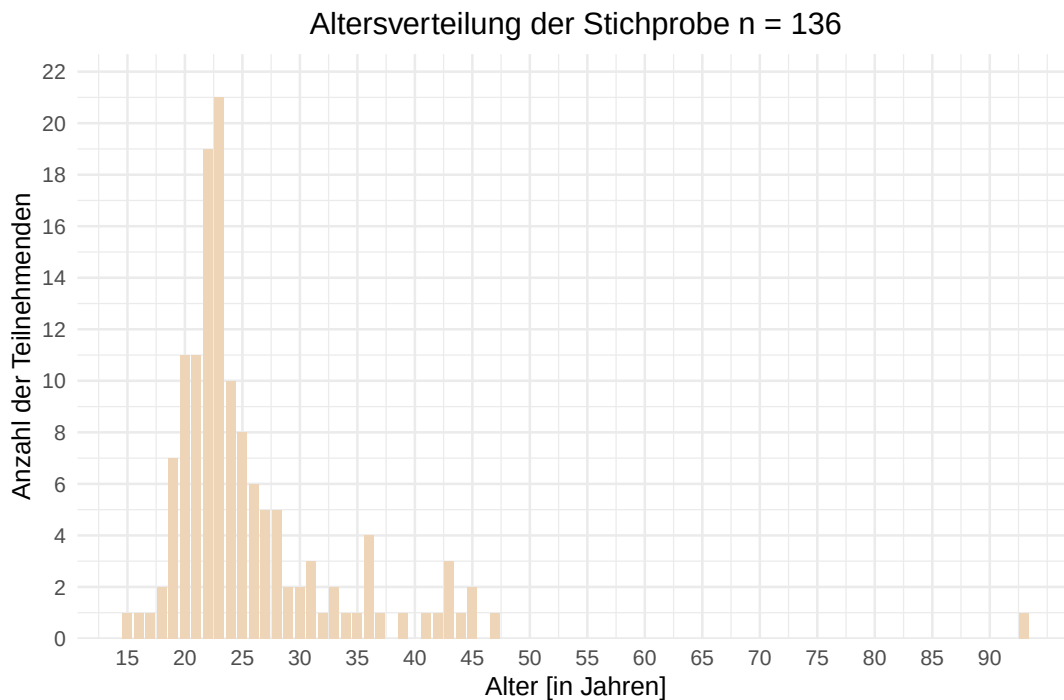
```

# Alter in numerische Variable umwandeln
salienz <- salienz %>%
  mutate(Alter = as.numeric(Alter))

# Häufigkeiten pro Alter berechnen
alter_plot <- salienz %>%
  filter(!is.na(Alter)) %>%
  count(Alter)

# Balkendiagramm machen
zeige_und_speichere_grafik(
  ggplot(alter_plot, aes(x = Alter, y = n)) +
    geom_col(fill = "#EED5B7", width = 0.8) +
    scale_x_continuous(
      breaks = seq(
        floor(min(alter_plot$Alter)),
        ceiling(max(alter_plot$Alter)),
        by = 5)) +
    scale_y_continuous(
      breaks = scales::pretty_breaks(n = 10),
      expand = expansion(mult = c(0, 0.08))) +
    labs(
      title = "Altersverteilung der Stichprobe n = 136",
      x = "Alter [in Jahren]",
      y = "Anzahl der Teilnehmenden") +
    theme_minimal() +
    theme(
      plot.title = element_text(
        hjust = 0.5)),
  "Altersverteilung der Stichprobe"
)

```



```

#Geschlechtsverteilung
# Häufigkeiten und Prozentwerte berechnen
geschlecht_plot <- salienz %>%

```

```

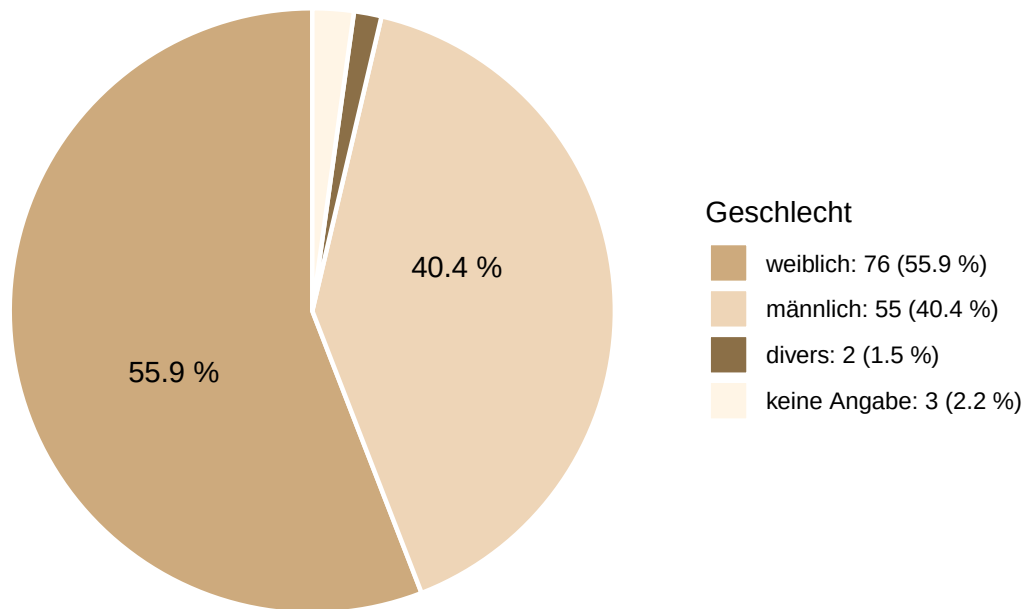
filter(!is.na(Geschlecht)) %>%
count(Geschlecht) %>%
mutate(
  Prozent = n / sum(n) * 100,
  Geschlecht_Label = factor(
    as.character(Geschlecht),
    levels = as.character(1:4),
    labels = c(
      "weiblich",
      "männlich",
      "divers",
      "keine Angabe")))

geschlecht_legende <- setNames(
  paste0(
    as.character(geschlecht_plot$Geschlecht_Label),
    ": ",
    geschlecht_plot$n,
    " (",
    round(geschlecht_plot$Prozent, 1),
    " %)" ),
  as.character(geschlecht_plot$Geschlecht_Label)
)

# Kreisdiagramm
# Kleine Segmente werden nicht im Kreis beschriftet, damit sich Texte nicht überlagern.
# Die vollständigen Angaben stehen in der Legende.
zeige_und_speichere_grafik(
  ggplot(geschlecht_plot, aes(x = "", y = n, fill = Geschlecht_Label)) +
  geom_col(width = 1, color = "white", linewidth = 0.8) +
  coord_polar(theta = "y") +
  geom_text(
    data = geschlecht_plot %>% filter(Prozent >= 5),
    aes(label = paste0(round(Prozent, 1), " %")),
    position = position_stack(vjust = 0.5),
    size = 4) +
  scale_fill_manual(
    values = c(
      "weiblich" = "#CDAA7D",
      "männlich" = "#EED5B7",
      "divers" = "#8B6F47",
      "keine Angabe" = "#FFF5E6"),
    labels = geschlecht_legende) +
  labs(
    title = "Geschlechterverteilung der Stichprobe n = 136",
    fill = "Geschlecht") +
  theme_void() +
  theme(
    plot.title = element_text(hjust = 0.5),
    legend.position = "right") +
  guides(fill = guide_legend(title.position = "top")),
  "Geschlechterverteilung der Stichprobe"
)

```


Geschlechterverteilung der Stichprobe n = 136



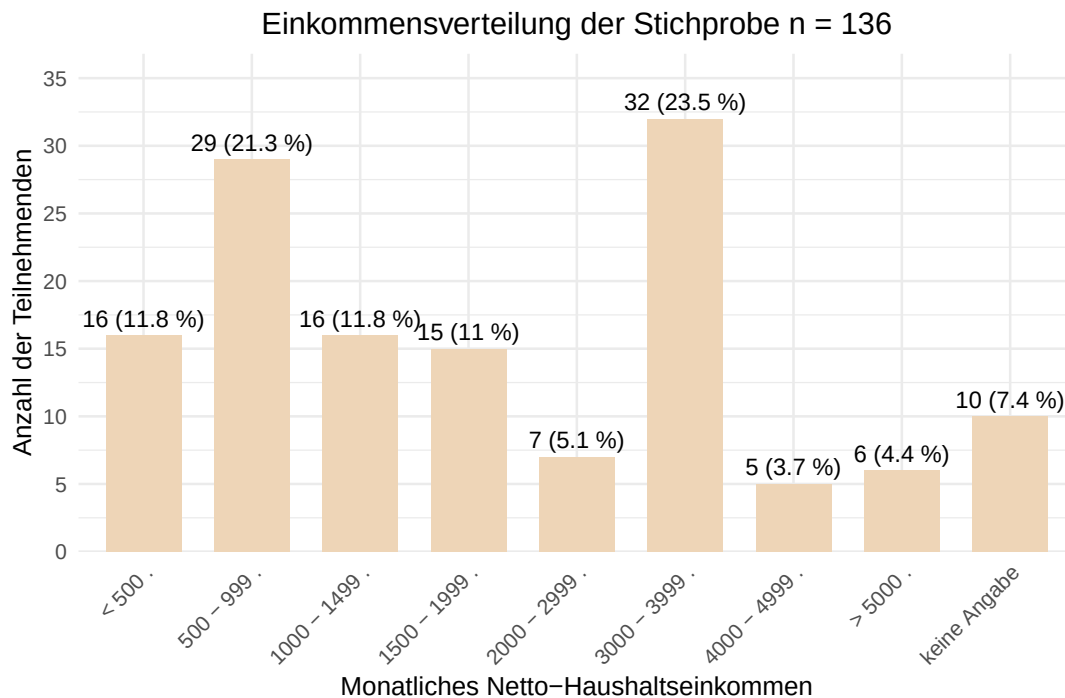
```
# Einkommensverteilung
# Reihenfolge der Einkommenskategorien festlegen
einkommen_plot <- salienz %>%
  filter(!is.na(Einkommen)) %>%
  count(Einkommen) %>%
  mutate(
    Prozent = n / sum(n) * 100,
    Einkommen_Label = factor(
      as.character(Einkommen),
      levels = as.character(1:9),
      labels = c(
        "< 500 €",
        "500 - 999 €",
        "1000 - 1499 €",
        "1500 - 1999 €",
        "2000 - 2999 €",
        "3000 - 3999 €",
        "4000 - 4999 €",
        "> 5000 €",
        "keine Angabe"))))

# Balkendiagramm
zeige_und_speichere_grafik(
  ggplot(einkommen_plot, aes(x = Einkommen_Label, y = n)) +
  geom_col(fill = "#EED5B7", width = 0.7) +
  geom_text(
    aes(label = paste0(n, " (", round(Prozent, 1), "%)")),
    vjust = -0.5,
    size = 3.5) +
  scale_y_continuous(
    breaks = scales::pretty_breaks(n = 10),
    expand = expansion(
      mult = c(0, 0.15))) +
  labs(
    title = "Einkommensverteilung der Stichprobe n = 136",
    x = "Monatliches Netto-Haushaltseinkommen",
```

```

  y = "Anzahl der Teilnehmenden") +
  theme_minimal() +
  theme(
    plot.title = element_text(
      hjust = 0.5),
    axis.text.x = element_text(
      angle = 45,
      hjust = 1)),
  "Einkommensverteilung der Stichprobe"
)

```



```

# Tätigkeitsverteilung
# Häufigkeiten und Prozentwerte berechnen
taetigkeit_plot <- salienz %>%
  filter(!is.na(Beruf)) %>%
  count(Beruf) %>%
  mutate(
    Prozent = n / sum(n) * 100,
    Beruf_Label = factor(
      as.character(Beruf),
      levels = as.character(1:3),
      labels = c(
        "Student/in",
        "Berufstätig",
        "Sonstiges")))

# Kreisdiagramm
zeige_und_speichere_grafik(
  ggplot(taetigkeit_plot, aes(x = "", y = n, fill = Beruf_Label)) +
  geom_col(width = 1, color = "white") +
  coord_polar(theta = "y") +
  geom_text(
    aes(label = paste0(Beruf_Label, "\n", n, " (", round(Prozent, 1), " %)")),
    position = position_stack(vjust = 0.5),
    size = 3.8) +
  scale_fill_manual(values = c(

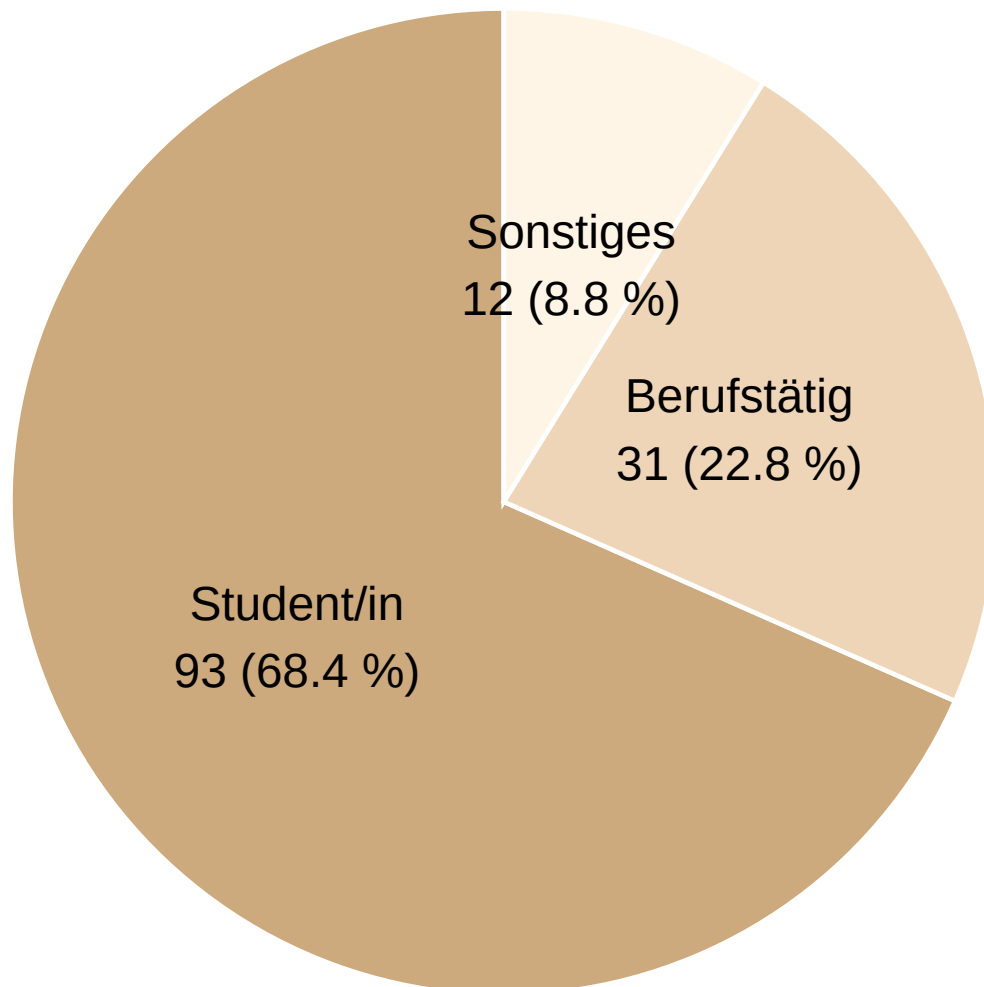
```

```

"Student/in" = "#CDAA7D",
"Berufstätig" = "#EED5B7",
"Sonstiges" = "#FFF5E6")) +
labs(
  title = "Tätigkeitsverteilung der Stichprobe n = 136",
  fill = "Aktuelle Haupttätigkeit") +
theme_void() +
theme(
  plot.title = element_text(hjust = 0.5),
  legend.position = "none"),
"Tätigkeitsverteilung der Stichprobe"
)

```

Tätigkeitsverteilung der Stichprobe n = 136



```

# Ausreißer mit Id ? 32 genauer betrachten
# Jetzt die demografischen Werte vom Verdächtigen mit ID 32 von der
# Antwortmusteranalyse betrachten
salienz %>%
  filter(as.character(number) == "32") %>%
  dplyr::select(

```

```

number,
Alter,
Geschlecht,
Einkommen,
Beruf,
Suspicion.Probe
)
# Sein Alter und Suspicion Probe sind sehr suspicious. Grund genug, den Datensatz zu
# entfernen.

```

Table 15: Altersverteilung nach den automatischen Filtern

Alter	n
15	1
16	1
17	1
18	2
19	7
20	11
21	11
22	19
23	21
24	10
25	8
26	6
27	5
28	5
29	2
30	2
31	3
32	1
33	2
34	1
35	1
36	4
37	1
39	1
41	1
42	1
43	3
44	1
45	2
47	1
93	1

Table 16: Geschlechtsverteilung nach den automatischen Filtern

Geschlecht	n	Prozent	Geschlecht_Label
1	76	55.882	weiblich
2	55	40.441	männlich
3	2	1.471	divers
4	3	2.206	keine Angabe

Table 17: Einkommensverteilung nach den automatischen Filtern

Einkommen	n	Prozent	Einkommen_Label
1	16	11.765	< 500 €
2	29	21.324	500 - 999 €
3	16	11.765	1000 - 1499 €
4	15	11.029	1500 - 1999 €
5	7	5.147	2000 - 2999 €
6	32	23.529	3000 - 3999 €
7	5	3.676	4000 - 4999 €
8	6	4.412	> 5000 €
9	10	7.353	keine Angabe

Table 18: Tätigkeitsverteilung nach den automatischen Filtern

Beruf	n	Prozent	Beruf_Label
1	93	68.382	Student/in
2	31	22.794	Berufstätig
3	12	8.824	Sonstiges

Table 19: Manuell betrachteter Teilnehmer mit ID 32

Kennwert	Wert
number	32
Alter	93
Geschlecht	3
Einkommen	8
Beruf	2
Suspicion.Probe	Du hättest viel mehr Teilnehmer, wenn du paar Gutscheine verlosen würdest, DU GEIZHALS!!!!!!!!!!!!!!

2.6 Finale Stichprobe, Reshaping und Umpolung der Inkonsistenzskala

```

#Teilnehmer mit ID 32 aus dem Datensatz entfernen
salienz <- salienz %>%
  filter(as.character(number) != "32")

# Es bleiben 135 Teilnehmer
nrow(salienz)

# Finale Daten reshapen ins Long-Format

# Eine Zeile pro Person und Outfit erzeugen
salienz_resaped <- salienz %>%
  pivot_longer(
    cols = all_of(outfit_columns),
    # Der Teil vorm letzten Unterstrich wird zum Spaltenname
    # Der Teil danach wird zur Condition
    names_to = c(".value", "Condition"),
    names_pattern = paste0(
      "^(.*)_",
      condition_regex,
      "$")

# Anzeigen der reshaped Data
print(salienz_resaped)

# Konsistenz zu Inkonsistenz umpolen
# Die ursprünglichen Konsistenz-Items sind so kodiert:
# 1 = hohe Inkonsistenz
# 5 = hohe Konsistenz
# Für die Forschungsfrage polen wir sie aber der Einfachheit halber um:
# 1 = hohe Konsistenz
# 5 = hohe Inkonsistenz
# Bei unserer Skala von 1 bis 5 müssen wir also immer 6 - ursprünglicher Wert rechnen
salienz_resaped <- salienz_resaped %>%
  mutate(
    across(
      c(Konsistenz1, Konsistenz2, Konsistenz3),
      ~ 6 - as.numeric(.x)))

# Umbenennung der Variablen Konsistenz zu Inkonsistenz
salienz_resaped <- salienz_resaped %>%
  rename(
    Inkonsistenz1 = Konsistenz1,
    Inkonsistenz2 = Konsistenz2,
    Inkonsistenz3 = Konsistenz3)

# Zeilenanzahl prüfen, 135 x 5 = 675
nrow(salienz_resaped)

```

Table 20: Finale Analysestichprobe

Kennwert	Wert
Finale Teilnehmendenzahl	135
Zeilen im Long-Format	675

3 Reliabilität und Skalenbildung

Die internen Konsistenzen werden zunächst über alle Outfits und anschließend getrennt nach Outfit und Konstrukt geprüft. Danach werden die Skalenmittelwerte gebildet.

```
# Cronbach Alpha

# Prüfen ob die Variablen in einer Form sind mit der wir arbeiten können
summary(salienz_reshaped)

# Umwandlung in numerisch notwendig!
# Inkonsistenz
salienz_reshaped <- salienz_reshaped %>% mutate(across(Inkonsistenz1:Inkonsistenz3, as.numeric))

# Interesse
salienz_reshaped <- salienz_reshaped %>% mutate(across(Interesse1:Interesse2, as.numeric))

# Liking
salienz_reshaped <- salienz_reshaped %>% mutate(across(Liking1:Liking3, as.numeric))

# Fluency
salienz_reshaped <- salienz_reshaped %>% mutate(across(Fluency1:Fluency4, as.numeric))

# Kreativität
salienz_reshaped <- salienz_reshaped %>% mutate(across(Kreativitaet1:Kreativitaet5, as.numeric))

# Authentizität
salienz_reshaped <- salienz_reshaped %>% mutate(across(Authentizitaet1:Authentizitaet4, as.numeric))

# Sophisticated Taste
salienz_reshaped <- salienz_reshaped %>% mutate(across(SophisticatedTaste1:SophisticatedTaste4,
  ↪ as.numeric))

# CVPA
salienz_reshaped <- salienz_reshaped %>% mutate(across(CVPA1:CVPA9, as.numeric))

# Berechnung von Cronbach Alpha und Interpretation nach Faustregel nach (Blaž, 2015):
# mind. 0.8

# Einmal über alle Outfits hinweg
# Inkonsistenz
inkonsistenz_items <- salienz_reshaped %>%
  dplyr::select(Inkonsistenz1, Inkonsistenz2, Inkonsistenz3)
# Berechnung von Cronbach's Alpha
cronbach_alpha_inkonsistenz <- psych::alpha(inkonsistenz_items)
# Ergebnis anzeigen
print(cronbach_alpha_inkonsistenz)
# 0.90 (raw_alpha)
# Sehr gute Konsistenz der Skala. Aber potenziell redundante Items. Wir können die
# Skala so verwenden.

# Interesse
interesse_items <- salienz_reshaped %>%
  dplyr::select(Interesse1, Interesse2)
# Berechnung von Cronbach's Alpha
cronbach_alpha_interesse <- psych::alpha(interesse_items)
# Ergebnis anzeigen
print(cronbach_alpha_interesse)
```

```

# 0.87 (raw_alpha)
# Gute Konsistenz der Skala. Wir können die Skala so verwenden.

# Liking
liking_items <- salienz_reshaped %>%
  dplyr::select(Liking1, Liking2, Liking3)
# Berechnung von Cronbach's Alpha
cronbach_alpha_liking <- psych::alpha(liking_items)
# Ergebnis anzeigen
print(cronbach_alpha_liking)
# 0.95 (raw_alpha)
# Sehr gute Konsistenz der Skala. Aber potenziell redundante Items. Wir können die
# Skala so verwenden.

# Fluency
fluency_items <- salienz_reshaped %>%
  dplyr::select(Fluency1, Fluency2, Fluency3, Fluency4)
# Berechnung von Cronbach's Alpha
cronbach_alpha_fluency <- psych::alpha(fluency_items)
# Ergebnis anzeigen
print(cronbach_alpha_fluency)
# 0.85 (raw_alpha)
# Gute Konsistenz der Skala. Wir können die Skala so verwenden.

# Kreativitaet
creativity_items <- salienz_reshaped %>%
  dplyr::select(Kreativitaet1, Kreativitaet2, Kreativitaet3, Kreativitaet4, Kreativitaet5)
# Berechnung von Cronbach's Alpha
cronbach_alpha_creativity <- psych::alpha(creativity_items)
# Ergebnis anzeigen
print(cronbach_alpha_creativity)
# 0.87 (raw_alpha)
# Gute Konsistenz der Skala. Wir können die Skala so verwenden.

# Authentizität
authentizitaet_items <- salienz_reshaped %>%
  dplyr::select(Authentizitaet1, Authentizitaet2, Authentizitaet3, Authentizitaet4)
# Berechnung von Cronbach's Alpha
cronbach_alpha_authentizitaet <- psych::alpha(authentizitaet_items)
# Ergebnis anzeigen
print(cronbach_alpha_authentizitaet)
# 0.84 (raw_alpha)
# Gute Konsistenz der Skala. Wir können die Skala so verwenden.

# SophisticatedTaste
sophisticated_items <- salienz_reshaped %>%
  dplyr::select(SophisticatedTaste1, SophisticatedTaste2, SophisticatedTaste3, SophisticatedTaste4)
# Berechnung von Cronbach's Alpha
cronbach_alpha_sophisticated <- psych::alpha(sophisticated_items)
# Ergebnis anzeigen
print(cronbach_alpha_sophisticated)
# 0.86 (raw_alpha)
# Gute Konsistenz der Skala. Wir können die Skala so verwenden.

# CVPA
cvpa_items <- salienz_reshaped %>%
  dplyr::select(CVPA1, CVPA2, CVPA3, CVPA4, CVPA5, CVPA6, CVPA7, CVPA8, CVPA9)
# Berechnung von Cronbach's Alpha
cronbach_alpha_cvpa <- psych::alpha(cvpa_items)
# Ergebnis anzeigen
print(cronbach_alpha_cvpa)

```



```

# 0.84 (raw_alpha)
# Gute Konsistenz der Skala. Wir können die Skala so verwenden.

#Und eInmal für jedes Outfit getrennt mit for
# Konstrukte definieren
konstrukte <- list(
  Inkonsistenz = c("Inkonsistenz1", "Inkonsistenz2", "Inkonsistenz3"),
  Interesse = c("Interesse1", "Interesse2"),
  Liking = c("Liking1", "Liking2", "Liking3"),
  Fluency = c("Fluency1", "Fluency2", "Fluency3", "Fluency4"),
  Kreativitaet = c(
    "Kreativitaet1", "Kreativitaet2", "Kreativitaet3",
    "Kreativitaet4", "Kreativitaet5"
  ),
  Authentizitaet = c(
    "Authentizitaet1", "Authentizitaet2",
    "Authentizitaet3", "Authentizitaet4"
  ),
  SophisticatedTaste = c(
    "SophisticatedTaste1", "SophisticatedTaste2",
    "SophisticatedTaste3", "SophisticatedTaste4"
  )
)

# Leeren Ergebnis-Datensatz erstellen
cronbach_pro_outfit <- data.frame()

# Cronbach Alpha für jedes Outfit und jedes Konstrukt berechnen mit loop
for (outfit in conditions) {
  for (konstrukt in names(konstrukte)) {
    daten <- salienz_reshaped %>%
      filter(
        Condition == outfit
      ) %>%
      dplyr::select(
        all_of(konstrukte[[konstrukt]]))
    # Cronbach Alpha berechnen und speichern
    alpha_result <- psych::alpha(daten, warnings = FALSE)
    cronbach_pro_outfit <- rbind(
      cronbach_pro_outfit,
      data.frame(
        Outfit = outfit,
        Konstrukt = konstrukt,
        Cronbach_Alpha = alpha_result$total$raw_alpha))}
  }

# Cronbach Alpha auf drei Nachkommastellen runden
cronbach_pro_outfit$Cronbach_Alpha <- round(cronbach_pro_outfit$Cronbach_Alpha, 3)

# Interpretation ergänzen nach Blanz (2015) damit bisschen übersichtlicher
cronbach_pro_outfit <- cronbach_pro_outfit %>%
  mutate(
    Interpretation = case_when(
      Cronbach_Alpha > 0.9 ~ "Sehr gut",
      Cronbach_Alpha > 0.8 ~ "Gut",
      Cronbach_Alpha > 0.7 ~ "Akzeptabel",
      Cronbach_Alpha > 0.6 ~ "Fragwürdig",
      TRUE ~ "Nicht ausreichend"),
    Outfit = kurz_bedingung(Outfit),
    Konstrukt = ifelse(Konstrukt == "Inkonsistenz", "WSI", Konstrukt))
  }

```

```

print(cronbach_pro_outfit)

# Alle Skalen sind konsistent Mittelwerte können gebildet werden!

#Mittelwerte bilden
# Bilde den Mittelwert zwischen den Konsistenz Spalten
salienz_reshaped$Mittelwert_Inkonsistenz <- rowMeans(
  salienz_reshaped[, c(
    "Inkonsistenz1", "Inkonsistenz2", "Inkonsistenz3"
  )],
  na.rm = TRUE
)

# Bilde den Mittelwert zwischen den Interesse Spalten
salienz_reshaped$Mittelwert_Interesse <- rowMeans(
  salienz_reshaped[, c("Interesse1", "Interesse2")],
  na.rm = TRUE
)

# Bilde den Mittelwert zwischen den Liking Spalten
salienz_reshaped$Mittelwert_Liking <- rowMeans(
  salienz_reshaped[, c("Liking1", "Liking2", "Liking3")],
  na.rm = TRUE
)

# Bilde den Mittelwert zwischen den Fluency Spalten
salienz_reshaped$Mittelwert_Fluency <- rowMeans(
  salienz_reshaped[, c(
    "Fluency1", "Fluency2", "Fluency3", "Fluency4"
  )],
  na.rm = TRUE
)

# Bilde den Mittelwert zwischen den Kreativitaet Spalten
salienz_reshaped$Mittelwert_Kreativitaet <- rowMeans(
  salienz_reshaped[, c(
    "Kreativitaet1", "Kreativitaet2", "Kreativitaet3",
    "Kreativitaet4", "Kreativitaet5"
  )],
  na.rm = TRUE
)

# Bilde den Mittelwert zwischen den Authentizitaet Spalten
salienz_reshaped$Mittelwert_Authentizitaet <- rowMeans(
  salienz_reshaped[, c(
    "Authentizitaet1", "Authentizitaet2",
    "Authentizitaet3", "Authentizitaet4"
  )],
  na.rm = TRUE
)

# Bilde den Mittelwert zwischen den SophisticatedTaste Spalten
salienz_reshaped$Mittelwert_SophisticatedTaste <- rowMeans(
  salienz_reshaped[, c(
    "SophisticatedTaste1", "SophisticatedTaste2",
    "SophisticatedTaste3", "SophisticatedTaste4"
  )],
  na.rm = TRUE
)

```

Table 21: Cronbachs Alpha – Inkonsistenz

Kennwert	Wert
raw_alpha	0.897735581752036
std.alpha	0.898583234950449
G6(smc)	0.868475894565382
average_r	0.74705535925359
S/N	8.86030267787985
ase	0.00698837749945146
mean	2.58617283950617
sd	1.09442420719219
median_r	0.758760514390928

Table 22: Cronbachs Alpha – Interesse

Kennwert	Wert
raw_alpha	0.87181694783789
std.alpha	0.871820625399641
G6(smc)	0.77276773980066
average_r	0.77276773980066
S/N	6.80156716412315
ase	0.00986735278255968
mean	2.83333333333333
sd	1.06467420240655
median_r	0.77276773980066

Table 23: Cronbachs Alpha – Liking

Kennwert	Wert
raw_alpha	0.952822522226652
std.alpha	0.952822765088518
G6(smc)	0.931414028584868
average_r	0.870671058932723
S/N	20.1966640663934
ase	0.00314933790331425
mean	2.88345679012346
sd	1.11840162306423
median_r	0.870800422944362

Table 24: Cronbachs Alpha – Fluency

Kennwert	Wert
raw_alpha	0.85195719294185
std.alpha	0.853014041740067

Kennwert	Wert
G6(smc)	0.8288581761535
average_r	0.591977084597577
S/N	5.80337096031702
ase	0.00932562047673684
mean	3.53666666666667
sd	0.824022665413839
median_r	0.592660915298466

Table 25: Cronbachs Alpha – Kreativität

Kennwert	Wert
raw_alpha	0.872698318094765
std.alpha	0.873990158486623
G6(smc)	0.902996400432078
average_r	0.581095267991496
S/N	6.93588808612098
ase	0.00829564724654969
mean	2.71822222222222
sd	0.913114040110697
median_r	0.479737459161591

Table 26: Cronbachs Alpha – Authentizität

Kennwert	Wert
raw_alpha	0.836828297120587
std.alpha	0.837323907961533
G6(smc)	0.806954426305463
average_r	0.562706987092627
S/N	5.14718479814191
ase	0.0103492595698977
mean	3.19222222222222
sd	0.845104304719594
median_r	0.566855120863933

Table 27: Cronbachs Alpha – Sophisticated Taste

Kennwert	Wert
raw_alpha	0.85680443193527
std.alpha	0.856002486024091
G6(smc)	0.829400157324664
average_r	0.597770212456381
S/N	5.94456433579333
ase	0.00897695025948791

Kennwert	Wert
mean	3.29962962962963
sd	0.893711468557812
median_r	0.584340164277278

Table 28: Cronbachs Alpha – CVPA

Kennwert	Wert
raw_alpha	0.84461866643209
std.alpha	0.847875838867972
G6(smc)	0.868983005704661
average_r	0.382444025903621
S/N	5.57357774438016
ase	0.00913113685476245
mean	3.34320987654321
sd	0.745167561298037
median_r	0.341186150182593

Table 29: Cronbachs Alpha nach Outfit und Konstrukt

Outfit	Konstrukt	Cronbach_Alpha	Interpretation
GDS	WSI	0.853	Gut
GDS	Interesse	0.872	Gut
GDS	Liking	0.946	Sehr gut
GDS	Fluency	0.871	Gut
GDS	Kreativitaet	0.879	Gut
GDS	Authentizitaet	0.822	Gut
GDS	SophisticatedTaste	0.837	Gut
GDL	WSI	0.837	Gut
GDL	Interesse	0.885	Gut
GDL	Liking	0.949	Sehr gut
GDL	Fluency	0.869	Gut
GDL	Kreativitaet	0.850	Gut
GDL	Authentizitaet	0.803	Gut
GDL	SophisticatedTaste	0.841	Gut
KDS	WSI	0.839	Gut
KDS	Interesse	0.876	Gut
KDS	Liking	0.943	Sehr gut
KDS	Fluency	0.829	Gut
KDS	Kreativitaet	0.854	Gut
KDS	Authentizitaet	0.816	Gut
KDS	SophisticatedTaste	0.798	Akzeptabel
GZS	WSI	0.854	Gut
GZS	Interesse	0.824	Gut
GZS	Liking	0.949	Sehr gut

Outfit	Konstrukt	Cronbach_Alpha	Interpretation
GZS	Fluency	0.846	Gut
GZS	Kreativitaet	0.897	Gut
GZS	Authentizitaet	0.807	Gut
GZS	SophisticatedTaste	0.818	Gut
M	WSI	0.800	Akzeptabel
M	Interesse	0.893	Gut
M	Liking	0.948	Sehr gut
M	Fluency	0.820	Gut
M	Kreativitaet	0.851	Gut
M	Authentizitaet	0.864	Gut
M	SophisticatedTaste	0.860	Gut

4 Explorative Bewertung der fünf Outfits

```
# Boxplots: Bewertung der fünf Outfits explorativ vergleichen

# Mittelwerte wieder ins Long-Format bringen um alle Konstrukte zusammen zu plotten
bewertungen_long <- salienz_reshaped %>%
  pivot_longer(
    cols = c(
      Mittelwert_Inkonsistenz,
      Mittelwert_Interesse,
      Mittelwert_Liking,
      Mittelwert_Fluency,
      Mittelwert_Kreativitaet,
      Mittelwert_Authentizitaet,
      Mittelwert_SophisticatedTaste
    ), names_to = "Variable", values_to = "Wert")

bewertungen_long <- bewertungen_long %>%
  mutate(
    Variable = dplyr::recode(
      Variable,
      "Mittelwert_Inkonsistenz" = "WSI",
      "Mittelwert_Interesse" = "Interesse",
      "Mittelwert_Liking" = "Liking",
      "Mittelwert_Fluency" = "Fluency",
      "Mittelwert_Kreativitaet" = "Kreativität",
      "Mittelwert_Authentizitaet" = "Authentizität",
      "Mittelwert_SophisticatedTaste" = "Sophisticated Taste"))

# Festlegen der Reihenfolge der Outfits
bewertungen_long$Condition <- factor(
  bewertungen_long$Condition,
  levels = c(
    "Min-Baseline",
    "Jacke-leicht-dezentral",
    "Jacke-stark-dezentral",
    "Jacke-stark-zentral",
    "Schuh-stark-dezentral"))

# Alle Boxplots in eine Abbildung
boxplots_outfits <- ggplot(
  bewertungen_long,
```

```

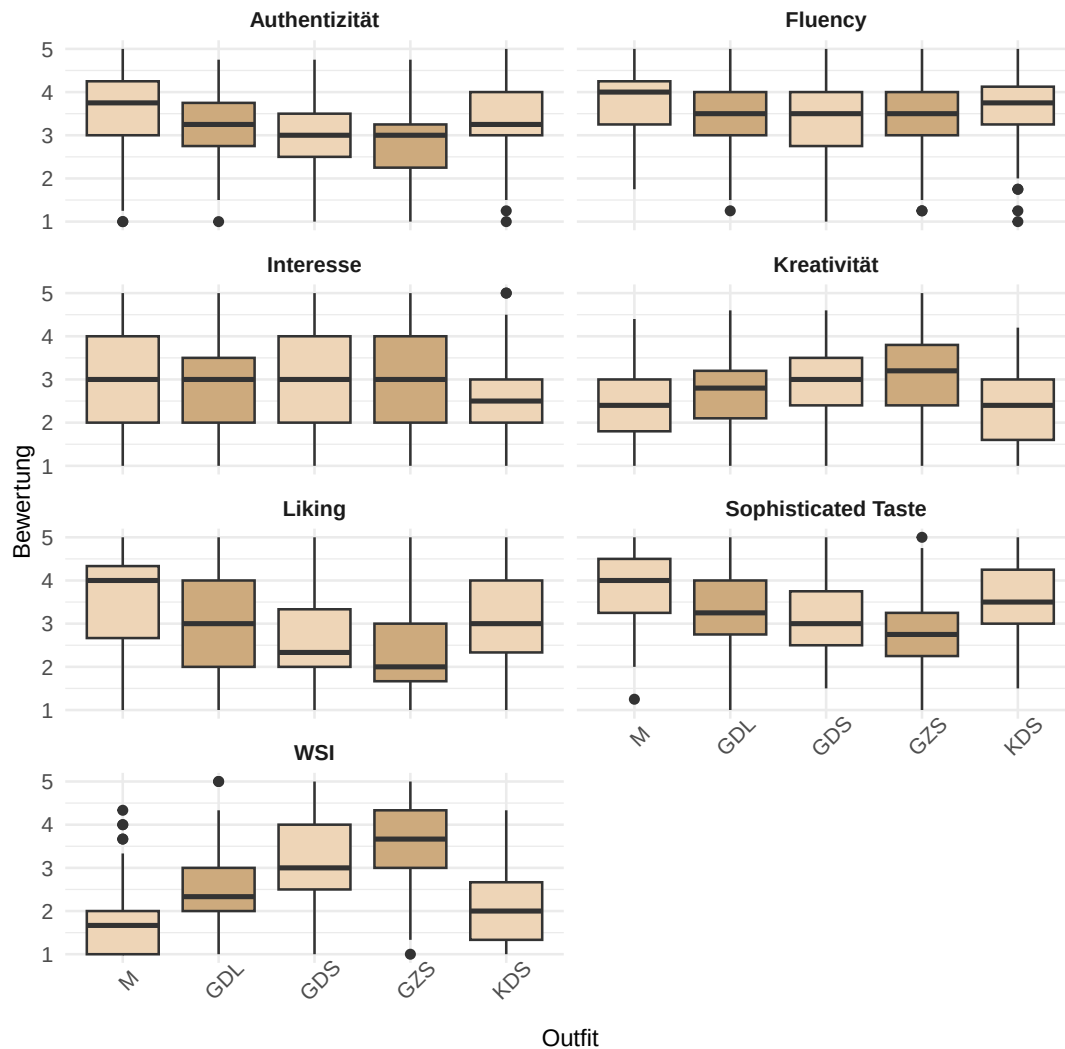
aes(x = Condition, y = Wert, fill = Condition)) +
geom_boxplot() +
facet_wrap(~ Variable, ncol = 2) +
scale_fill_manual(
  values = c(
    "Min-Baseline" = "#EED5B7",
    "Jacke-leicht-dezentral" = "#CDAA7D",
    "Jacke-stark-dezentral" = "#EED5B7",
    "Jacke-stark-zentral" = "#CDAA7D",
    "Schuh-stark-dezentral" = "#EED5B7")) +
scale_y_continuous(breaks = 1:5, limits = c(1, 5)) +
scale_x_discrete(labels = bedingung_labels) +
labs(
  title = "Bewertung der fünf Outfits (WSI und weitere Konstrukte)",
  subtitle = "Verteilung der Skalenmittelwerte nach Outfit",
  x = "Outfit",
  y = "Bewertung") +
theme_box_density_gross +
theme(
  legend.position = "none",
  plot.title = element_text(
    hjust = 0.5),
  plot.subtitle = element_text(
    hjust = 0.5),
  axis.text.x = element_text(
    angle = 45,
    hjust = 0.5))

# Boxplots anzeigen
print(boxplots_outfits)

```

Bewertung der fünf Outfits (WSI und weitere Konstrukte)

Verteilung der Skalenmittelwerte nach Outfit



```
# Boxplots als PNG speichern
ggsave(
  filename = "Boxplots_Bewertung_aller_Outfits.png",
  plot = boxplots_outfits,
  width = 8,
  height = 7,
  dpi = 300
)

# Datenset schreiben
write.csv(
  salienz_reshaped,
  "pretest_salienz-reshaped.csv",
  row.names = FALSE
)
```


5 Hauptanalyse: Wahrgenommene stilistische Inkonsistenz (WSI)

5.1 Hilfsfunktionen für die Hauptanalysen

```
# Data Analysis
# Statistische Tests für Inkonsistenz

# Daten für Conditions zum Plotten vorbereiten
prepare_plot_data <- function(conditions, levels, labels) {

  salienz_reshaped %>%
    filter(Condition %in% conditions) %>%
    drop_na(Mittelwert_Inkonsistenz) %>%
    mutate(
      Condition = factor(
        Condition,
        levels = levels,
        labels = labels
      )
    )
}

# Daten für zwei Conditions für gepaarte Tests vorbereiten
prepare_paired_data <- function(condition1, condition2) {

  salienz_reshaped %>%
    filter(Condition %in% c(condition1, condition2)) %>%
    dplyr::select(number, Condition, Mittelwert_Inkonsistenz) %>%
    pivot_wider(
      names_from = Condition,
      values_from = Mittelwert_Inkonsistenz
    ) %>%
    drop_na(all_of(c(condition1, condition2)))
}

# Box und Density Plots erstellen
plot_box_density <- function(data, farben, subtitle) {

  # Boxplot
  boxplot <- ggplot(
    data,
    aes(
      x = Condition,
      y = Mittelwert_Inkonsistenz,
      fill = Condition
    )
  ) +
  geom_boxplot() +
  scale_fill_manual(
    values = rep(
      c("#EED5B7", "#CDAA7D"),
      length.out = length(unique(data$Condition))
    )
  ) +
  labs(
    title = stringr::str_wrap(paste("Boxplot der WSI:", subtitle), width = 48),
    x = "Bedingung",
    y = "WSI"
  ) +
  theme_box_density_gross +
  theme(
```

```

    legend.position = "none",
    plot.title = element_text(hjust = 0.5)
  )

speichere_grafik(
  boxplot,
  paste0("Boxplot_Inkonsistenz_", subtitle)
)
print(boxplot)

# Density Plot
density_plot <- ggplot(
  data,
  aes(
    x = Mittelwert_Inkonsistenz,
    color = Condition
  )
) +
  geom_density(linewidth = 1) +
  scale_color_manual(values = farben) +
  labs(
    title = stringr::str_wrap(paste("Dichteplot der WSI:", subtitle), width = 48),
    x = "WSI",
    y = "Dichte",
    color = "Bedingung"
  ) +
  theme_box_density_gross +
  theme(
    legend.position = "bottom",
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  )

speichere_grafik(
  density_plot,
  paste0("Density_Inkonsistenz_", subtitle)
)
print(density_plot)
}

# Deskriptive Kennwerte erstellen
deskriptive_kennwerte <- function(data) {

  data %>%
    group_by(Condition) %>%
    summarise(
      n = n(),
      Median = median(Mittelwert_Inkonsistenz, na.rm = TRUE),
      IQR = IQR(Mittelwert_Inkonsistenz, na.rm = TRUE),
      Mittelwert = mean(Mittelwert_Inkonsistenz, na.rm = TRUE),
      SD = sd(Mittelwert_Inkonsistenz, na.rm = TRUE)
    )
}

# Grafische Normalverteilungsdarstellung nur zur Beschreibung.
# WICHTIG: Die Grafiken werden NICHT für die Testwahl berücksichtigt.
# Die Testwahl erfolgt ausschließlich anhand der p-Werte der formalen Annahmentests.
plot_differenz <- function(data, subtitle) {

  # Density Plot

```

```

density_plot <- ggplot(
  data,
  aes(x = Differenz_Inkonsistenz)
) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",
    alpha = 0.4,
    linewidth = 1
  ) +
  labs(
    title = stringr::str_wrap(paste("Dichteplot der Differenzwerte:", subtitle), width = 48),
    x = "Differenz der WSI",
    y = "Dichte"
  ) +
  theme_box_density_gross +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  )

speichere_grafik(
  density_plot,
  paste0("Density_Differenzwerte_", subtitle)
)
print(density_plot)
}

```

5.2 Produktgröße: GDS vs. KDS

```

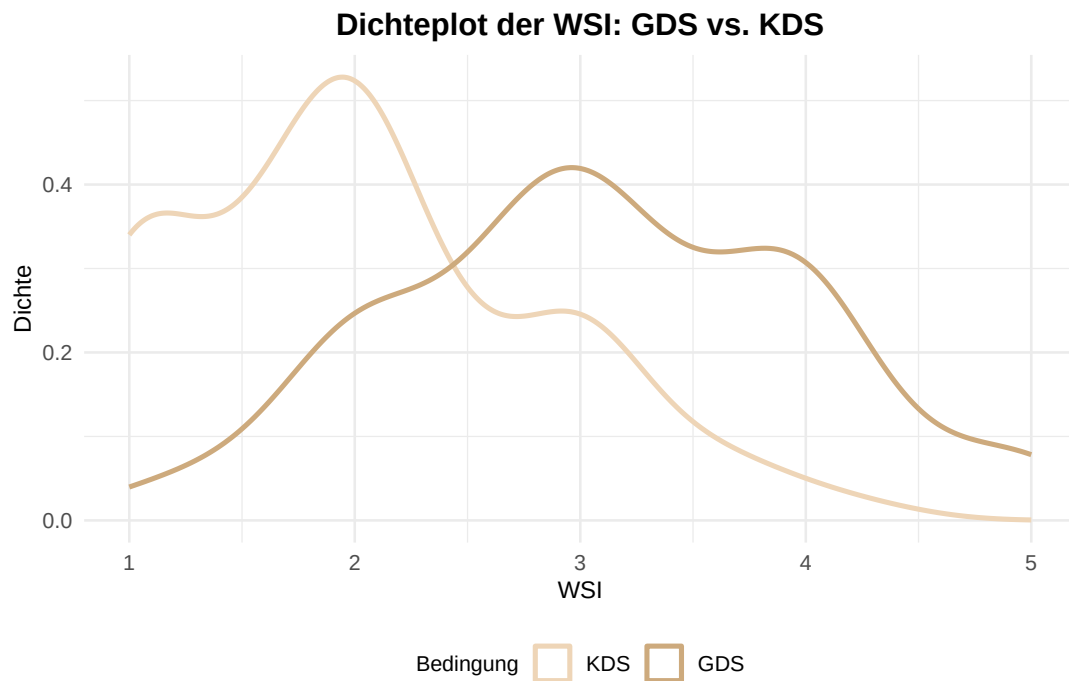
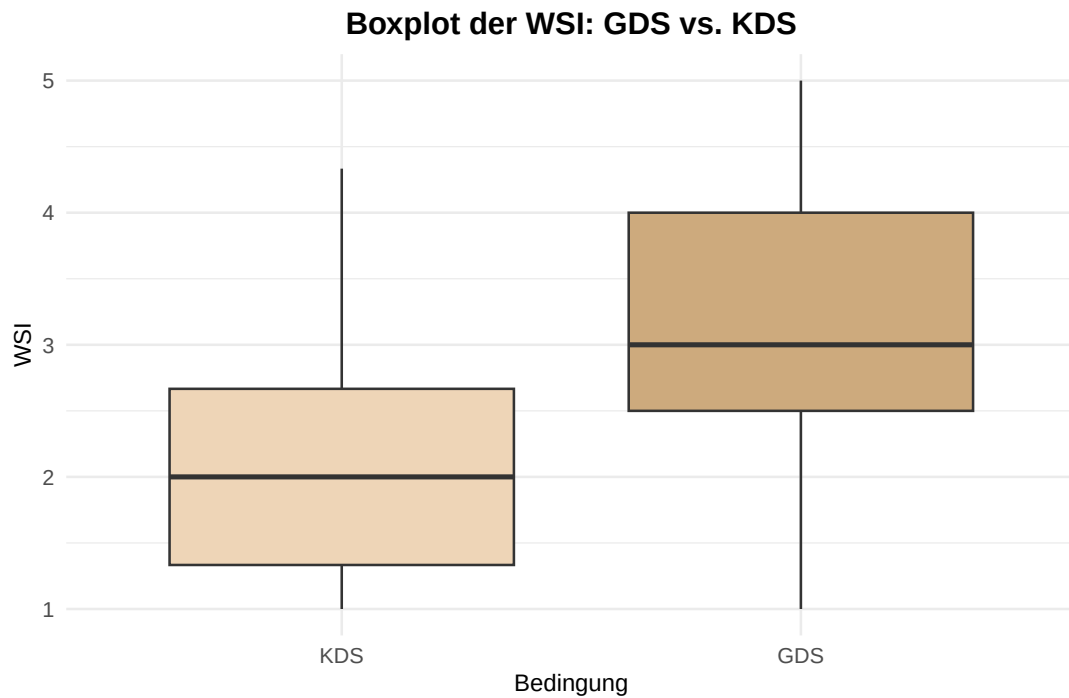
# Modell Inkonsistenz: Jacke-stark-dezentral vs Schuh-stark-dezentral
# DV: Stilistische Inkonsistenz
# IV: Conditions/ Outfits (Groß vs klein -> Kategorisch 2 Ausprägungen)
# Within-Subjects daher dependent sample
# -> Dependent/Gepaarter-t-Test

# Daten für die beiden Conditions auswählen
plot_groesse <- prepare_plot_data(
  conditions = c(
    "Jacke-stark-dezentral",
    "Schuh-stark-dezentral"
  ),
  levels = c(
    "Schuh-stark-dezentral",
    "Jacke-stark-dezentral"
  ),
  labels = c(
    "KDS",
    "GDS"
  )
)

# Box und Density erstellen
plot_box_density(
  plot_groesse,
  farben = c(
    "KDS" = "#EED5B7",
    "GDS" = "#CDAA7D"
  )
)

```

```
),
  subtitle = "GDS vs. KDS"
)
```



```
# Deskriptive Kennwerte
deskriptiv_groesse <- deskriptive_kennwerte(plot_groesse)
deskriptiv_groesse

# Daten für die beiden Conditions auswählen
inkonsistenz_groesse <- prepare_paired_data(
  "Jacke-stark-dezentral",
  "Schuh-stark-dezentral"
```

```

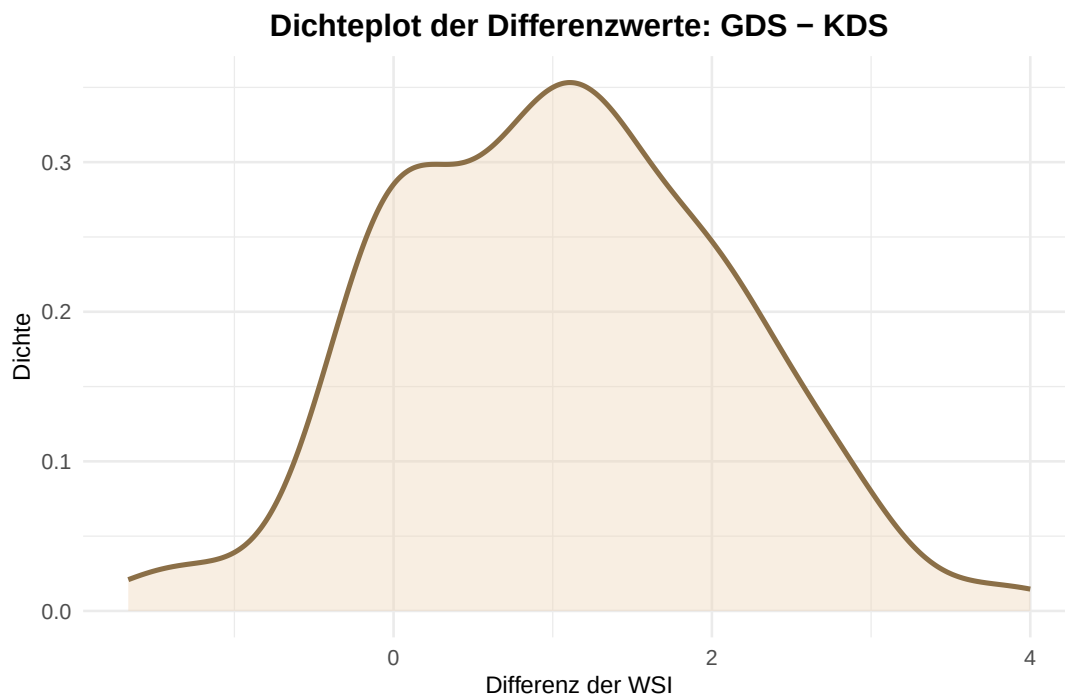
)

# Differenzwert bilden:
# positive Werte = Jacke wurde inkonsistenter bewertet als Schuh
inkonsistenz_groesse <- inkonsistenz_groesse %>%
  mutate(
    Differenz_Inkonsistenz =
      `Jacke-stark-dezentral` -
      `Schuh-stark-dezentral`
  )

# Anzahl vollständiger Paare kontrollieren
nrow(inkonsistenz_groesse)

# Normalverteilung grafisch darstellen (nur deskriptiv; nicht für die Testwahl)
plot_differenz(
  inkonsistenz_groesse,
  "GDS - KDS"
)

```



```

# Normalverteilung mit Shapiro-Wilk prüfen
# H0: Die Differenzwerte sind normalverteilt
# H1: Die Differenzwerte sind nicht normalverteilt
shapiro_auto_01 <- shapiro.test(
  inkonsistenz_groesse$Differenz_Inkonsistenz
)
shapiro_auto_01

if (shapiro_auto_01$p.value > 0.05) {
  t_test_groesse <- t.test(
    inkonsistenz_groesse$`Jacke-stark-dezentral`,
    inkonsistenz_groesse$`Schuh-stark-dezentral`,
    paired = TRUE,
    alternative = "greater"
  )
}

```

```

t_test_groesse
} else {
  wilcoxon_groesse <- wilcox.test(
    inkonsistenz_groesse$`Jacke-stark-dezentral`,
    inkonsistenz_groesse$`Schuh-stark-dezentral`,
    paired = TRUE,
    alternative = "greater",
    exact = FALSE
  )
  wilcoxon_groesse
}

```

Table 30: Deskriptive Kennwerte – Produktgröße - Teil 1 von 2

Condition	n	Median	IQR	Mittelwert
KDS	135	2	1.333	2.044
GDS	135	3	1.500	3.101

Table 31: Deskriptive Kennwerte – Produktgröße - Teil 2 von 2

Condition	SD
KDS	0.800
GDS	0.915

Table 32: Shapiro-Wilk-Test der Differenzwerte – Produktgröße

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.985
p-Wert	0.1427

Table 33: Gerichteter gepaarter t-Test – Produktgröße

Kennwert	Wert
Methode	Paired t-test
t	11.705
df	134
mean difference	1.057
Konfidenzintervall	0.907, Inf
p-Wert	< 1e-04
Alternative	greater

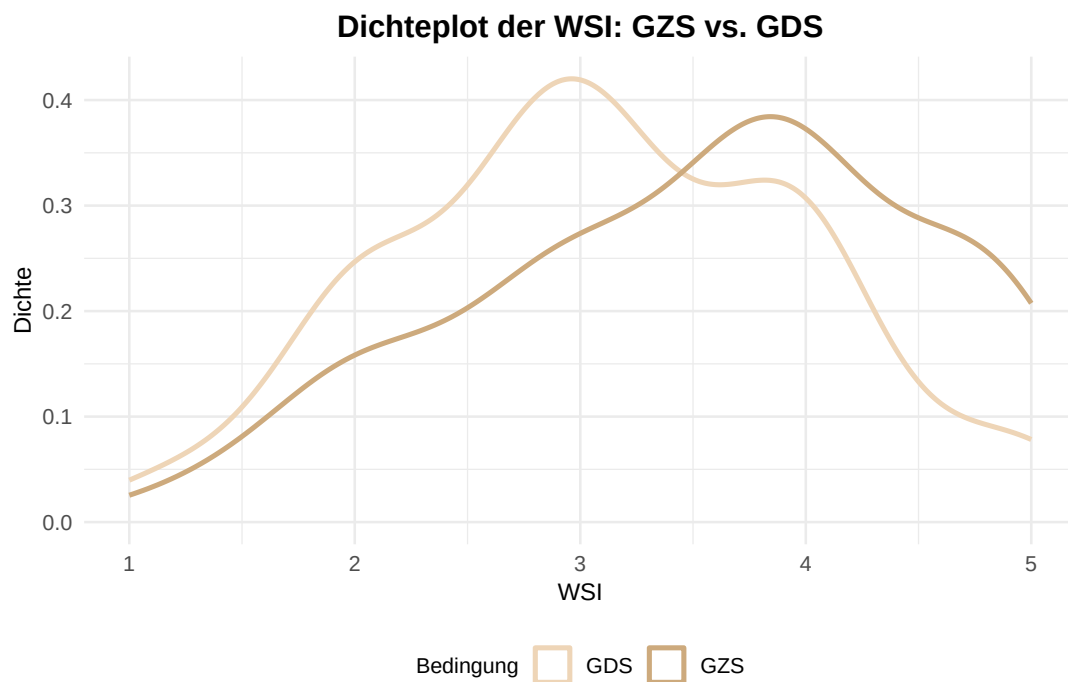
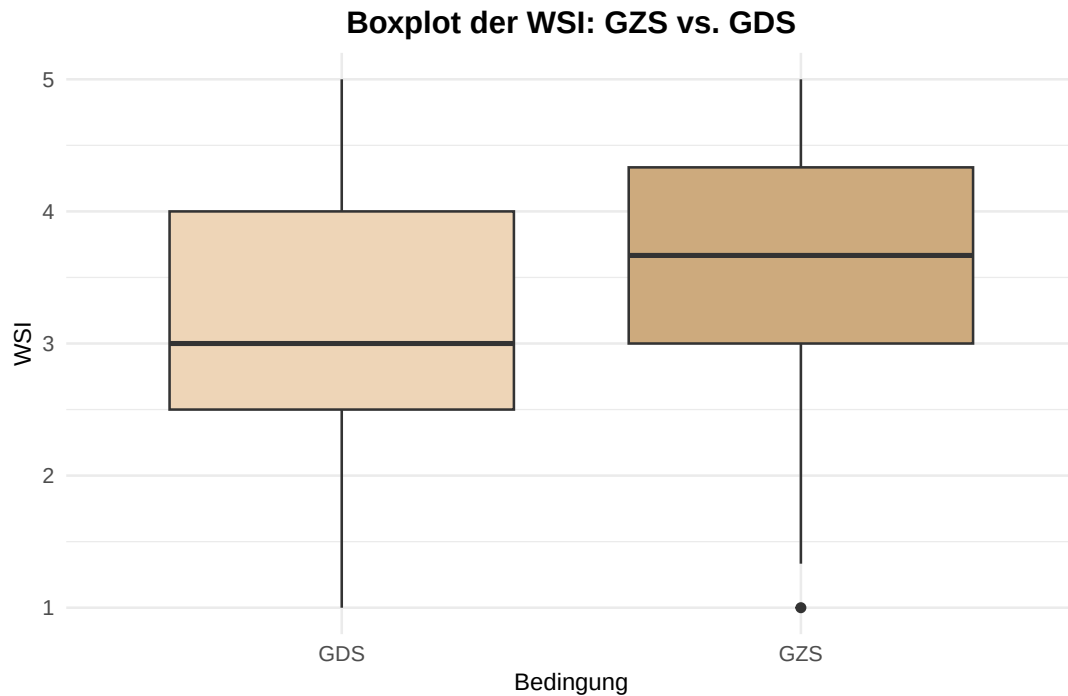
5.3 Position: GZS vs. GDS

```
# Modell Inkonsistenz: Jacke-stark-zentral vs Jacke-stark-dezentral
# DV: Stilistische Inkonsistenz
# IV: Conditions/ Outfits (Zentral vs dezentral -> Kategorisch 2 Ausprägungen)
# Within-Subjects daher dependent sample
# #####
# -> Dependent-t-Test

# Daten für die beiden Conditions auswählen
plot_position <- prepare_plot_data(
  conditions = c(
    "Jacke-stark-zentral",
    "Jacke-stark-dezentral"
  ),
  levels = c(
    "Jacke-stark-dezentral",
    "Jacke-stark-zentral"
  ),
  labels = c(
    "GDS",
    "GZS"
  )
)

inkonsistenz_position <- prepare_paired_data(
  "Jacke-stark-zentral",
  "Jacke-stark-dezentral"
)

# Box und Density Plots erstellen
plot_box_density(
  plot_position,
  farben = c(
    "GDS" = "#EED5B7",
    "GZS" = "#CDAA7D"
  ),
  subtitle = "GZS vs. GDS"
)
```



```
# Deskriptive Kennwerte erstellen
deskriptiv_position <- deskriptive_kennwerte(plot_position)
deskriptiv_position

# Differenzwert bilden:
# positive Werte = zentrale Jacke wurde inkonsistenter bewertet
# als dezentrale Jacke
inkonsistenz_position <- inkonsistenz_position %>%
  mutate(
    Differenz_Inkonsistenz =
      `Jacke-stark-zentral` -
      `Jacke-stark-dezentral`
  )
```



```

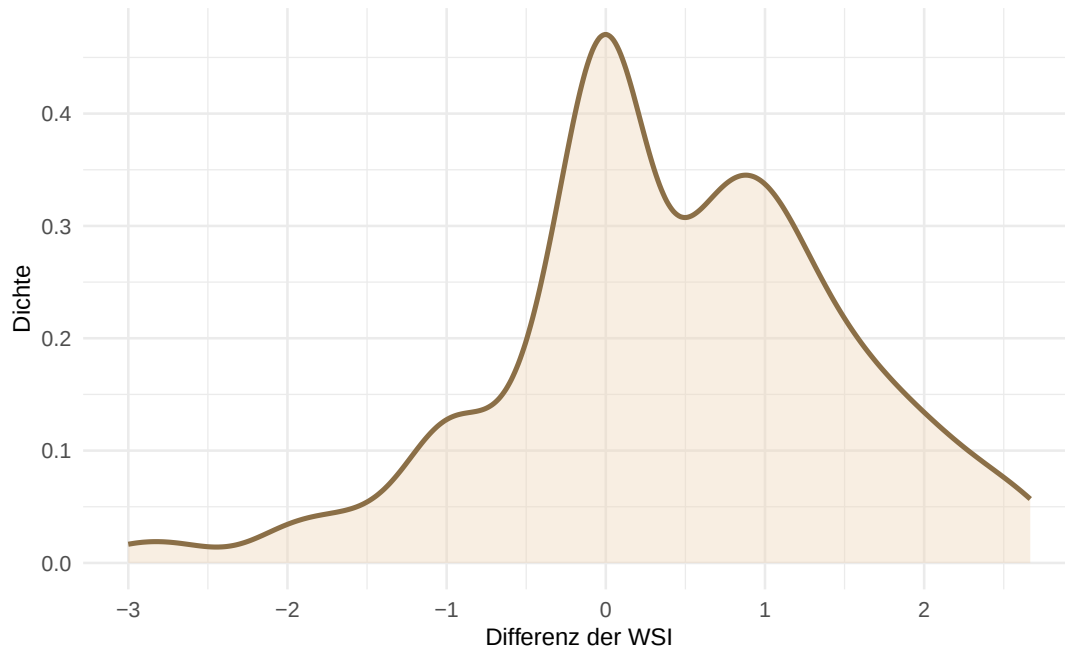
)

# Anzahl vollständiger Paare kontrollieren
nrow(inkonsistenz_position)

# Normalverteilung grafisch darstellen (nur deskriptiv; nicht für die Testwahl)
plot_differenz(
  inkonsistenz_position,
  "GZS - GDS"
)

```

Dichteplot der Differenzwerte: GZS – GDS



```

# Normalverteilung mit Shapiro-Wilk prüfen
# H0: Die Differenzwerte sind normalverteilt
# H1: Die Differenzwerte sind nicht normalverteilt
shapiro_auto_02 <- shapiro.test(
  inkonsistenz_position$Differenz_Inkonsistenz
)
shapiro_auto_02

if (shapiro_auto_02$p.value > 0.05) {
  t_test_position <- t.test(
    inkonsistenz_position$`Jacke-stark-zentral`,
    inkonsistenz_position$`Jacke-stark-dezentral`,
    paired = TRUE,
    alternative = "greater"
  )
  t_test_position
} else {
  wilcoxon_position <- wilcox.test(
    inkonsistenz_position$`Jacke-stark-zentral`,
    inkonsistenz_position$`Jacke-stark-dezentral`,
    paired = TRUE,
    alternative = "greater",
    exact = FALSE
  )
}

```

```
wilcoxon_position
}
```

Table 34: Deskriptive Kennwerte – Position - Teil 1 von 2

Condition	n	Median	IQR	Mittelwert
GDS	135	3.000	1.500	3.101
GZS	135	3.667	1.333	3.531

Table 35: Deskriptive Kennwerte – Position - Teil 2 von 2

Condition	SD
GDS	0.915
GZS	0.992

Table 36: Shapiro-Wilk-Test der Differenzwerte – Position

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.973
p-Wert	0.009516

Table 37: Gerichteter Wilcoxon-Vorzeichen-Rang-Test – Position

Kennwert	Wert
Methode	Wilcoxon signed rank test with continuity correction
V	4053
p-Wert	< 1e-04
Alternative	greater

5.4 Stärke (2 Stufen): GDS vs. GDL

```
# Modell Inkonsistenz: Jacke-stark-dezentral vs Jacke-schwach-dezentral
# DV: Stilistische Inkonsistenz
# IV: Conditions/ Outfits (Stark vs schwach -> Kategorisch 2 Ausprägungen)
# Within-Subjects daher dependent sample
# -> Dependent-t-Test

# Daten für die beiden Conditions auswählen
plot_staerke <- prepare_plot_data(
  conditions = c(
    "Jacke-stark-dezentral",
    "Jacke-leicht-dezentral"
  ),
  levels = c(
```

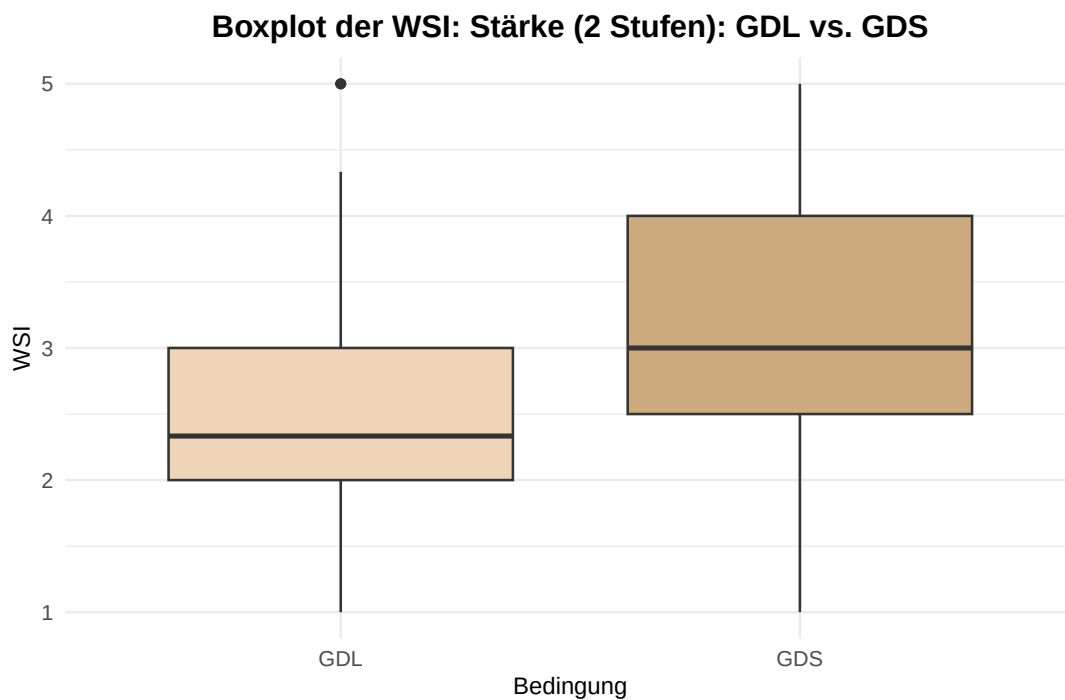
```

    "Jacke-leicht-dezentral",
    "Jacke-stark-dezentral"
  ),
  labels = c(
    "GDL",
    "GDS"
  )
)

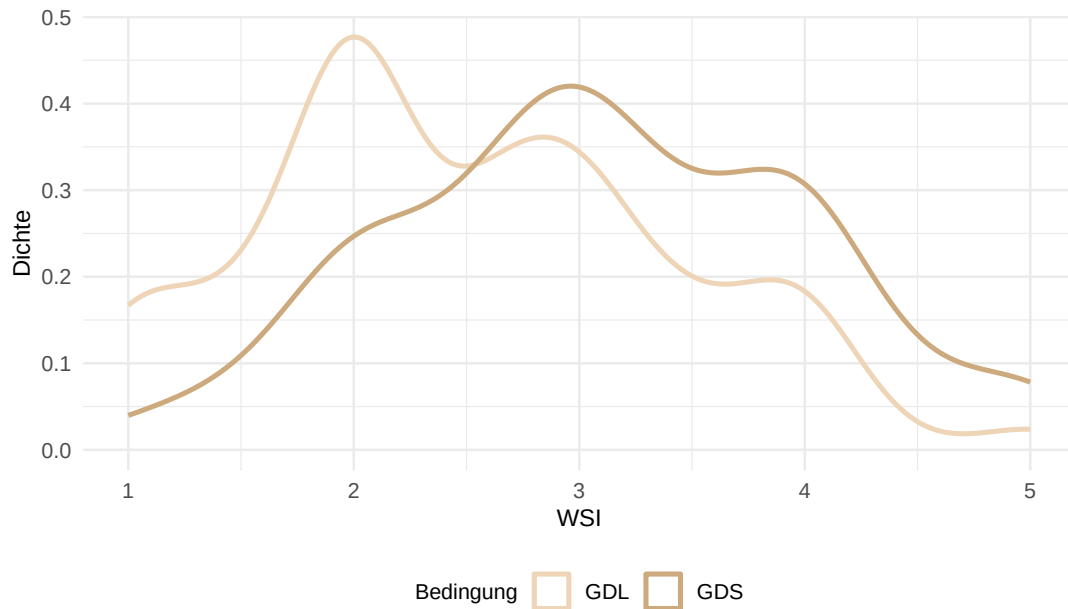
inkonsistenz_staerke <- prepare_paired_data(
  "Jacke-stark-dezentral",
  "Jacke-leicht-dezentral"
)

# Box und Density Plots erstellen
plot_box_density(
  plot_staerke,
  farben = c(
    "GDL" = "#EED5B7",
    "GDS" = "#CDAA7D"
  ),
  subtitle = "Stärke (2 Stufen): GDL vs. GDS"
)

```



Dichteplot der WSI: Stärke (2 Stufen): GDL vs. GDS



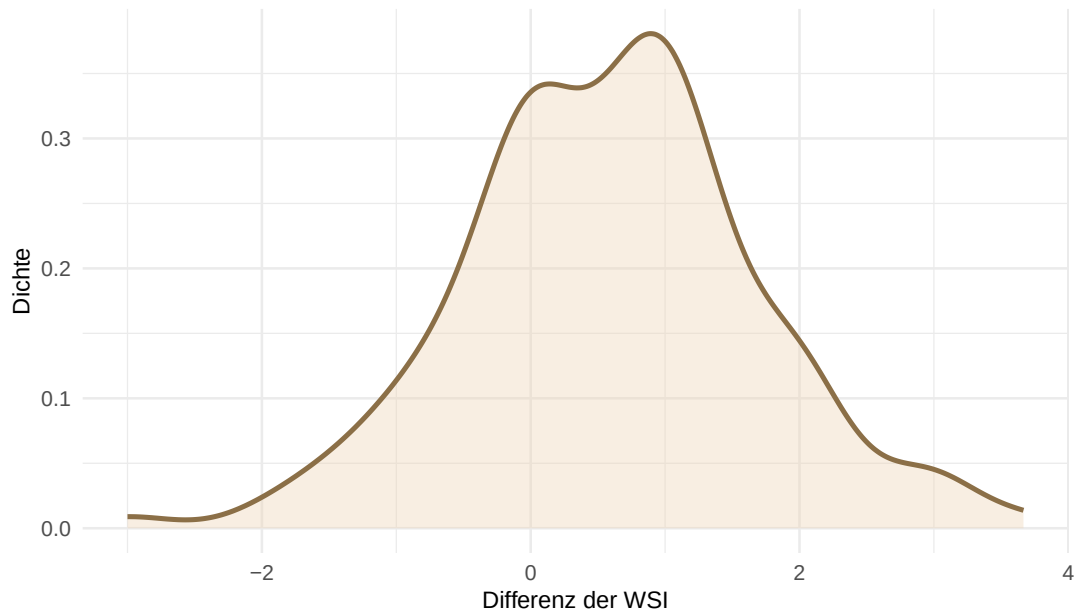
```
# Deskriptive Kennwerte erstellen
deskriptiv_staerke_zwei <- deskriptive_kennwerte(plot_staerke)
deskriptiv_staerke_zwei

# Differenzwert bilden:
# positive Werte = stark abweichende Jacke wurde inkonsistenter bewertet
# als leicht abweichende Jacke
inkonsistenz_staerke <- inkonsistenz_staerke %>%
  mutate(
    Differenz_Inkonsistenz =
      `Jacke-stark-dezentral` -
      `Jacke-leicht-dezentral`
  )

# Anzahl vollständiger Paare kontrollieren
nrow(inkonsistenz_staerke)

# Normalverteilung grafisch darstellen (nur deskriptiv; nicht für die Testwahl)
plot_differenz(
  inkonsistenz_staerke,
  "Stärke (2 Stufen): GDS - GDL"
)
```

Dichteplot der Differenzwerte: Stärke (2 Stufen): GDS – GDL



```
# Normalverteilung mit Shapiro-Wilk prüfen
# H0: Die Differenzwerte sind normalverteilt
# H1: Die Differenzwerte sind nicht normalverteilt
shapiro_auto_03 <- shapiro.test(
  inkonsistenz_staerke$Differenz_Inkonsistenz
)
shapiro_auto_03

if (shapiro_auto_03$p.value > 0.05) {
  t_test_staerke <- t.test(
    inkonsistenz_staerke$`Jacke-stark-dezentral`,
    inkonsistenz_staerke$`Jacke-leicht-dezentral`,
    paired = TRUE,
    alternative = "greater"
  )
  t_test_staerke
} else {
  wilcoxon_staerke <- wilcox.test(
    inkonsistenz_staerke$`Jacke-stark-dezentral`,
    inkonsistenz_staerke$`Jacke-leicht-dezentral`,
    paired = TRUE,
    alternative = "greater",
    exact = FALSE
  )
  wilcoxon_staerke
}
```

Table 38: Deskriptive Kennwerte – Stärke (2 Stufen) - Teil 1 von 2

Condition	n	Median	IQR	Mittelwert
GDL	135	2.333	1.0	2.514
GDS	135	3.000	1.5	3.101

Table 39: Deskriptive Kennwerte – Stärke (2 Stufen) - Teil 2 von 2

Condition	SD
GDL	0.919
GDS	0.915

Table 40: Shapiro-Wilk-Test der Differenzwerte – Stärke (2 Stufen)

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.983
p-Wert	0.0954

Table 41: Gerichteter gepaarter t-Test – Stärke (2 Stufen)

Kennwert	Wert
Methode	Paired t-test
t	6.243
df	134
mean difference	0.588
Konfidenzintervall	0.432, Inf
p-Wert	< 1e-04
Alternative	greater

5.5 Stärke (3 Stufen): M vs. GDL vs. GDS

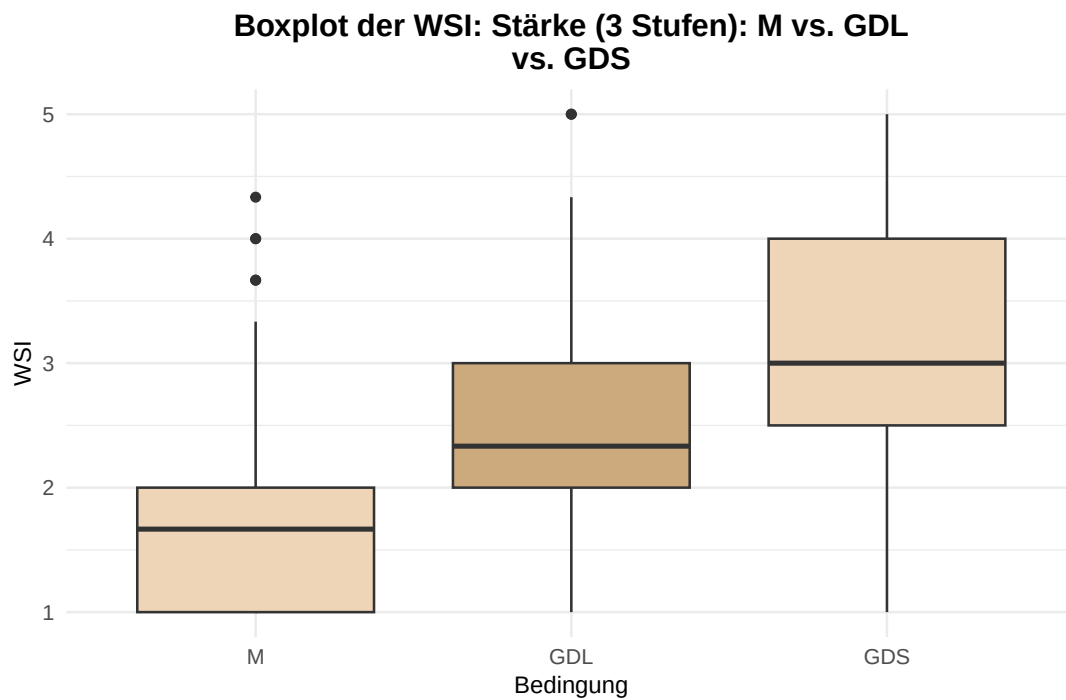
```
# Zusätzliches Modell Inkonsistenz: Min Jacke vs schwach Jacke vs stark Jacke
# DV: Stilistische Inkonsistenz
# IV: Conditions/ Outfits (Stark vs schwach vs gar nicht -> Kategorisch 3 Ausprägungen)
# Within-Subjects daher dependent sample
# -> One-way repeated measures ANOVA
# die drei Conditions auswählen
inkonsistenz_anova_staerke <- prepare_plot_data(
  conditions = c(
    "Min-Baseline",
    "Jacke-leicht-dezentral",
    "Jacke-stark-dezentral"
  ),
  levels = c(
    "Min-Baseline",
    "Jacke-leicht-dezentral",
    "Jacke-stark-dezentral"
  ),
  labels = c(
    "M",
    "GDL",
    "GDS"
  )
)
```

```

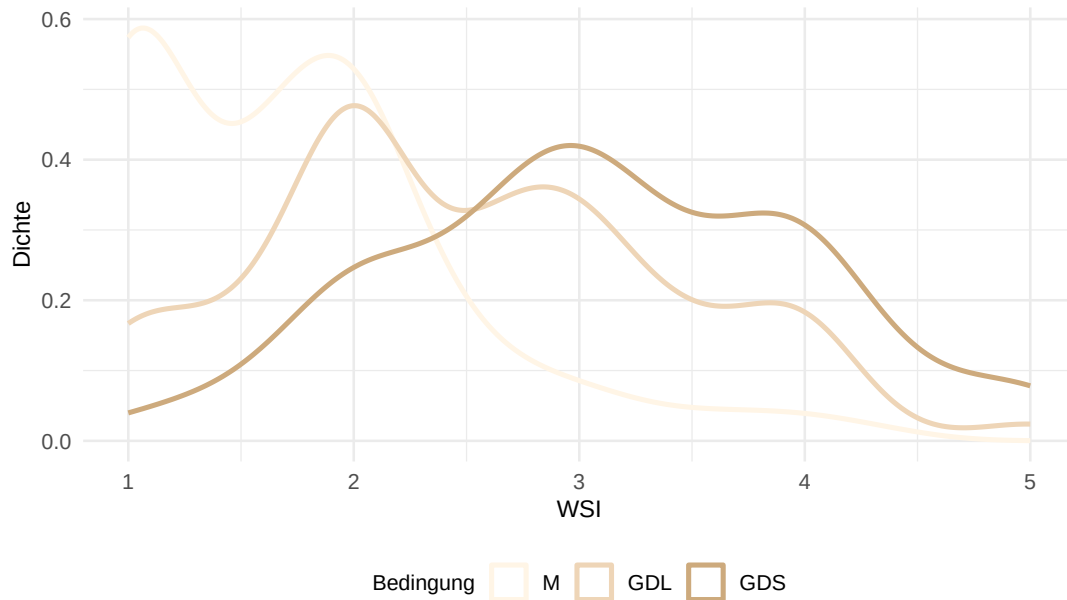
)
)

# Box und Density Plots erstellen
plot_box_density(
  inkonsistenz_anova_staerke,
  farben = c(
    "M" = "#FFF5E6",
    "GDL" = "#EED5B7",
    "GDS" = "#CDAA7D"
  ),
  subtitle = "Stärke (3 Stufen): M vs. GDL vs. GDS"
)

```



Dichteplot der WSI: Stärke (3 Stufen): M vs. GDL vs. GDS



```
# Deskriptive Kennwerte erstellen
deskriptiv_staerke <- deskriptive_kennwerte(
  inkonsistenz_anova_staerke
)
deskriptiv_staerke

# Repeated-Measures-ANOVA zunächst schätzen,
# damit anschließend die Residuen geprüft werden können
anova_staerke <- aov_ez(
  id = "number",
  dv = "Mittelwert_Inkonsistenz",
  within = "Condition",
  data = inkonsistenz_anova_staerke
)

# Residuen extrahieren
residuen_staerke <- residuals(anova_staerke$lm)

zeige_und_speichere_grafik(
  ggplot(
    data.frame(Residuen = as.numeric(residuen_staerke)),
    aes(x = Residuen)
  ) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",
    alpha = 0.4,
    linewidth = 1
  ) +
  labs(
    title = stringr::str_wrap(
      "Dichteplot der Residuen: Repeated-Measures-ANOVA Stärke (3 Stufen)",
      width = 52),
    x = "Residuen",
    y = "Dichte"
  ) +

```

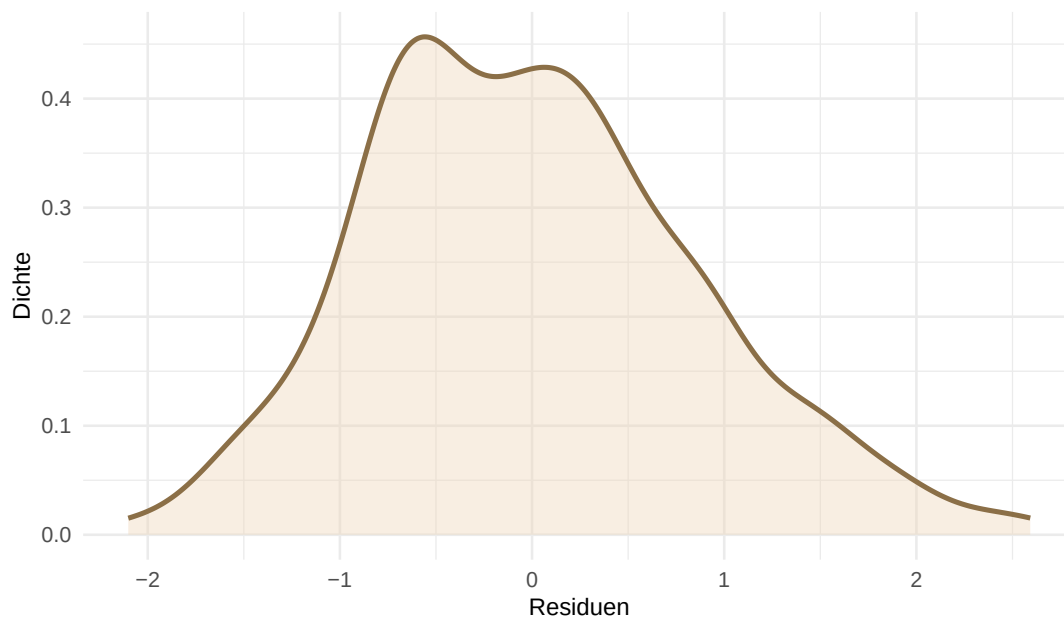


```

theme_box_density_gross +
theme(
  plot.title = element_text(hjust = 0.5),
  plot.subtitle = element_text(hjust = 0.5)
),
"Density Plot der Residuen - Stärke"
)

```

Dichteplot der Residuen: Repeated-Measures-ANOVA Stärke (3 Stufen)



```

# Normalverteilung mit Shapiro-Wilk prüfen
shapiro_auto_04 <- shapiro.test(residuen_staerke)
shapiro_auto_04

if (shapiro_auto_04$p.value > 0.05) {
  # Normalverteilung nicht signifikant verletzt -> Repeated-Measures-ANOVA
  anova_staerke
  paarweise_staerke <- pairs(
    emmeans(anova_staerke, ~ Condition),
    adjust = "holm"
  )
  paarweise_staerke
} else {
  # Signifikante Abweichung von der Normalverteilung -> Friedman-Test
  friedman_staerke <- friedman.test(
    Mittelwert_Inkonsistenz ~ Condition | number,
    data = inkonsistenz_anova_staerke
  )
  friedman_staerke

  # Paarweise Wilcoxon-Vorzeichen-Rang-Tests mit Holm-Korrektur
  paarweise_staerke <- inkonsistenz_anova_staerke %>%
    pairwise_wilcox_test(
      Mittelwert_Inkonsistenz ~ Condition,
      paired = TRUE,
      p.adjust.method = "holm"
    )
}

```

```

    paarweise_staerke
}

```

Table 42: Deskriptive Kennwerte – Stärke (3 Stufen) - Teil 1 von 2

Condition	n	Median	IQR	Mittelwert
M	135	1.667	1.0	1.741
GDL	135	2.333	1.0	2.514
GDS	135	3.000	1.5	3.101

Table 43: Deskriptive Kennwerte – Stärke (3 Stufen) - Teil 2 von 2

Condition	SD
M	0.729
GDL	0.919
GDS	0.915

Table 44: Shapiro-Wilk-Test der ANOVA-Residuen – Stärke (3 Stufen)

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.982
p-Wert	< 1e-04

Table 45: Friedman-Test – Stärke (3 Stufen)

Kennwert	Wert
Methode	Friedman rank sum test
Friedman chi-squared	139.453
df	2
p-Wert	< 1e-04

Table 46: Paarweise Wilcoxon-Tests mit Holm-Korrektur - Teil 1 von 3

.y.	group1	group2	n1	n2
Mittelwert_Inkonsistenz	M	GDL	135	135
Mittelwert_Inkonsistenz	M	GDS	135	135
Mittelwert_Inkonsistenz	GDL	GDS	135	135

Table 47: Paarweise Wilcoxon-Tests mit Holm-Korrektur -
Teil 2 von 3

.y.	group1	statistic	p	p.adj
Mittelwert_Inkonsistenz	M	598.5	0	0
Mittelwert_Inkonsistenz	M	112.0	0	0
Mittelwert_Inkonsistenz	GDL	1212.5	0	0

Table 48: Paarweise Wilcoxon-Tests mit Holm-Korrektur -
Teil 3 von 3

.y.	group1	p.adj.signif
Mittelwert_Inkonsistenz	M	****
Mittelwert_Inkonsistenz	M	****
Mittelwert_Inkonsistenz	GDL	****

5.6 Schuh-Stärke (2 Stufen): M vs. KDS

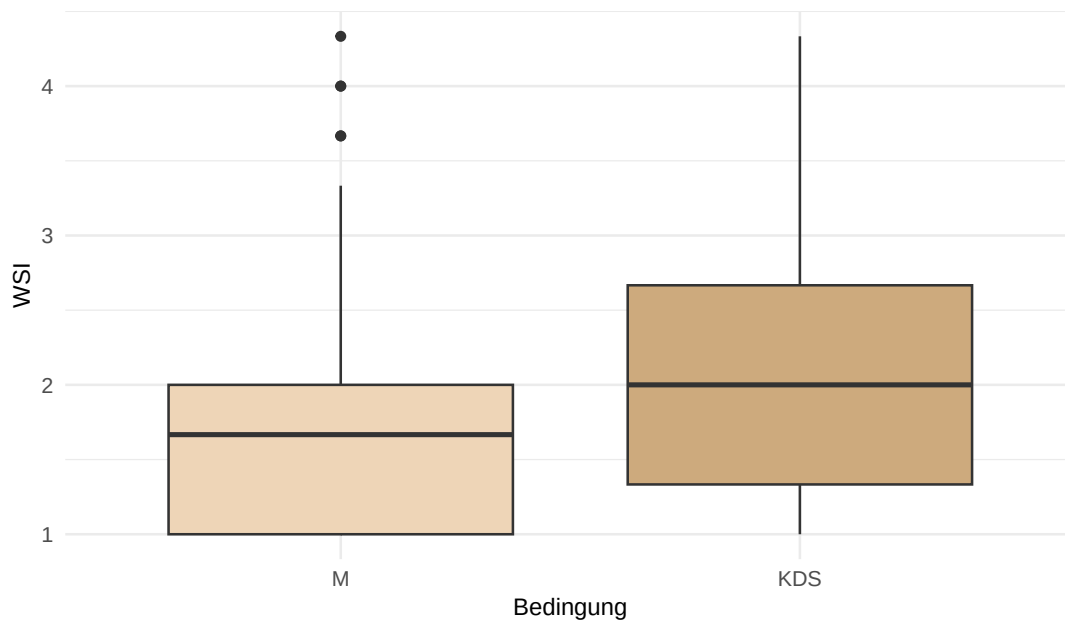
```
# Zusätzliches Modell Inkonsistenz: Min Schuh vs stark Hiphop Schuh
# DV: Stilistische Inkonsistenz
# IV: Conditions/ Outfits (Stark vs gar nicht -> Kategorisch 2 Ausprägungen)
# Within-Subjects daher dependent sample
# -> Dependent-t-Test

# Daten für die beiden Conditions auswählen
plot_schuh <- prepare_plot_data(
  conditions = c(
    "Min-Baseline",
    "Schuh-stark-dezentral"
  ),
  levels = c(
    "Min-Baseline",
    "Schuh-stark-dezentral"
  ),
  labels = c(
    "M",
    "KDS"
  )
)

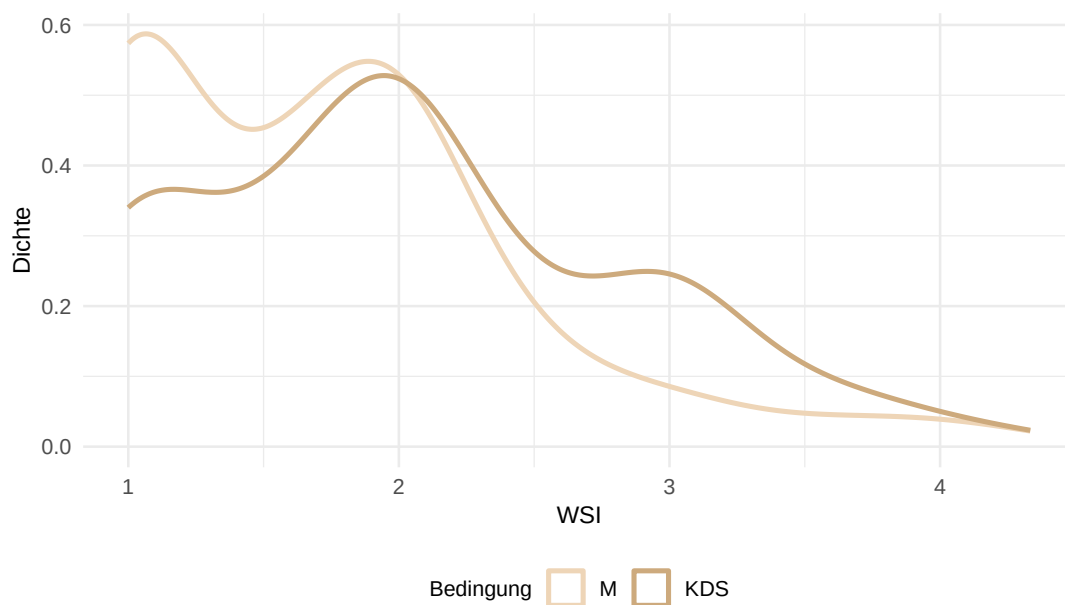
inkonsistenz_schuh <- prepare_paired_data(
  "Min-Baseline",
  "Schuh-stark-dezentral"
)

# Box und Density Plots erstellen
plot_box_density(
  plot_schuh,
  farben = c(
    "M" = "#EED5B7",
    "KDS" = "#CDAA7D"
  ),
  subtitle = "Schuh-Stärke (2 Stufen): M vs. KDS"
)
```

Boxplot der WSI: Schuh-Stärke (2 Stufen): M vs. KDS



Dichteplot der WSI: Schuh-Stärke (2 Stufen): M vs. KDS



```
# Deskriptive Kennwerte erstellen
deskriptiv_schuh <- deskriptive_kennwerte(plot_schuh)
deskriptiv_schuh

# Differenzwert bilden:
# positive Werte = stark abweichender Hip-Hop-Schuh wurde
# inkonsistenter bewertet als die minimalistische Baseline
inkonsistenz_schuh <- inkonsistenz_schuh %>%
  mutate(
    Differenz_Inkonsistenz =
      `Schuh-stark-dezentral` -
      `Min-Baseline`
```

```

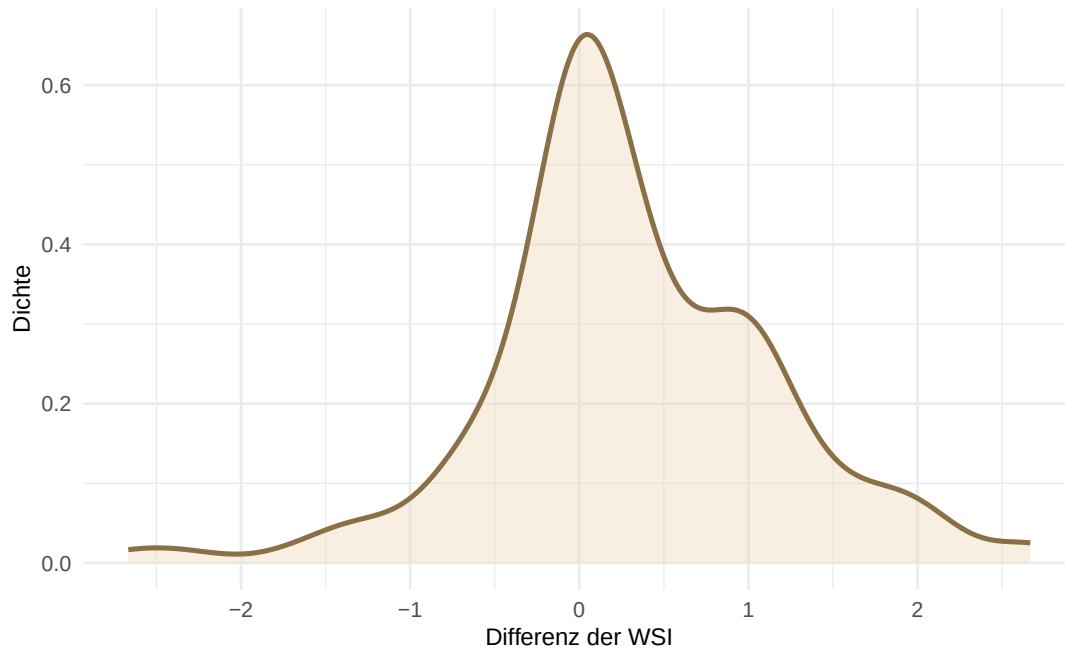
)

# Anzahl vollständiger Paare kontrollieren
nrow(inkonsistenz_schuh)

# Normalverteilung grafisch darstellen (nur deskriptiv; nicht für die Testwahl)
plot_differenz(
  inkonsistenz_schuh,
  "KDS - M"
)

```

Dichteplot der Differenzwerte: KDS – M



```

# Normalverteilung mit Shapiro-Wilk prüfen
# H0: Die Differenzwerte sind normalverteilt
# H1: Die Differenzwerte sind nicht normalverteilt
shapiro_schuh <- shapiro.test(
  inkonsistenz_schuh$Differenz_Inkonsistenz
)
shapiro_schuh

if (shapiro_schuh$p.value > 0.05) {
  t_test_schuh <- t.test(
    inkonsistenz_schuh$`Schuh-stark-dezentral`,
    inkonsistenz_schuh$`Min-Baseline`,
    paired = TRUE,
    alternative = "greater"
  )
  t_test_schuh
} else {
  wilcoxon_schuh <- wilcox.test(
    inkonsistenz_schuh$`Schuh-stark-dezentral`,
    inkonsistenz_schuh$`Min-Baseline`,
    paired = TRUE,
    alternative = "greater",
    exact = FALSE
  )
}

```

```
wilcoxon_schuh
}
```

Table 49: Deskriptive Kennwerte – M vs. KDS - Teil 1 von 2

Condition	n	Median	IQR	Mittelwert
M	135	1.667	1.000	1.741
KDS	135	2.000	1.333	2.044

Table 50: Deskriptive Kennwerte – M vs. KDS - Teil 2 von 2

Condition	SD
M	0.729
KDS	0.800

Table 51: Shapiro-Wilk-Test der Differenzwerte – M vs. KDS

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.946
p-Wert	< 1e-04

Table 52: Gerichteter Wilcoxon-Vorzeichen-Rang-Test – M vs. KDS

Kennwert	Wert
Methode	Wilcoxon signed rank test with continuity correction
V	3198
p-Wert	< 1e-04
Alternative	greater

6 Power-Analyse und Fehlerwahrscheinlichkeiten

```
# Power-Analyse / Fehler 1. und 2. Grades
alpha <- 0.05

#### Hilfsfunktion 1: Gepaarter t-Test
power_t_paired <- function(differenz, modell, alpha = 0.05) {
  differenz <- na.omit(differenz)
  # Cohen's dz
  d <- mean(differenz) / sd(differenz)
  # Power
  power <- pwr.t.test(
    n = length(differenz),
    d = d,
```

```

    sig.level = alpha,
    type = "paired",
    alternative = "greater"
  )$power
data.frame(
  Modell = modell,
  Test = "Gepaarter t-Test",
  n = length(differenz),
  Cohen_dz = d,
  Alpha = alpha,
  Power = power,
  Beta = 1 - power)})

#### Hilfsfunktion 2: Wilcoxon
power_wilcoxon <- function(x, y, modell,
                           alpha = 0.05,
                           B = 5000,
                           seed = 123) {
  # Nur vollständige Paare
  komplett <- complete.cases(x, y)
  x <- x[komplett]
  y <- y[komplett]
  n <- length(x)
  set.seed(seed)
  p_werte <- replicate(
    B,
    {index <- sample(
      seq_len(n),
      size = n,
      replace = TRUE)
     wilcox.test(
       x[index],
       y[index],
       paired = TRUE,
       alternative = "greater",
       exact = FALSE
     )$p.value})
  power <- mean(p_werte < alpha, na.rm = TRUE)
  data.frame(
    Modell = modell,
    Test = "Wilcoxon",
    n = n,
    Cohen_dz = NA,
    Alpha = alpha,
    Power = power,
    Beta = 1 - power)})

#### Hilfsfunktion 3: Friedman
power_friedman_test <- function(matrix,
                                 modell,
                                 alpha = 0.05,
                                 B = 1000,
                                 seed = 123) {
  matrix <- matrix[complete.cases(matrix), ]
  n <- nrow(matrix)
  set.seed(seed)
  p_werte <- replicate(
    B,
    {
      index <- sample(
        seq_len(n),

```

```

    size = n,
    replace = TRUE )
  friedman.test(
    matrix[index, , drop = FALSE]
  )$p.value } )
power <- mean(p_werte < alpha)
data.frame(
  Modell = modell,
  Test = "Friedman",
  n = n,
  Cohen_dz = NA,
  Alpha = alpha,
  Power = power,
  Beta = 1 - power)})

# Fall 1: Größe
power_groesse <- power_t_paired(
  inkonsistenz_groesse$Differenz_Inkonsistenz,
  "Größe: GDS vs. KDS")

# Fall 2: Position
power_position <- power_wilcoxon(
  inkonsistenz_position$`Jacke-stark-zentral`,
  inkonsistenz_position$`Jacke-stark-dezentral`,
  "Position: GZS vs. GDS")

# Fall 3: Stärke
power_staerke <- power_t_paired(
  inkonsistenz_staerke$Differenz_Inkonsistenz,
  "Stärke: GDS vs. GDL")

# Fall 4: Baseline vs. leicht vs. stark
friedman_matrix <- inkonsistenz_anova_staerke %>%
  dplyr::select(
    number,
    Condition,
    Mittelwert_Inkonsistenz
  ) %>%
  pivot_wider(
    names_from = Condition,
    values_from = Mittelwert_Inkonsistenz
  ) %>%
  dplyr::select(
    `M`,
    `GDL`,
    `GDS`
  ) %>%
  drop_na() %>%
  as.matrix()
power_friedman <- power_friedman_test(
  friedman_matrix,
  "Stärke: M vs. GDL vs. GDS")

# Fall 5: Baseline vs. starker Hip-Hop-Schuh
power_schuh <- power_wilcoxon(
  inkonsistenz_schuh$`Schuh-stark-dezentral`,
  inkonsistenz_schuh$`Min-Baseline`,
  "Schuh: M vs. KDS")

# Übersicht
power_uebersicht <- bind_rows(

```



```

power_groesse,
power_position,
power_staerke,
power_friedman,
power_schuh
) %>%
mutate(
  Alpha_Prozent = round(Alpha * 100, 2),
  Power_Prozent = round(Power * 100, 2),
  Beta_Prozent = round(Beta * 100, 2),
  Cohen_dz = round(Cohen_dz, 3))
power_uebersicht

```

Table 53: Übersicht der Power-Analysen - Teil 1 von 3

Modell	Test	n	Cohen_dz	Alpha
Größe: GDS vs. KDS	Gepaarter t-Test	135	1.007	0.05
Position: GZS vs. GDS	Wilcoxon	135		0.05
Stärke: GDS vs. GDL	Gepaarter t-Test	135	0.537	0.05
Stärke: M vs. GDL vs. GDS	Friedman	135		0.05
Schuh: M vs. KDS	Wilcoxon	135		0.05

Table 54: Übersicht der Power-Analysen - Teil 2 von 3

Modell	Test	Power	Beta	Alpha_Prozent
Größe: GDS vs. KDS	Gepaarter t-Test	1.000	0.000	5
Position: GZS vs. GDS	Wilcoxon	0.999	0.001	5
Stärke: GDS vs. GDL	Gepaarter t-Test	1.000	0.000	5
Stärke: M vs. GDL vs. GDS	Friedman	1.000	0.000	5
Schuh: M vs. KDS	Wilcoxon	0.996	0.004	5

Table 55: Übersicht der Power-Analysen - Teil 3 von 3

Modell	Test	Power_Prozent	Beta_Prozent
Größe: GDS vs. KDS	Gepaarter t-Test	100.00	0.00
Position: GZS vs. GDS	Wilcoxon	99.90	0.10
Stärke: GDS vs. GDL	Gepaarter t-Test	100.00	0.00
Stärke: M vs. GDL vs. GDS	Friedman	100.00	0.00
Schuh: M vs. KDS	Wilcoxon	99.64	0.36

7 Unterschiede nach Geschlecht

7.1 Hilfsfunktionen für die Mixed-ANOVAs

```

# Unterschiedliche Gruppen: Geschlechtsunterschiede
# Die wiederkehrenden Arbeitsschritte werden in Hilfsfunktionen zusammengefasst.
# Methodik, Filterung, Faktorstufen, Tests und Grafiken bleiben unverändert.

```

```

bereite_geschlechtsdaten_vor <- function(conditions, faktor, zuordnung, stufen) {
  salienz_reshaped %>%
    filter(Condition %in% conditions) %>%
    mutate(
      Geschlecht = case_when(
        as.character(Geschlecht) %in% c("1", "weiblich") ~ "weiblich",
        as.character(Geschlecht) %in% c("2", "männlich") ~ "männlich",
        TRUE ~ NA_character_
      ),
      !!rlang::sym(faktor) := unname(zuordnung[as.character(Condition)])
    ) %>%
    filter(!is.na(Geschlecht), !is.na(Mittelwert_Inkonsistenz)) %>%
    mutate(
      Geschlecht = factor(Geschlecht, levels = c("weiblich", "männlich")),
      !!rlang::sym(faktor) := factor(.data[[faktor]], levels = stufen)
    ) %>%
    group_by(number) %>%
    filter(n_distinct(.data[[faktor]]) == length(stufen)) %>%
    ungroup()
}

erstelle_boxplot_geschlecht <- function(data, faktor, stufe, subtitle,
                                       manuelle_farben = FALSE,
                                       titel_zentriert = FALSE) {
  p <- data %>%
    filter(.data[[faktor]] == stufe) %>%
    ggplot(aes(
      x = Geschlecht,
      y = Mittelwert_Inkonsistenz,
      fill = Geschlecht
    )) +
    geom_boxplot() +
    scale_fill_manual(
      values = c(
        "weiblich" = "#EED5B7",
        "männlich" = "#CDAA7D"
      )
    )
  p <- p +
    scale_y_continuous(breaks = 1:5, limits = c(1, 5)) +
    labs(
      title = stringr::str_wrap(paste("WSI nach Geschlecht:", subtitle), width = 48),
      x = "Geschlecht",
      y = "WSI"
    ) +
    theme_box_density_gross +
    theme(legend.position = "none")

  if (titel_zentriert) {
    p <- p + theme(
      plot.title = element_text(hjust = 0.5)
    )
  }

  p
}

deskriptiv_geschlecht <- function(data, faktor) {
  data %>%
    group_by(Geschlecht, !!rlang::sym(faktor)) %>%

```

```

    summarise(
      n = n(),
      Median = median(Mittelwert_Inkonsistenz, na.rm = TRUE),
      IQR = IQR(Mittelwert_Inkonsistenz, na.rm = TRUE),
      Mittelwert = mean(Mittelwert_Inkonsistenz, na.rm = TRUE),
      SD = sd(Mittelwert_Inkonsistenz, na.rm = TRUE),
      .groups = "drop"
    )
  }

schaetze_mixed_anova <- function(data, faktor) {
  modell <- aov_ez(
    id = "number",
    dv = "Mittelwert_Inkonsistenz",
    within = faktor,
    between = "Geschlecht",
    data = data
  )
  modell
}

density_residuen <- function(residuen, subtitle, farbig = TRUE, zentriert = TRUE) {
  p <- ggplot(
    data.frame(Residuen = as.numeric(residuen)),
    aes(x = Residuen)
  )

  if (farbig) {
    p <- p + geom_density(
      fill = "#EED5B7",
      color = "#8B6F47",
      alpha = 0.4,
      linewidth = 1
    )
  } else {
    p <- p + geom_density(
      fill = "#EED5B7",
      color = "#8B6F47",
      alpha = 0.4,
      linewidth = 1
    )
  }

  p <- p +
    labs(
      title = stringr::str_wrap(paste("Dichteplot der Residuen:", subtitle), width = 48),
      x = "Residuen",
      y = "Dichte"
    ) +
    theme_box_density_gross

  if (zentriert) {
    p <- p + theme(
      legend.position = "none",
      plot.title = element_text(hjust = 0.5),
      plot.subtitle = element_text(hjust = 0.5)
    )
  }

  p
}

```

```

# Für alle folgenden Mixed-ANOVA-Blöcke gilt ebenfalls:
# Density-Plots dienen nur der Beschreibung. Die Entscheidung zwischen klassischem und
# robustem Test
# erfolgt ausschließlich anhand der p-Werte von Shapiro-Wilk und Levene (alpha = 0.05).

shapiro_residuen <- function(residuen) {
  test <- shapiro.test(as.numeric(residuen))
  test
}

levene_stufe <- function(data, faktor, stufe) {
  test <- leveneTest(
    Mittelwert_Inkonsistenz ~ Geschlecht,
    data = data %>% filter(.data[[faktor]] == stufe)
  )
  test
}

```

7.2 Position × Geschlecht

```

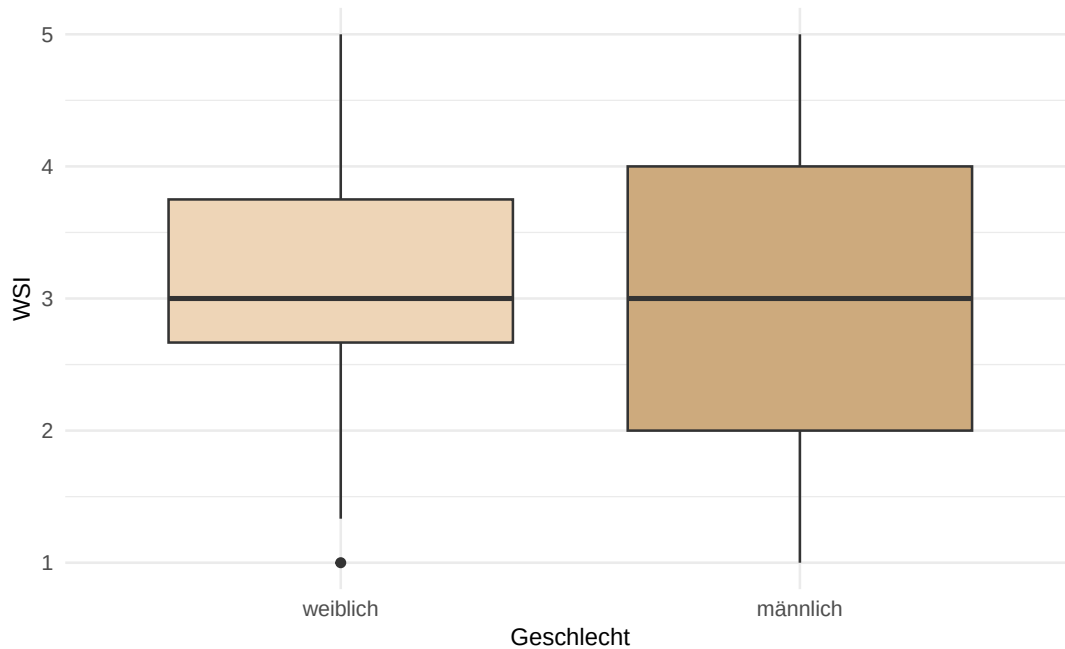
# 1. Position × Geschlecht
inkonsistenz_position_geschlecht <- bereite_geschlechtsdaten_vor(
  conditions = c("Jacke-stark-dezentral", "Jacke-stark-zentral"),
  faktor = "Position",
  zuordnung = c(
    "Jacke-stark-dezentral" = "dezentral",
    "Jacke-stark-zentral" = "zentral"
  ),
  stufen = c("dezentral", "zentral")
)

# Kontrolle
table(inkonsistenz_position_geschlecht$number)
inkonsistenz_position_geschlecht %>%
  distinct(number, Geschlecht) %>%
  count(Geschlecht)

# Boxplots
boxplot_position_dezentral <- erstelle_boxplot_geschlecht(
  inkonsistenz_position_geschlecht, "Position", "dezentral",
  "GDS",
  manuelle_farben = TRUE,
  titel_zentriert = TRUE
)
speichere_grafik(boxplot_position_dezentral, "WSI nach Geschlecht")
print(boxplot_position_dezentral)

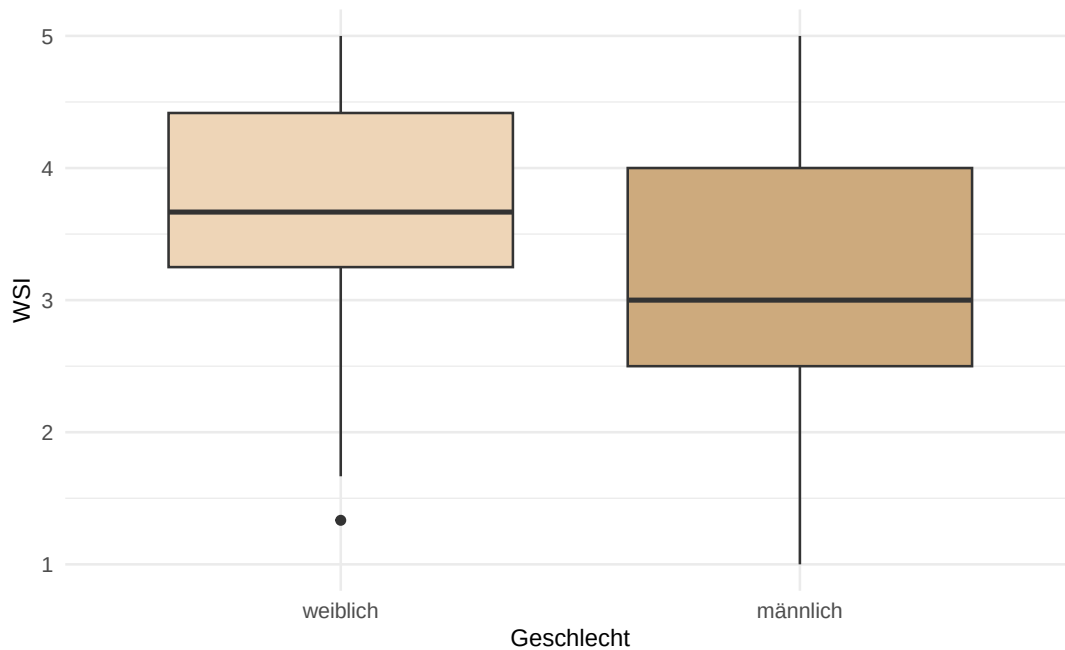
```

WSI nach Geschlecht: GDS



```
boxplot_position_zentral <- erstelle_boxplot_geschlecht(
  inkonsistenz_position_geschlecht, "Position", "zentral",
  "GZS",
  manuelle_farben = TRUE,
  titel_zentriert = TRUE
)
speichere_grafik(boxplot_position_zentral, "WSI nach Geschlecht")
print(boxplot_position_zentral)
```

WSI nach Geschlecht: GZS



```
# Deskriptive Kennwerte
deskriptiv_position_geschlecht <- deskriptiv_geschlecht(
  inkonsistenz_position_geschlecht, "Position"
```

```

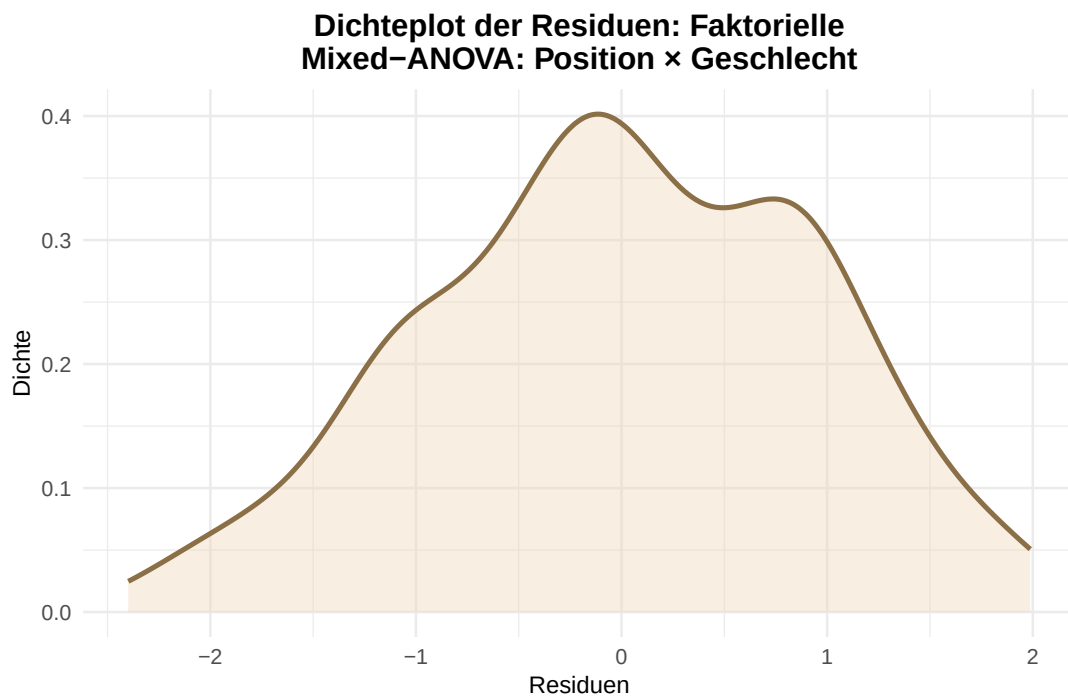
)
deskriptiv_position_geschlecht

# Mixed ANOVA zunächst schätzen, um Residuen zu prüfen
anova_position_geschlecht <- schaeetze_mixed_anova(
  inkonsistenz_position_geschlecht,
  "Position"
)

residuen_position_geschlecht <- residuals(anova_position_geschlecht$lm)

zeige_und_speichere_grafik(
  density_residuen(
    residuen_position_geschlecht,
    "Faktorielle Mixed-ANOVA: Position × Geschlecht",
    farbig = TRUE,
    zentriert = TRUE
  ),
  "Dichteplot der Residuen"
)

```



```

shapiro_auto_05 <- shapiro_residuen(
  residuen_position_geschlecht
)
shapiro_auto_05

# Varianzhomogenität
levene_auto_01 <- levene_stufe(
  inkonsistenz_position_geschlecht, "Position", "dezentral"
)
levene_auto_01

levene_auto_02 <- levene_stufe(
  inkonsistenz_position_geschlecht, "Position", "zentral"
)

```

```

levene_auto_02

if (shapiro_auto_05$p.value > 0.05 &&
    levene_auto_01$`Pr(>F)`[1] > 0.05 &&
    levene_auto_02$`Pr(>F)`[1] > 0.05) {
  anova_position_geschlecht <- schaetze_mixed_anova(
    inkonsistenz_position_geschlecht,
    "Position"
  )
  anova_position_geschlecht
} else {
  robust_anova_position_geschlecht <- bwtrim(
    Mittelwert_Inkonsistenz ~ Geschlecht * Position,
    id = number,
    data = inkonsistenz_position_geschlecht,
    tr = 0.20
  )
  robust_anova_position_geschlecht
}

```

Table 56: Deskriptive Kennwerte – Position $\times$ Geschlecht -
Teil 1 von 2

Geschlecht	Position	n	Median	IQR
weiblich	dezentral	76	3.000	1.083
weiblich	zentral	76	3.667	1.167
männlich	dezentral	55	3.000	2.000
männlich	zentral	55	3.000	1.500

Table 57: Deskriptive Kennwerte – Position $\times$ Geschlecht -
Teil 2 von 2

Geschlecht	Mittelwert	SD
weiblich	3.171	0.892
weiblich	3.732	0.933
männlich	3.012	0.973
männlich	3.206	0.997

Table 58: Shapiro-Wilk-Test der Residuen – Position $\times$
Geschlecht

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.988
p-Wert	0.02554

Table 59: Levene-Test – GDS nach Geschlecht

Df	F value	Pr(>F)
1	1.544	0.216
129		

Table 60: Levene-Test – GZS nach Geschlecht

Df	F value	Pr(>F)
1	0.899	0.345
129		

Table 61: Robuste Mixed-ANOVA – Position $\times$ Geschlecht

Ausgabe
Call:
bwtrim(formula = Mittelwert_Inkonsistenz ~ Geschlecht * Position, id = number, data = inkonsistenz_position_geschlecht, tr = 0.2)
value df1 df2 p.value
Geschlecht 6.0199 1 74.5059 0.0165
Position 14.4917 1 83.5582 0.0003
Geschlecht:Position 2.2732 1 83.5582 0.1354

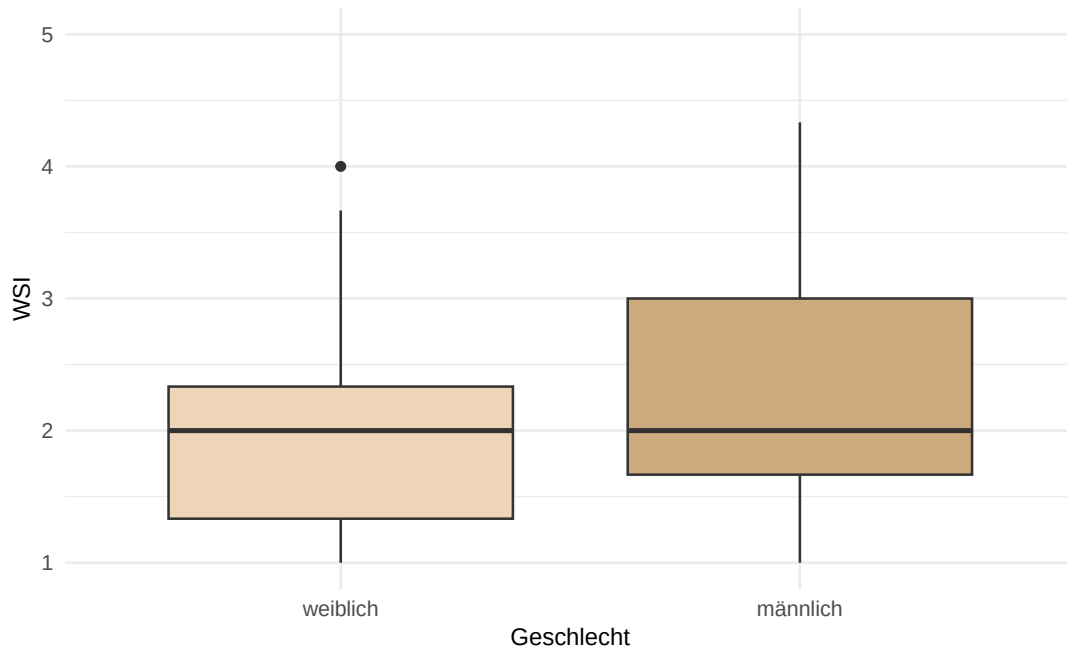
7.3 Größe $\times$ Geschlecht

```
# 2. Größe  $\times$  Geschlecht
inkonsistenz_groesse_geschlecht <- bereite_geschlechtsdaten_vor(
  conditions = c("Schuh-stark-dezentral", "Jacke-stark-dezentral"),
  faktor = "Groesse",
  zuordnung = c(
    "Schuh-stark-dezentral" = "klein",
    "Jacke-stark-dezentral" = "gross"
  ),
  stufen = c("klein", "gross")
)

# Kontrolle
table(inkonsistenz_groesse_geschlecht$number)
inkonsistenz_groesse_geschlecht %>%
  distinct(number, Geschlecht) %>%
  count(Geschlecht)

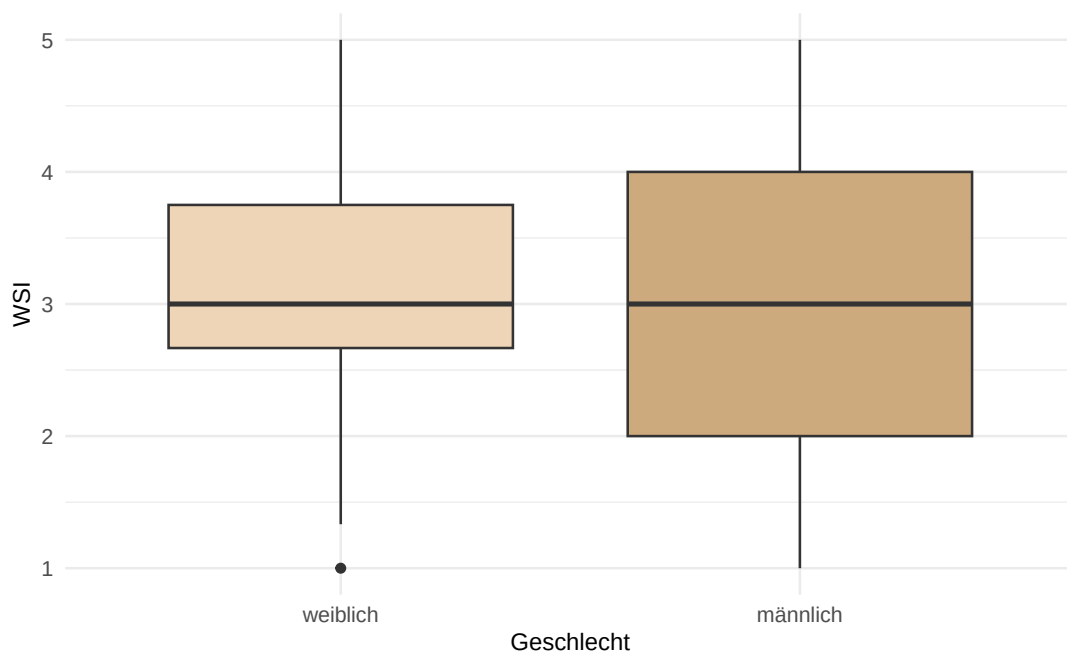
# Boxplots
boxplot_groesse_klein <- erstelle_boxplot_geschlecht(
  inkonsistenz_groesse_geschlecht, "Groesse", "klein",
  "KDS"
)
speichere_grafik(boxplot_groesse_klein, "WSI nach Geschlecht")
print(boxplot_groesse_klein)
```


WSI nach Geschlecht: KDS



```
boxplot_groesse_gross <- erstelle_boxplot_geschlecht(
  inkonsistenz_groesse_geschlecht, "Groesse", "gross",
  "GDS"
)
speichere_grafik(boxplot_groesse_gross, "WSI nach Geschlecht")
print(boxplot_groesse_gross)
```

WSI nach Geschlecht: GDS



```
# Deskriptive Kennwerte
deskriptiv_groesse_geschlecht <- deskriptiv_geschlecht(
  inkonsistenz_groesse_geschlecht, "Groesse"
)
deskriptiv_groesse_geschlecht
```

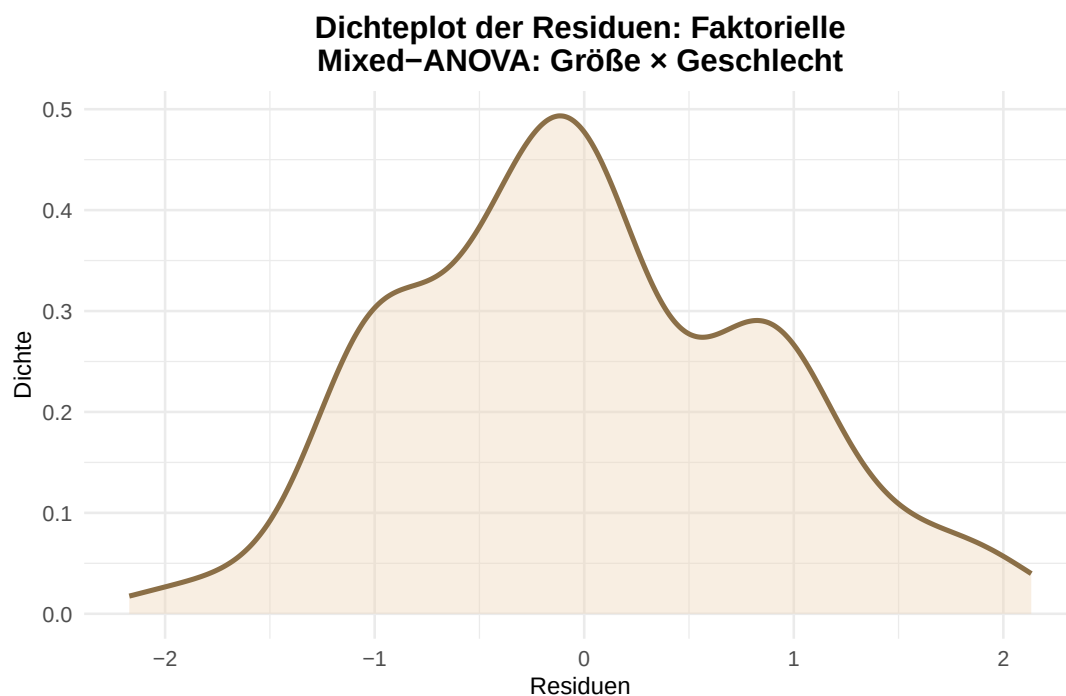
```

# Mixed ANOVA zunächst schätzen
anova_groesse_geschlecht <- schaeetze_mixed_anova(
  inkonsistenz_groesse_geschlecht,
  "Groesse"
)

residuen_groesse_geschlecht <- residuals(anova_groesse_geschlecht$lm)

zeige_und_speichere_grafik(
  density_residuen(
    residuen_groesse_geschlecht,
    "Faktorielle Mixed-ANOVA: Größe × Geschlecht"
  ),
  "Dichteplot der Residuen"
)

```



```

shapiro_auto_06 <- shapiro_residuen(
  residuen_groesse_geschlecht
)
shapiro_auto_06

# Varianzhomogenität
levene_auto_03 <- levene_stufe(
  inkonsistenz_groesse_geschlecht, "Groesse", "klein"
)
levene_auto_03

levene_auto_04 <- levene_stufe(
  inkonsistenz_groesse_geschlecht, "Groesse", "gross"
)
levene_auto_04

if (shapiro_auto_06$p.value > 0.05 &&

```

```

    levene_auto_03$`Pr(>F)`[1] > 0.05 &&
    levene_auto_04$`Pr(>F)`[1] > 0.05) {
anova_groesse_geschlecht <- schaeetze_mixed_anova(
  inkonsistenz_groesse_geschlecht,
  "Groesse"
)
anova_groesse_geschlecht
} else {
robust_anova_groesse_geschlecht <- bwtrim(
  Mittelwert_Inkonsistenz ~ Geschlecht * Groesse,
  id = number,
  data = inkonsistenz_groesse_geschlecht,
  tr = 0.20
)
robust_anova_groesse_geschlecht
}

```

Table 62: Deskriptive Kennwerte – Größe × Geschlecht - Teil
1 von 2

Geschlecht	Groesse	n	Median	IQR
weiblich	klein	76	2	1.000
weiblich	gross	76	3	1.083
männlich	klein	55	2	1.333
männlich	gross	55	3	2.000

Table 63: Deskriptive Kennwerte – Größe × Geschlecht - Teil
2 von 2

Geschlecht	Mittelwert	SD
weiblich	1.961	0.726
weiblich	3.171	0.892
männlich	2.200	0.862
männlich	3.012	0.973

Table 64: Shapiro-Wilk-Test der Residuen – Größe × Geschlecht

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.985
p-Wert	0.006153

Table 65: Levene-Test – KDS nach Geschlecht

Df	F value	Pr(>F)
1	1.644	0.202

Df	F value	Pr(>F)
129		

Table 66: Levene-Test – GDS nach Geschlecht

Df	F value	Pr(>F)
1	1.544	0.216
129		

Table 67: Robuste Mixed-ANOVA – Größe $\times$ Geschlecht

Ausgabe				
Call:				
bwtrim(formula = Mittelwert_Inkonsistenz ~ Geschlecht * Groesse, id = number, data = inkonsistenz_groesse_geschlecht, tr = 0.2)				
	value	df1	df2	p.value
Geschlecht	0.0032	1	73.011	0.9550
Groesse	79.3613	1	74.107	0.0000
Geschlecht:Groesse	3.8614	1	74.107	0.0532

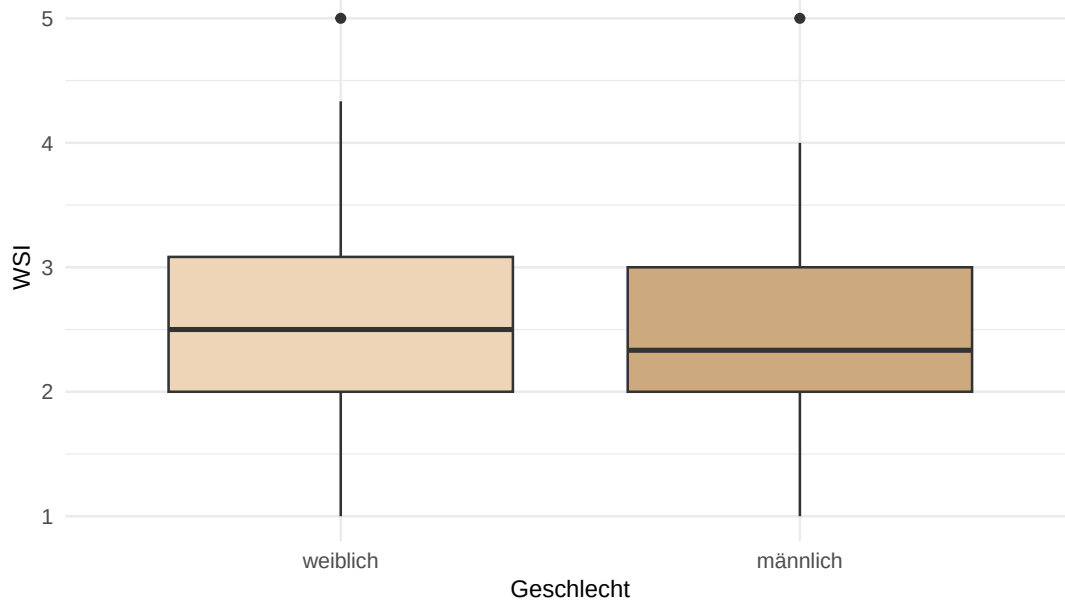
7.4 Stärke (2 Stufen) $\times$ Geschlecht

```
# 3. Stärke  $\times$  Geschlecht
inkonsistenz_staerke_geschlecht <- bereite_geschlechtsdaten_vor(
  conditions = c("Jacke-leicht-dezentral", "Jacke-stark-dezentral"),
  faktor = "Staerke",
  zuordnung = c(
    "Jacke-leicht-dezentral" = "leicht",
    "Jacke-stark-dezentral" = "stark"
  ),
  stufen = c("leicht", "stark")
)

# Kontrolle
table(inkonsistenz_staerke_geschlecht$number)
inkonsistenz_staerke_geschlecht %>%
  distinct(number, Geschlecht) %>%
  count(Geschlecht)

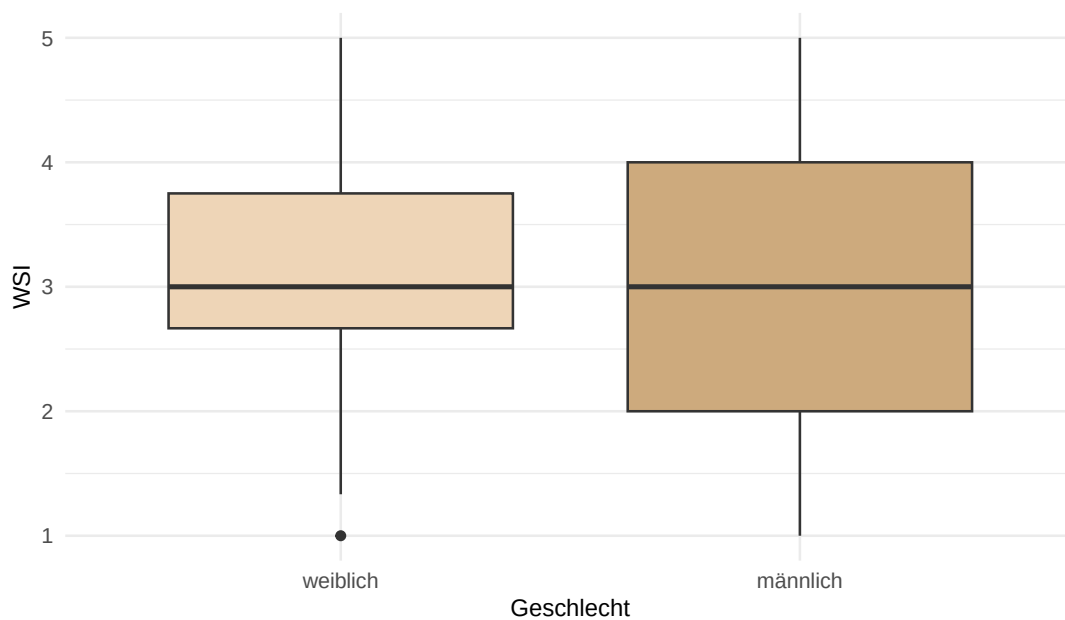
# Boxplots
boxplot_staerke_leicht <- erstelle_boxplot_geschlecht(
  inkonsistenz_staerke_geschlecht, "Staerke", "leicht",
  "Stärke (2 Stufen) - leichte stilistische Abweichung: GDL"
)
speichere_grafik(boxplot_staerke_leicht, "WSI nach Geschlecht")
print(boxplot_staerke_leicht)
```

WSI nach Geschlecht: Stärke (2 Stufen) – leichte stilistische Abweichung: GDL



```
boxplot_staerke_stark <- erstelle_boxplot_geschlecht(
  inkonsistenz_staerke_geschlecht, "Staerke", "stark",
  "Stärke (2 Stufen) - starke stilistische Abweichung: GDS"
)
speichere_grafik(boxplot_staerke_stark, "WSI nach Geschlecht")
print(boxplot_staerke_stark)
```

WSI nach Geschlecht: Stärke (2 Stufen) – starke stilistische Abweichung: GDS



```
# Deskriptive Kennwerte
deskriptiv_staerke_geschlecht <- deskriptiv_geschlecht(
  inkonsistenz_staerke_geschlecht, "Staerke"
)
deskriptiv_staerke_geschlecht
```

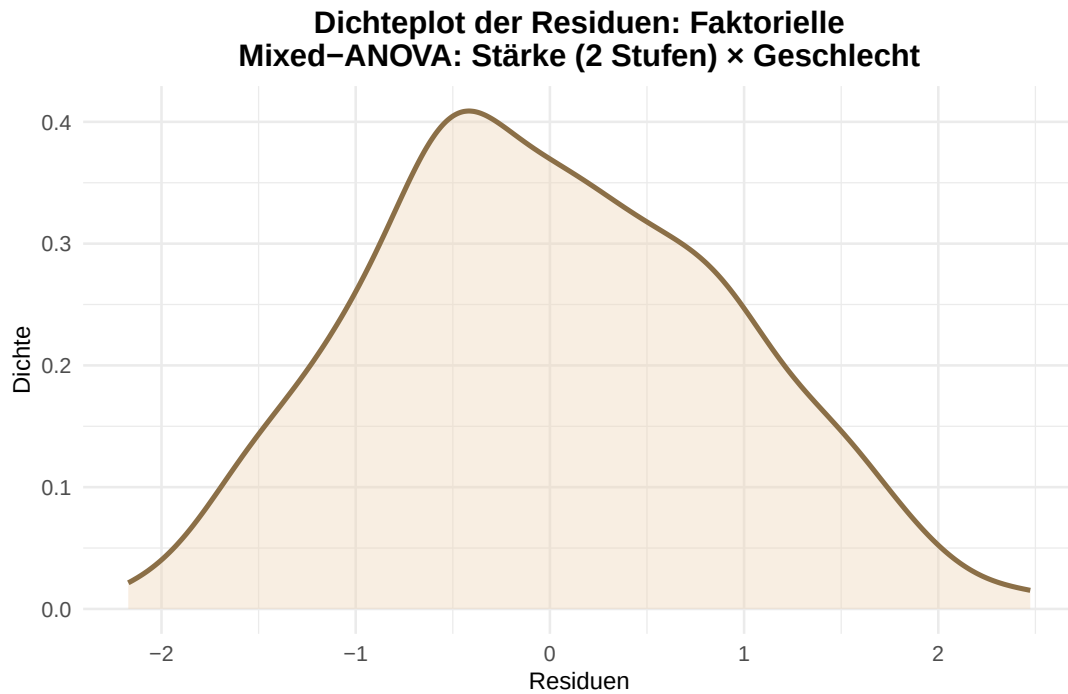
```

# Mixed ANOVA zunächst schätzen
anova_staerke_geschlecht <- schaetze_mixed_anova(
  inkonsistenz_staerke_geschlecht,
  "Staerke"
)

residuen_staerke_geschlecht <- residuals(anova_staerke_geschlecht$lm)

zeige_und_speichere_grafik(
  density_residuen(
    residuen_staerke_geschlecht,
    "Faktorielle Mixed-ANOVA: Stärke (2 Stufen) × Geschlecht",
    farbig = TRUE,
    zentriert = TRUE
  ),
  "Dichteplot der Residuen"
)

```



```

shapiro_auto_07 <- shapiro_residuen(
  residuen_staerke_geschlecht
)
shapiro_auto_07

# Varianzhomogenität
levene_auto_05 <- levene_stufe(
  inkonsistenz_staerke_geschlecht, "Staerke", "leicht"
)
levene_auto_05

levene_auto_06 <- levene_stufe(
  inkonsistenz_staerke_geschlecht, "Staerke", "stark"
)
levene_auto_06

```

```

if (shapiro_auto_07$p.value > 0.05 &&
    levene_auto_05$`Pr(>F)`[1] > 0.05 &&
    levene_auto_06$`Pr(>F)`[1] > 0.05) {
  anova_staerke_geschlecht <- schaeetze_mixed_anova(
    inkonsistenz_staerke_geschlecht,
    "Staerke"
  )
  anova_staerke_geschlecht
} else {
  robust_anova_staerke_geschlecht <- bwtrim(
    Mittelwert_Inkonsistenz ~ Geschlecht * Staerke,
    id = number,
    data = inkonsistenz_staerke_geschlecht,
    tr = 0.20
  )
  robust_anova_staerke_geschlecht
}

```

Table 68: Deskriptive Kennwerte – Stärke (2 Stufen) × Geschlecht - Teil 1 von 2

Geschlecht	Staerke	n	Median	IQR
weiblich	leicht	76	2.500	1.083
weiblich	stark	76	3.000	1.083
männlich	leicht	55	2.333	1.000
männlich	stark	55	3.000	2.000

Table 69: Deskriptive Kennwerte – Stärke (2 Stufen) × Geschlecht - Teil 2 von 2

Geschlecht	Mittelwert	SD
weiblich	2.526	0.954
weiblich	3.171	0.892
männlich	2.527	0.893
männlich	3.012	0.973

Table 70: Shapiro-Wilk-Test der Residuen – Stärke (2 Stufen) × Geschlecht

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.986
p-Wert	0.01071

Table 71: Levene-Test – GDL nach Geschlecht

Df	F value	Pr(>F)
1	0.287	0.593
129		

Table 72: Levene-Test – GDS nach Geschlecht

Df	F value	Pr(>F)
1	1.544	0.216
129		

Table 73: Robuste Mixed-ANOVA – Stärke (2 Stufen) × Geschlecht

Ausgabe					
Call:					
bwtrim(formula = Mittelwert_Inkonsistenz ~ Geschlecht * Staerke, id = number, data = inkonsistenz_staerke_geschlecht, tr = 0.2)					
	value	df1	df2	p.value	
Geschlecht	0.6991	1	81.0963	0.4056	
Staerke	29.5182	1	81.4742	0.0000	
Geschlecht:Staerke	0.7013	1	81.4742	0.4048	

7.5 Schuh-Stärke (2 Stufen) × Geschlecht

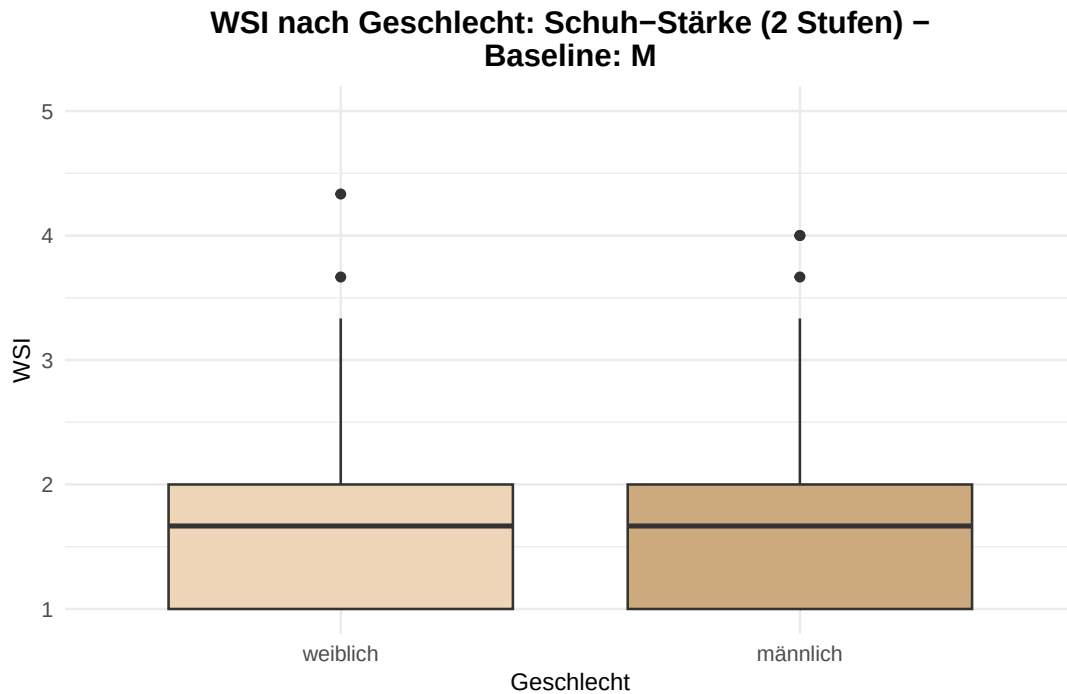
```
# 4. Schuh-Stärke × Geschlecht
inkonsistenz_staerke_schuh_geschlecht <- bereite_geschlechtsdaten_vor(
  conditions = c("Min-Baseline", "Schuh-stark-dezentral"),
  faktor = "Staerke",
  zuordnung = c(
    "Min-Baseline" = "baseline",
    "Schuh-stark-dezentral" = "stark"
  ),
  stufen = c("baseline", "stark")
)

# Kontrolle
table(inkonsistenz_staerke_schuh_geschlecht$number)
inkonsistenz_staerke_schuh_geschlecht %>%
  distinct(number, Geschlecht) %>%
  count(Geschlecht)

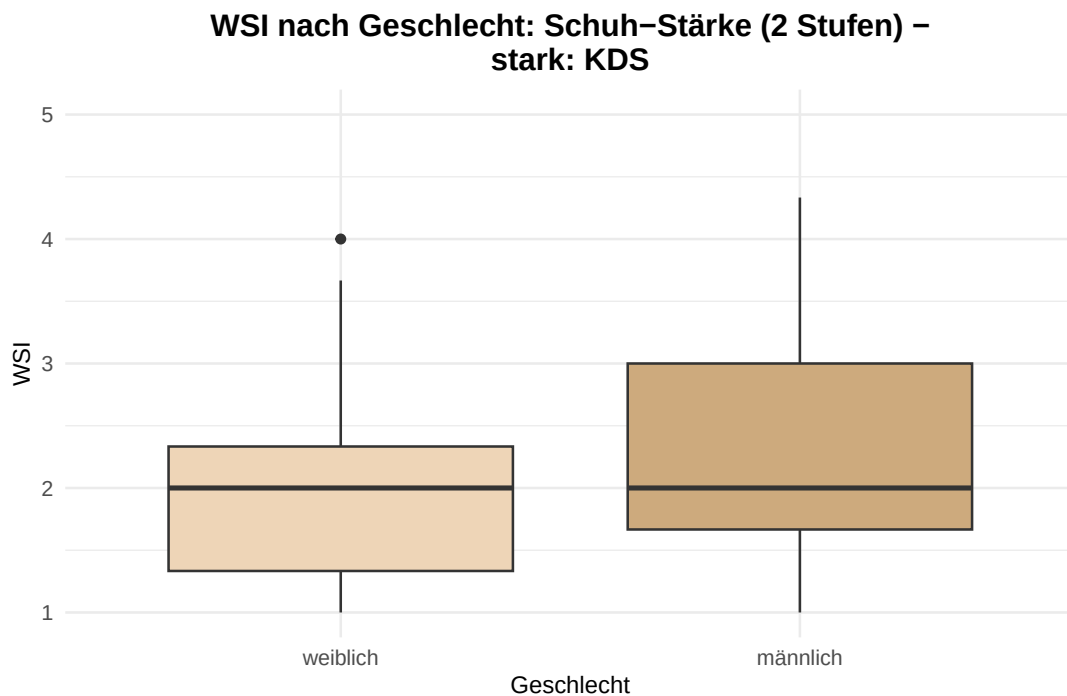
# Boxplots
boxplot_staerke_schuh_baseline <- erstelle_boxplot_geschlecht(
  inkonsistenz_staerke_schuh_geschlecht, "Staerke", "baseline",
  "Schuh-Stärke (2 Stufen) - Baseline: M"
)
```



```
speichere_grafik(boxplot_staerke_schuh_baseline, "WSI nach Geschlecht")
print(boxplot_staerke_schuh_baseline)
```



```
boxplot_staerke_schuh_stark <- erstelle_boxplot_geschlecht(
  inkonsistenz_staerke_schuh_geschlecht, "Staerke", "stark",
  "Schuh-Stärke (2 Stufen) – stark: KDS"
)
speichere_grafik(boxplot_staerke_schuh_stark, "WSI nach Geschlecht")
print(boxplot_staerke_schuh_stark)
```



```

# Deskriptive Kennwerte
deskriptiv_staerke_schuh_geschlecht <- deskriptiv_geschlecht(
  inkonsistenz_staerke_schuh_geschlecht, "Staerke"
)
deskriptiv_staerke_schuh_geschlecht

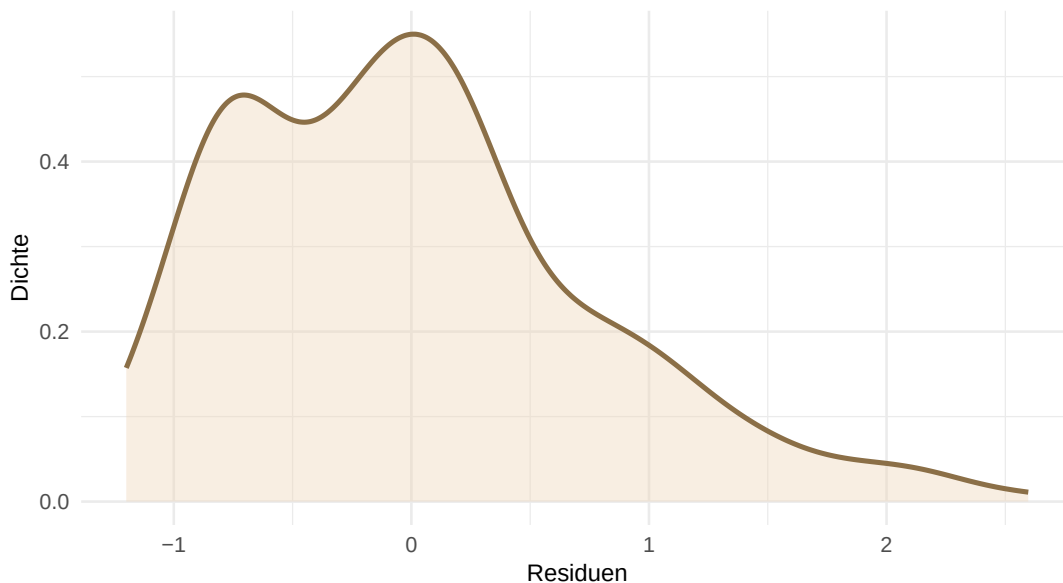
# Klassische Mixed ANOVA zunächst nur zur Annahmenprüfung schätzen
anova_staerke_schuh_geschlecht <- schaeetze_mixed_anova(
  inkonsistenz_staerke_schuh_geschlecht,
  "Staerke"
)

residuen_staerke_schuh_geschlecht <- residuals(
  anova_staerke_schuh_geschlecht$lm
)

zeige_und_speichere_grafik(
  density_residuen(
    residuen_staerke_schuh_geschlecht,
    "Faktorielle Mixed-ANOVA: Schuh-Stärke (2 Stufen) × Geschlecht"
  ),
  "Dichteplot der Residuen"
)

```

**Dichteplot der Residuen: Faktorielle
Mixed-ANOVA: Schuh-Stärke (2 Stufen) ×
Geschlecht**



```

shapiro_auto_08 <- shapiro_residuen(
  residuen_staerke_schuh_geschlecht
)
shapiro_auto_08

# Varianzhomogenität
levene_auto_07 <- levene_stufe(
  inkonsistenz_staerke_schuh_geschlecht, "Staerke", "baseline"
)
levene_auto_07

```

```

levene_auto_08 <- levene_stufe(
  inkonsistenz_staerke_schuh_geschlecht, "Staerke", "stark"
)
levene_auto_08

if (shapiro_auto_08$p.value > 0.05 &&
    levene_auto_07$`Pr(>F)`[1] > 0.05 &&
    levene_auto_08$`Pr(>F)`[1] > 0.05) {
  anova_staerke_schuh_geschlecht <- schaetze_mixed_anova(
    inkonsistenz_staerke_schuh_geschlecht,
    "Staerke"
  )
  anova_staerke_schuh_geschlecht
} else {
  # Robuste deskriptive Kennwerte: bwtrim() verwendet 20 % getrimmte Mittelwerte
  deskriptiv_staerke_schuh_robust <- inkonsistenz_staerke_schuh_geschlecht %>%
    group_by(Geschlecht, Staerke) %>%
    summarise(
      n = n(),
      Getrimmter_Mittelwert = mean(
        Mittelwert_Inkonsistenz,
        trim = 0.20,
        na.rm = TRUE
      ),
      Median = median(Mittelwert_Inkonsistenz, na.rm = TRUE),
      IQR = IQR(Mittelwert_Inkonsistenz, na.rm = TRUE),
      .groups = "drop"
    )

  deskriptiv_staerke_schuh_robust

  robust_anova_staerke_schuh_geschlecht <- bwtrim(
    Mittelwert_Inkonsistenz ~ Geschlecht * Staerke,
    id = number,
    data = inkonsistenz_staerke_schuh_geschlecht,
    tr = 0.20
  )
  robust_anova_staerke_schuh_geschlecht
}

```

Table 74: Deskriptive Kennwerte – Schuh-Stärke (2 Stufen)
× Geschlecht - Teil 1 von 2

Geschlecht	Staerke	n	Median	IQR
weiblich	baseline	76	1.667	1.000
weiblich	stark	76	2.000	1.000
männlich	baseline	55	1.667	1.000
männlich	stark	55	2.000	1.333

Table 75: Deskriptive Kennwerte – Schuh-Stärke (2 Stufen)
× Geschlecht - Teil 2 von 2

Geschlecht	Mittelwert	SD
weiblich	1.737	0.706

Geschlecht	Mittelwert	SD
weiblich	1.961	0.726
männlich	1.788	0.771
männlich	2.200	0.862

Table 76: Shapiro-Wilk-Test der Residuen – Schuh-Stärke (2 Stufen) $\times$ Geschlecht

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.95
p-Wert	< 1e-04

Table 77: Levene-Test – M nach Geschlecht

Df	F value	Pr(>F)
1	0.091	0.764
129		

Table 78: Levene-Test – KDS nach Geschlecht

Df	F value	Pr(>F)
1	1.644	0.202
129		

Table 79: Robuste deskriptive Kennwerte – Schuh-Stärke (2 Stufen) $\times$ Geschlecht - Teil 1 von 2

Geschlecht	Staerke	n	Getrimmter_Mittelwert	Median
weiblich	baseline	76	1.623	1.667
weiblich	stark	76	1.899	2.000
männlich	baseline	55	1.657	1.667
männlich	stark	55	2.141	2.000

Table 80: Robuste deskriptive Kennwerte – Schuh-Stärke (2 Stufen) $\times$ Geschlecht - Teil 2 von 2

Geschlecht	IQR
weiblich	1.000
weiblich	1.000
männlich	1.000

Geschlecht	IQR
männlich	1.333

Table 81: Robuste Mixed-ANOVA – Schuh-Stärke (2 Stufen)
× Geschlecht

Ausgabe				
Call:				
bwtrim(formula = Mittelwert_Inkonsistenz ~ Geschlecht * Staerke, id = number, data = inkonsistenz_staerke_schuh_geschlecht, tr = 0.2)				
	value	df1	df2	p.value
Geschlecht	1.0987	1	78.9473	0.2977
Staerke	19.0902	1	79.6524	0.0000
Geschlecht:Staerke	1.4496	1	79.6524	0.2322

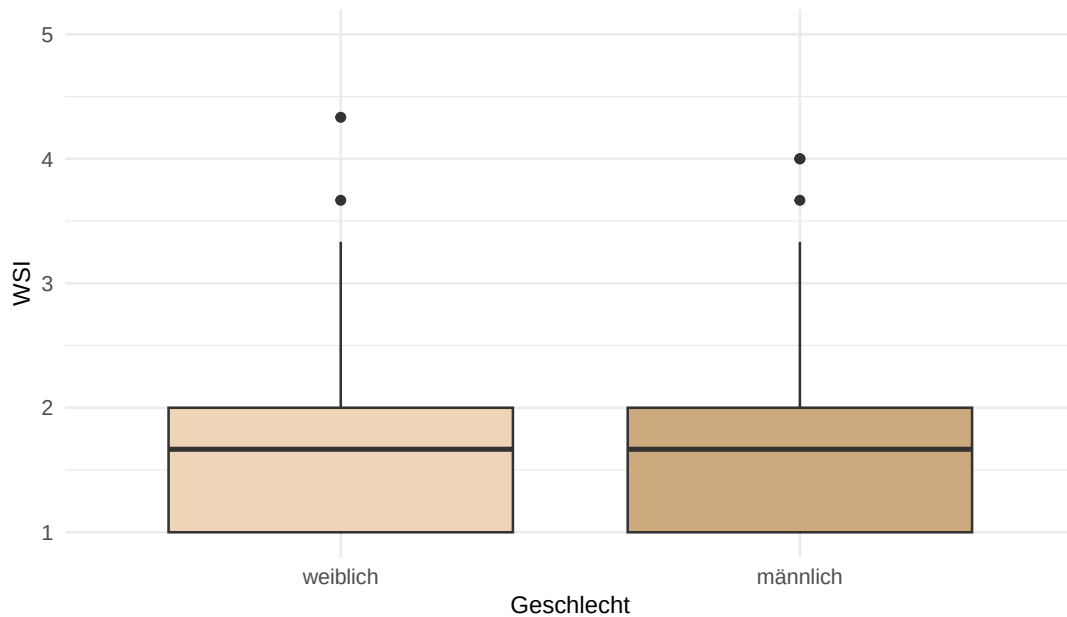
7.6 Stärke (3 Stufen) × Geschlecht

```
# 5. Stärke mit drei Stufen × Geschlecht
inkonsistenz_staerke3_geschlecht <- bereite_geschlechtsdaten_vor(
  conditions = c(
    "Min-Baseline",
    "Jacke-leicht-dezentral",
    "Jacke-stark-dezentral"
  ),
  faktor = "Staerke",
  zuordnung = c(
    "Min-Baseline" = "baseline",
    "Jacke-leicht-dezentral" = "leicht",
    "Jacke-stark-dezentral" = "stark"
  ),
  stufen = c("baseline", "leicht", "stark")
)

# Kontrolle
table(inkonsistenz_staerke3_geschlecht$number)
inkonsistenz_staerke3_geschlecht %>%
  distinct(number, Geschlecht) %>%
  count(Geschlecht)

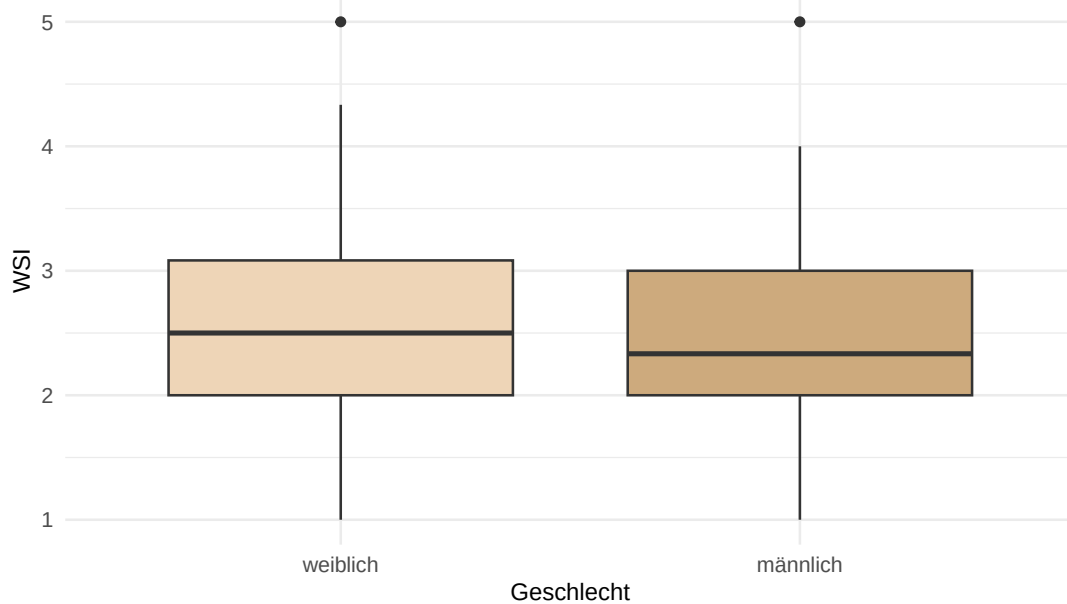
# Boxplots
boxplot_staerke3_baseline <- erstelle_boxplot_geschlecht(
  inkonsistenz_staerke3_geschlecht, "Staerke", "baseline",
  "Stärke (3 Stufen) - Baseline: M"
)
speichere_grafik(boxplot_staerke3_baseline, "WSI nach Geschlecht")
print(boxplot_staerke3_baseline)
```

WSI nach Geschlecht: Stärke (3 Stufen) – Baseline: M



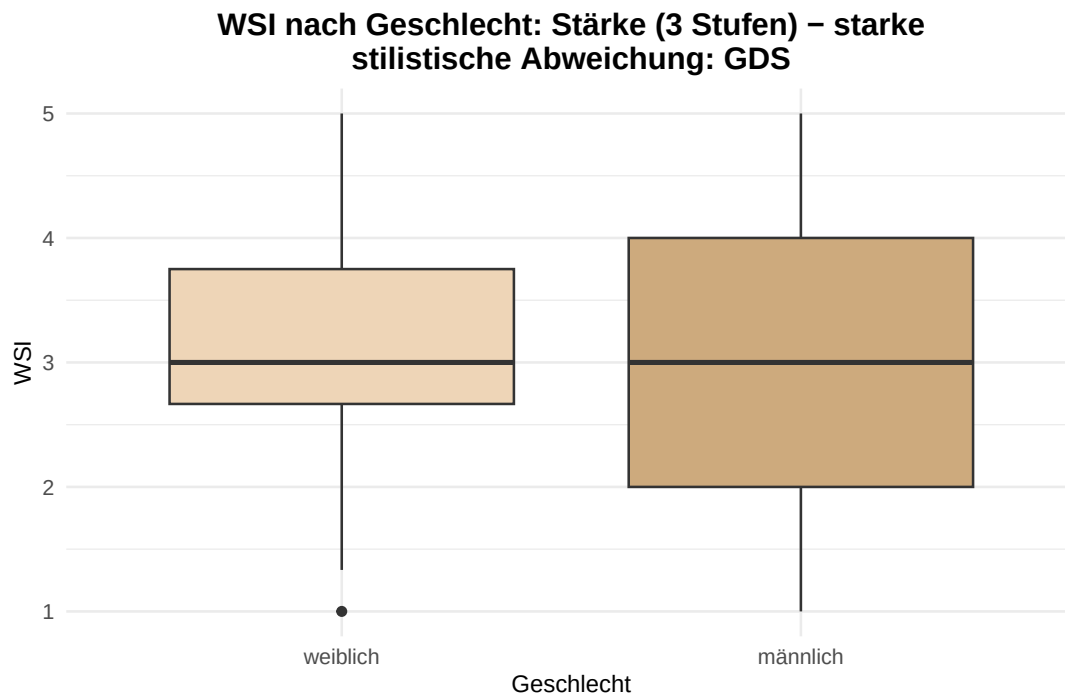
```
boxplot_staerke3_leicht <- erstelle_boxplot_geschlecht(
  inkonsistenz_staerke3_geschlecht, "Staerke", "leicht",
  "Stärke (3 Stufen) - leichte stilistische Abweichung: GDL"
)
speichere_grafik(boxplot_staerke3_leicht, "WSI nach Geschlecht")
print(boxplot_staerke3_leicht)
```

WSI nach Geschlecht: Stärke (3 Stufen) – leichte stilistische Abweichung: GDL



```
boxplot_staerke3_stark <- erstelle_boxplot_geschlecht(
  inkonsistenz_staerke3_geschlecht, "Staerke", "stark",
  "Stärke (3 Stufen) - starke stilistische Abweichung: GDS"
)
speichere_grafik(boxplot_staerke3_stark, "WSI nach Geschlecht")
```

```
print(boxplot_staerke3_stark)
```



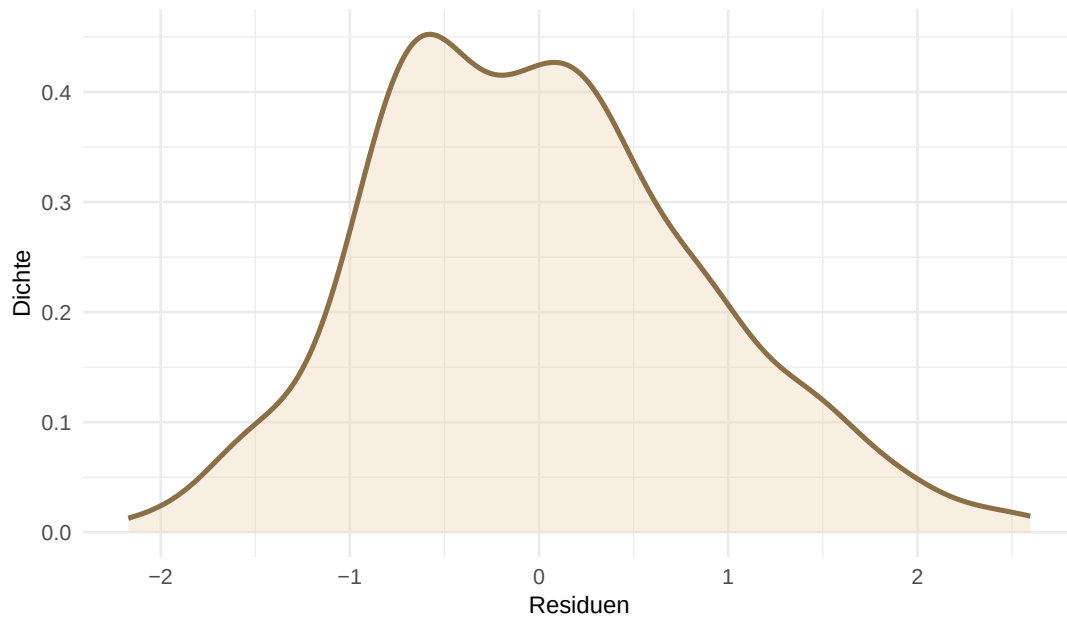
```
# Deskriptive Kennwerte
deskriptiv_staerke3_geschlecht <- deskriptiv_geschlecht(
  inkonsistenz_staerke3_geschlecht, "Staerke"
)
deskriptiv_staerke3_geschlecht

# Modell zunächst schätzen, um Residuen und Sphärizität zu prüfen
anova_staerke3_geschlecht <- schaeetze_mixed_anova(
  inkonsistenz_staerke3_geschlecht,
  "Staerke"
)

residuen_staerke3_geschlecht <- residuals(anova_staerke3_geschlecht$lm)

zeige_und_speichere_grafik(
  density_residuen(
    residuen_staerke3_geschlecht,
    "Faktorielle Mixed-ANOVA: Stärke (3 Stufen) × Geschlecht"
  ),
  "Dichteplot der Residuen"
)
```

Dichteplot der Residuen: Faktorielle Mixed-ANOVA: Stärke (3 Stufen) × Geschlecht



```
shapiro_auto_09 <- shapiro_residuen(
  residuen_staerke3_geschlecht
)
shapiro_auto_09

# Varianzhomogenität für alle drei Stufen
levene_auto_09 <- levene_stufe(
  inkonsistenz_staerke3_geschlecht, "Staerke", "baseline"
)
levene_auto_09

levene_auto_10 <- levene_stufe(
  inkonsistenz_staerke3_geschlecht, "Staerke", "leicht"
)
levene_auto_10

levene_auto_11 <- levene_stufe(
  inkonsistenz_staerke3_geschlecht, "Staerke", "stark"
)
levene_auto_11

# Sphärizität prüfen
summary_anova_staerke3_geschlecht <- summary(anova_staerke3_geschlecht)
summary_anova_staerke3_geschlecht

if (shapiro_auto_09$p.value > 0.05 &&
    levene_auto_09$`Pr(>F)`[1] > 0.05 &&
    levene_auto_10$`Pr(>F)`[1] > 0.05 &&
    levene_auto_11$`Pr(>F)`[1] > 0.05) {
  anova_staerke3_geschlecht <- schaetze_mixed_anova(
    inkonsistenz_staerke3_geschlecht,
    "Staerke"
  )
  anova_staerke3_geschlecht
} else {
```



```
robust_anova_staerke3_geschlecht <- bwtrim(
  Mittelwert_Inkonsistenz ~ Geschlecht * Staerke,
  id = number,
  data = inkonsistenz_staerke3_geschlecht,
  tr = 0.20
)
robust_anova_staerke3_geschlecht
}
```

Table 82: Deskriptive Kennwerte – Stärke (3 Stufen) × Geschlecht - Teil 1 von 2

Geschlecht	Staerke	n	Median	IQR
weiblich	baseline	76	1.667	1.000
weiblich	leicht	76	2.500	1.083
weiblich	stark	76	3.000	1.083
männlich	baseline	55	1.667	1.000
männlich	leicht	55	2.333	1.000
männlich	stark	55	3.000	2.000

Table 83: Deskriptive Kennwerte – Stärke (3 Stufen) × Geschlecht - Teil 2 von 2

Geschlecht	Mittelwert	SD
weiblich	1.737	0.706
weiblich	2.526	0.954
weiblich	3.171	0.892
männlich	1.788	0.771
männlich	2.527	0.893
männlich	3.012	0.973

Table 84: Shapiro-Wilk-Test der Residuen – Stärke (3 Stufen) × Geschlecht

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.985
p-Wert	0.000348

Table 85: Levene-Test – M nach Geschlecht

Df	F value	Pr(>F)
1	0.091	0.764
129		

Table 86: Levene-Test – GDL nach Geschlecht

Df	F value	Pr(>F)
1	0.287	0.593
129		

Table 87: Levene-Test – GDS nach Geschlecht

Df	F value	Pr(>F)
1	1.544	0.216
129		

Table 88: ANOVA-Zusammenfassung einschließlich Sphäritätsprüfung

Ausgabe

Univariate Type III Repeated-Measures ANOVA Assuming Sphericity

Sum Sq num Df Error SS den Df F value Pr(>F)
 (Intercept) 2317.63 1 158.33 129 1888.2475 <2e-16 ***
 Geschlecht 0.12 1 158.33 129 0.0991 0.7534
 Staerke 113.60 2 133.24 258 109.9857 <2e-16 ***
 Geschlecht:Staerke 0.77 2 133.24 258 0.7431 0.4767

—
 Signif. codes: 0 ‘’ **0.001** ’’ 0.01 ’’ 0.05 ‘ 0.1 ’ ’ 1

Mauchly Tests for Sphericity

Test statistic p-value
 Staerke 0.97012 0.14354
 Geschlecht:Staerke 0.97012 0.14354

Greenhouse-Geisser and Huynh-Feldt Corrections
for Departure from Sphericity

GG eps Pr(>F[GG])
 Staerke 0.97099 <2e-16 ***
 Geschlecht:Staerke 0.97099 0.473

—
 Signif. codes: 0 ‘’ **0.001** ’’ 0.01 ’’ 0.05 ‘ 0.1 ’ ’ 1

HF eps Pr(>F[HF])
 Staerke 0.9856038 8.398107e-35

Ausgabe

Geschlecht:Staerke 0.9856038 4.748766e-01

Table 89: Robuste Mixed-ANOVA – Stärke (3 Stufen) × Geschlecht

Ausgabe

Call:

bwtrim(formula = Mittelwert_Inkonsistenz ~ Geschlecht * Staerke,
id = number, data = inkonsistenz_staerke3_geschlecht, tr = 0.2)

	value	df1	df2	p.value
Geschlecht	0.3486	1	81.6806	0.5565
Staerke	97.8184	2	71.3608	0.0000
Geschlecht:Staerke	0.7578	2	71.3608	0.4724

8 Unterschiede nach CVPA

8.1 CVPA-Gruppen und Hilfsfunktionen

```
# CVPA-Unterschiede
# CVPA-Mittelwert und Gruppen
salienz_reshaped <- salienz_reshaped %>%
  mutate(Mittelwert_CVPA = rowMeans(across(CVPA1:CVPA9), na.rm = TRUE))

cvpa_personen <- salienz_reshaped %>%
  distinct(number, Mittelwert_CVPA)

cvpa_gesamtittelwert <- mean(cvpa_personen$Mittelwert_CVPA, na.rm = TRUE)
cvpa_gesamtittelwert

cvpa_personen <- cvpa_personen %>%
  mutate(
    CVPA_Gruppe = case_when(
      Mittelwert_CVPA < cvpa_gesamtittelwert ~ "unter Mittelwert",
      Mittelwert_CVPA > cvpa_gesamtittelwert ~ "über Mittelwert",
      Mittelwert_CVPA == cvpa_gesamtittelwert ~ "genau Mittelwert"
    )
  )

cvpa_personen %>% count(CVPA_Gruppe)

salienz_reshaped <- salienz_reshaped %>%
  left_join(
    cvpa_personen %>% dplyr::select(number, CVPA_Gruppe),
    by = "number"
  )

salienz_reshaped %>%
  distinct(number, Mittelwert_CVPA, CVPA_Gruppe) %>%
  count(CVPA_Gruppe)
```

```

# Hilfsfunktionen
bereite_cvpa_daten_vor <- function(conditions, faktor, zuordnung, stufen) {
  salienz_reshaped %>%
    filter(Condition %in% conditions) %>%
    mutate(
      "{faktor}" := unname(zuordnung[as.character(Condition)]),
      CVPA_Gruppe = factor(
        CVPA_Gruppe,
        levels = c("unter Mittelwert", "über Mittelwert")
      ),
      "{faktor}" := factor(.data[[faktor]], levels = stufen)
    ) %>%
    filter(
      !is.na(CVPA_Gruppe),
      !is.na(.data[[faktor]]),
      !is.na(Mittelwert_Inkonsistenz)
    ) %>%
    group_by(number) %>%
    filter(n_distinct(.data[[faktor]]) == length(stufen)) %>%
    ungroup()
}

erstelle_cvpa_boxplot <- function(data, faktor, stufe, subtitle) {
  data %>%
    filter(.data[[faktor]] == stufe) %>%
    ggplot(
      aes(
        x = CVPA_Gruppe,
        y = Mittelwert_Inkonsistenz,
        fill = CVPA_Gruppe
      )
    ) +
    geom_boxplot() +
    scale_fill_manual(
      values = c(
        "unter Mittelwert" = "#EED5B7",
        "über Mittelwert" = "#CDAA7D"
      )
    ) +
    scale_y_continuous(breaks = 1:5, limits = c(1, 5)) +
    labs(
      title = stringr::str_wrap(paste("Boxplot der WSI -", subtitle), width = 48),
      x = "CVPA-Gruppe",
      y = "WSI"
    ) +
    theme_box_density_gross +
    theme(
      legend.position = "none",
      plot.title = element_text(hjust = 0.5)
    )
}

deskriptiv_cvpa <- function(data, faktor) {
  data %>%
    group_by(CVPA_Gruppe, !!rlang::sym(faktor)) %>%
    summarise(
      n = n(),
      Median = median(Mittelwert_Inkonsistenz, na.rm = TRUE),
      IQR = IQR(Mittelwert_Inkonsistenz, na.rm = TRUE),
      Mittelwert = mean(Mittelwert_Inkonsistenz, na.rm = TRUE),
      SD = sd(Mittelwert_Inkonsistenz, na.rm = TRUE),

```

```

    .groups = "drop"
  )
}

deskriptiv_cvpa_robust <- function(data, faktor) {
  data %>%
    group_by(CVPA_Gruppe, !!rlang::sym(faktor)) %>%
    summarise(
      n = n(),
      Getrimmter_Mittelwert = mean(
        Mittelwert_Inkonsistenz,
        trim = 0.20,
        na.rm = TRUE
      ),
      Median = median(Mittelwert_Inkonsistenz, na.rm = TRUE),
      IQR = IQR(Mittelwert_Inkonsistenz, na.rm = TRUE),
      .groups = "drop"
    )
}

schaetze_cvpa_anova <- function(data, faktor) {
  modell <- aov_ez(
    id = "number",
    dv = "Mittelwert_Inkonsistenz",
    within = faktor,
    between = "CVPA_Gruppe",
    data = data
  )
  modell
}

zeige_residuenplots_cvpa <- function(residuen, subtitle) {
  zeige_und_speichere_grafik(
    ggplot(
      data.frame(Residuen = as.numeric(residuen)),
      aes(x = Residuen)
    ) +
    geom_density(
      fill = "#EED5B7",
      color = "#8B6F47",
      alpha = 0.4,
      linewidth = 1
    ) +
    labs(
      title = stringr::str_wrap(paste("Dichteplot der Residuen:", subtitle), width = 48),
      x = "Residuen",
      y = "Dichte"
    ) +
    theme_box_density_gross +
    theme(
      plot.title = element_text(hjust = 0.5),
      plot.subtitle = element_text(hjust = 0.5)
    ),
    "Dichteplot der Residuen"
  )
}

shapiro_residuen_cvpa <- function(residuen) {
  test <- shapiro.test(as.numeric(residuen))
  test
}

```

```

}

levene_cvpa <- function(data, faktor, stufe) {
  test <- leveneTest(
    Mittelwert_Inkonsistenz ~ CVPA_Gruppe,
    data = data %>% filter(.data[[faktor]] == stufe)
  )
  test
}

```

Table 90: Verteilung der CVPA-Gruppen

CVPA_Gruppe	n
unter Mittelwert	63
über Mittelwert	72

Table 91: CVPA-Gesamtmittelwert

Kennwert	Wert
Gesamtmittelwert CVPA	3.343

8.2 Position $\times$ CVPA

```

# 1. Position  $\times$  CVPA
inkonsistenz_position_cvpa <- bereite_cvpa_daten_vor(
  conditions = c("Jacke-stark-dezentral", "Jacke-stark-zentral"),
  faktor = "Position",
  zuordnung = c(
    "Jacke-stark-dezentral" = "dezentral",
    "Jacke-stark-zentral" = "zentral"
  ),
  stufen = c("dezentral", "zentral")
)

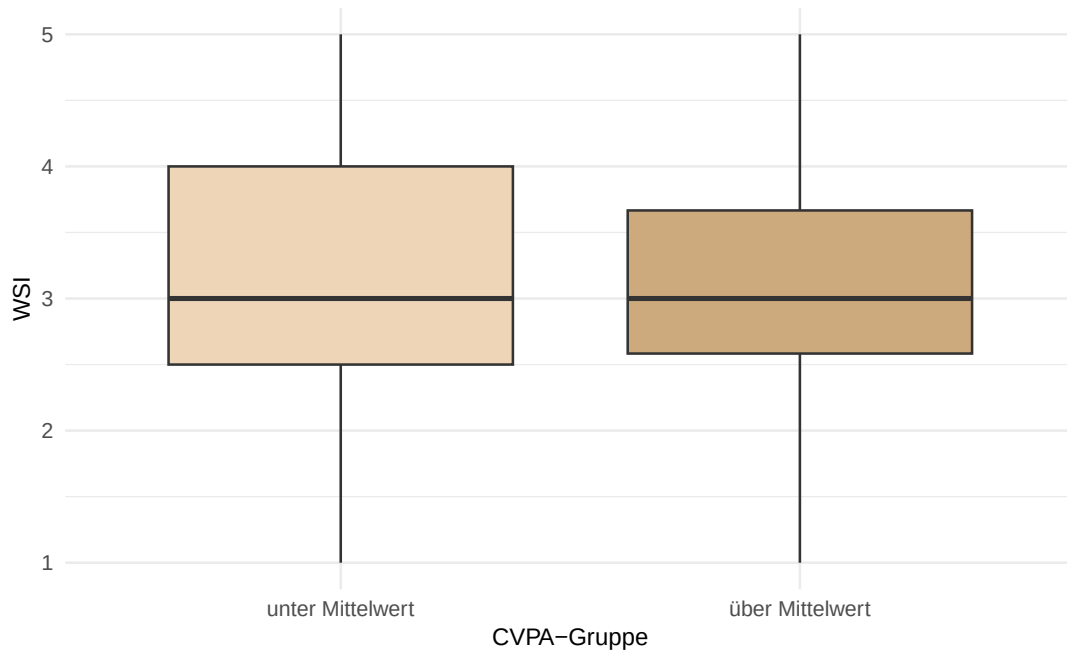
table(inkonsistenz_position_cvpa$number)

boxplot_position_dezentral_cvpa <- erstelle_cvpa_boxplot(
  inkonsistenz_position_cvpa,
  "Position",
  "dezentral",
  "GDS"
)

speichere_grafik(boxplot_position_dezentral_cvpa, "WSI nach CVPA")
print(boxplot_position_dezentral_cvpa)

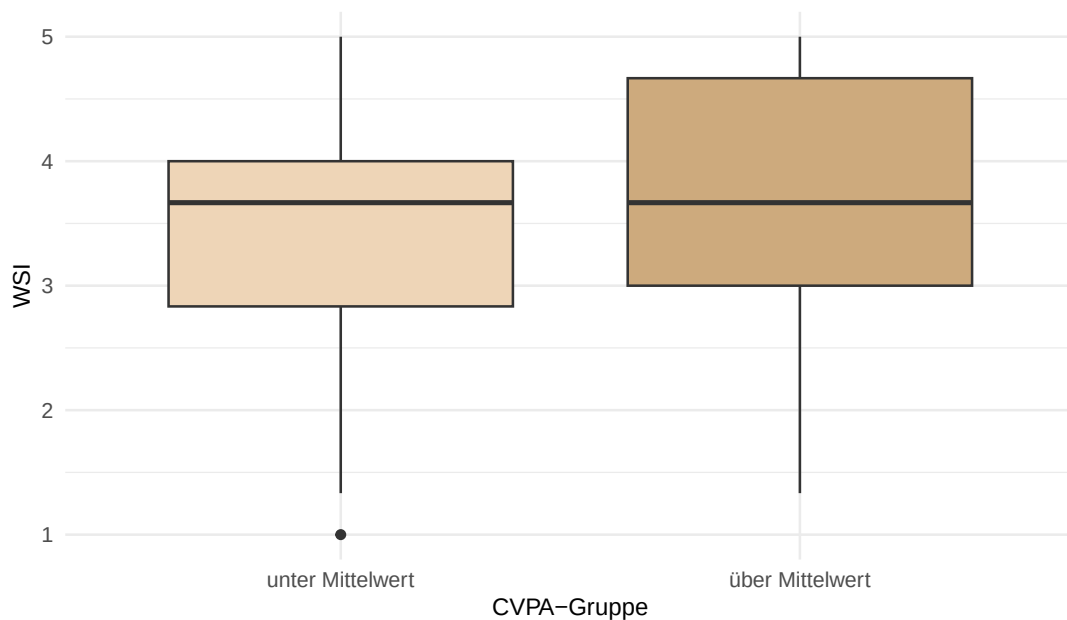
```

Boxplot der WSI – GDS



```
boxplot_position_zentral_cvpa <- erstelle_cvpa_boxplot(
  inkonsistenz_position_cvpa,
  "Position",
  "zentral",
  "Starke stilistische Abweichung: Jacke zentral"
)
speichere_grafik(boxplot_position_zentral_cvpa, "WSI nach CVPA")
print(boxplot_position_zentral_cvpa)
```

Boxplot der WSI – Starke stilistische Abweichung: Jacke zentral



```
deskriptiv_position_cvpa <- deskriptiv_cvpa(
  inkonsistenz_position_cvpa,
  "Position"
```

```

)
deskriptiv_position_cvpa

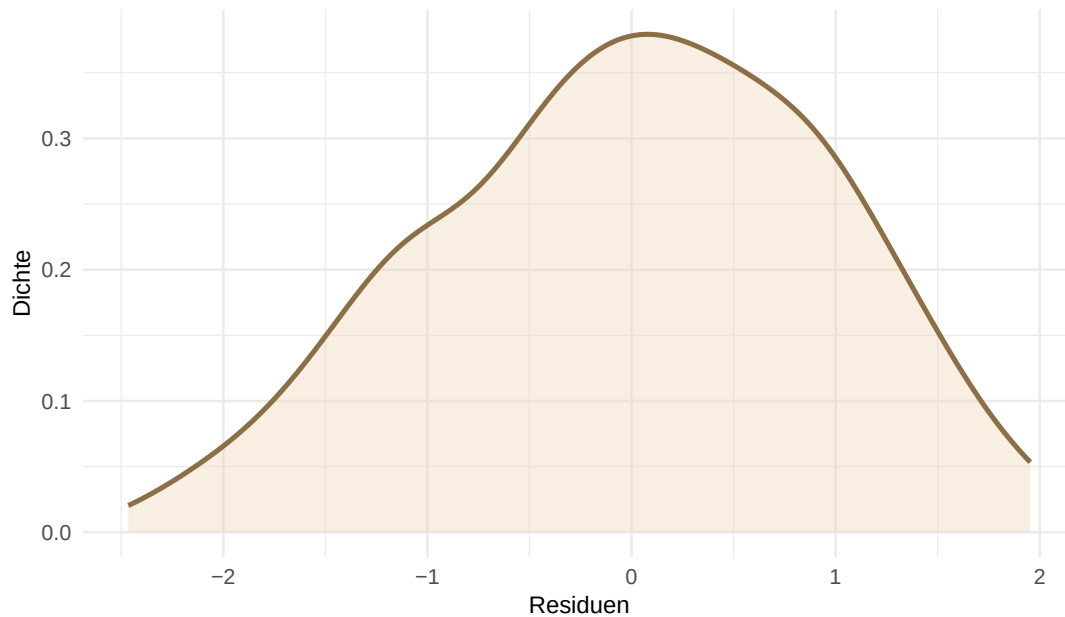
# Modell zunächst schätzen und Voraussetzungen prüfen
anova_position_cvpa <- schaetze_cvpa_anova(
  inkonsistenz_position_cvpa,
  "Position"
)

residuen_position_cvpa <- residuals(anova_position_cvpa$lm)

# Density Plot
zeige_und_speichere_grafik(
  ggplot(
    data.frame(Residuen = as.numeric(residuen_position_cvpa)),
    aes(x = Residuen)
  ) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",
    alpha = 0.4,
    linewidth = 1
  ) +
  labs(
    title = stringr::str_wrap(
      "Dichteplot der Residuen: Faktorielle Mixed-ANOVA Position × CVPA",
      width = 52),
    x = "Residuen",
    y = "Dichte"
  ) +
  theme_box_density_gross +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  ),
  "Dichteplot der Residuen"
)

```


Dichteplot der Residuen: Faktorielle Mixed-ANOVA Position × CVPA



```
shapiro_auto_10 <- shapiro_residuen_cvpa(
  residuen_position_cvpa
)
shapiro_auto_10

levene_auto_12 <- levene_cvpa(
  inkonsistenz_position_cvpa, "Position", "dezentral"
)
levene_auto_12

levene_auto_13 <- levene_cvpa(
  inkonsistenz_position_cvpa, "Position", "zentral"
)
levene_auto_13

if (shapiro_auto_10$p.value > 0.05 &&
    levene_auto_12$`Pr(>F)`[1] > 0.05 &&
    levene_auto_13$`Pr(>F)`[1] > 0.05) {
  anova_position_cvpa <- schaetze_cvpa_anova(
    inkonsistenz_position_cvpa,
    "Position"
  )
  anova_position_cvpa
} else {
  robust_anova_position_cvpa <- bwtrim(
    Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Position,
    id = number,
    data = inkonsistenz_position_cvpa,
    tr = 0.20
  )
  robust_anova_position_cvpa
}
```

Table 92: Deskriptive Kennwerte – Position $\times$ CVPA - Teil
1 von 2

CVPA_Gruppe	Position	n	Median	IQR
unter Mittelwert	dezentral	63	3.000	1.500
unter Mittelwert	zentral	63	3.667	1.167
über Mittelwert	dezentral	72	3.000	1.083
über Mittelwert	zentral	72	3.667	1.667

Table 93: Deskriptive Kennwerte – Position $\times$ CVPA - Teil
2 von 2

CVPA_Gruppe	Mittelwert	SD
unter Mittelwert	3.164	0.894
unter Mittelwert	3.466	0.936
über Mittelwert	3.046	0.937
über Mittelwert	3.588	1.041

Table 94: Shapiro-Wilk-Test der Residuen – Position $\times$
CVPA

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.987
p-Wert	0.0179

Table 95: Levene-Test – GDS nach CVPA

Df	F value	Pr(>F)
1	0.019	0.892
133		

Table 96: Levene-Test – GZS nach CVPA

Df	F value	Pr(>F)
1	1.756	0.187
133		

Table 97: Robuste Mixed-ANOVA – Position $\times$ CVPA

Ausgabe

Call:

```
bwtrim(formula = Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Position,
id = number, data = inkonsistenz_position_cvpa, tr = 0.2)
```

	value	df1	df2	p.value
CVPA_Gruppe	0.0000	1	75.9805	0.9979
Position	19.0095	1	71.7616	0.0000
CVPA_Gruppe:Position	1.8564	1	71.7616	0.1773

8.3 Größe $\times$ CVPA

```
# 2. Größe  $\times$  CVPA
inkonsistenz_groesse_cvpa <- bereite_cvpa_daten_vor(
  conditions = c("Schuh-stark-dezentral", "Jacke-stark-dezentral"),
  faktor = "Groesse",
  zuordnung = c(
    "Schuh-stark-dezentral" = "klein",
    "Jacke-stark-dezentral" = "gross"
  ),
  stufen = c("klein", "gross")
)

table(inkonsistenz_groesse_cvpa$number)

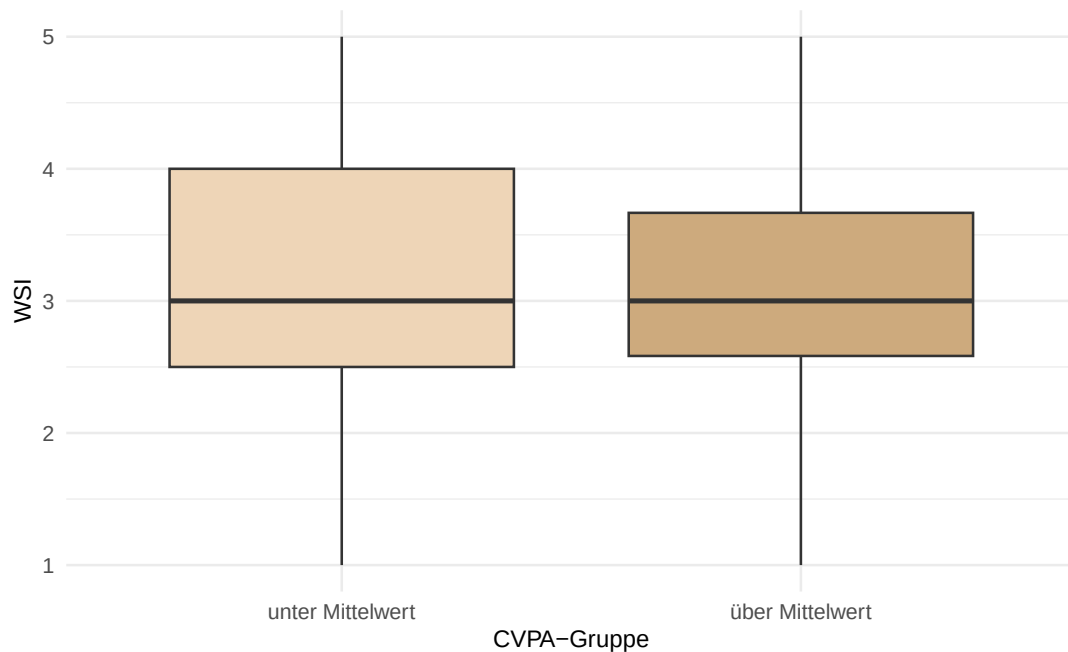
boxplot_groesse_klein_cvpa <- erstelle_cvpa_boxplot(
  inkonsistenz_groesse_cvpa,
  "Groesse",
  "klein",
  "KDS"
)
speichere_grafik(boxplot_groesse_klein_cvpa, "WSI nach CVPA")
print(boxplot_groesse_klein_cvpa)
```

Boxplot der WSI – KDS



```
boxplot_groesse_gross_cvpa <- erstelle_cvpa_boxplot(  
  inkonsistenz_groesse_cvpa,  
  "Groesse",  
  "gross",  
  "GDS"  
)  
speichere_grafik(boxplot_groesse_gross_cvpa, "WSI nach CVPA")  
print(boxplot_groesse_gross_cvpa)
```

Boxplot der WSI – GDS



```
deskriptiv_groesse_cvpa <- deskriptiv_cvpa(  
  inkonsistenz_groesse_cvpa,  
  "Groesse"
```

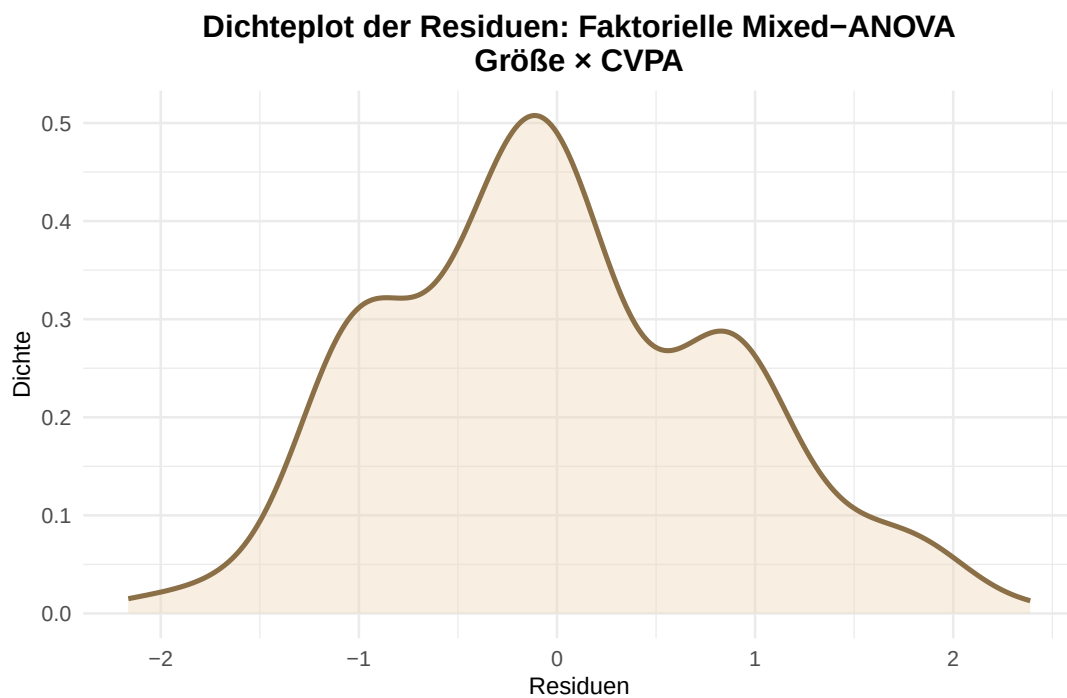
```

)
deskriptiv_groesse_cvpa

anova_groesse_cvpa <- schaetze_cvpa_anova(
  inkonsistenz_groesse_cvpa,
  "Groesse"
)
residuen_groesse_cvpa <- residuals(anova_groesse_cvpa$lm)

# Density Plot
zeige_und_speichere_grafik(
  ggplot(
    data.frame(Residuen = as.numeric(residuen_groesse_cvpa)),
    aes(x = Residuen)
  ) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",
    alpha = 0.4,
    linewidth = 1
  ) +
  labs(
    title = stringr::str_wrap(
      "Dichteplot der Residuen: Faktorielle Mixed-ANOVA Größe × CVPA",
      width = 52),
    x = "Residuen",
    y = "Dichte"
  ) +
  theme_box_density_gross +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  ),
  "Dichteplot der Residuen"
)

```



```

shapiro_auto_11 <- shapiro_residuen_cvpa(
  residuen_groesse_cvpa
)
shapiro_auto_11

levene_auto_14 <- levene_cvpa(
  inkonsistenz_groesse_cvpa, "Groesse", "klein"
)
levene_auto_14

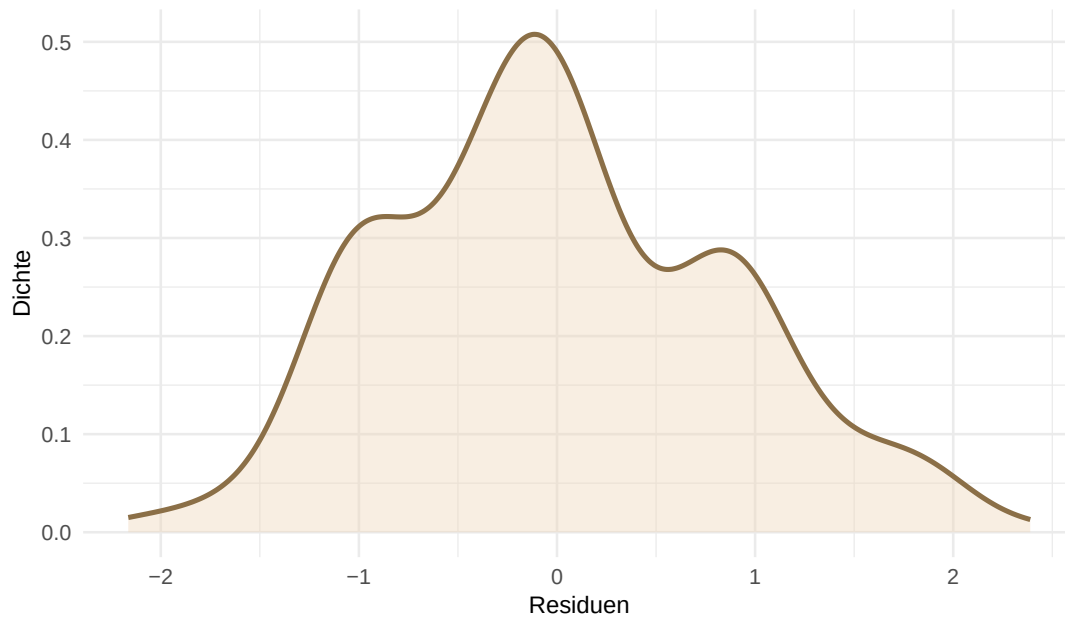
levene_auto_15 <- levene_cvpa(
  inkonsistenz_groesse_cvpa, "Groesse", "gross"
)
levene_auto_15

anova_groesse_cvpa <- schaeetze_cvpa_anova(
  inkonsistenz_groesse_cvpa,
  "Groesse"
)
residuen_groesse_cvpa <- residuals(anova_groesse_cvpa$lm)

# Density Plot
zeige_und_speichere_grafik(
  ggplot(
    data.frame(Residuen = as.numeric(residuen_groesse_cvpa)),
    aes(x = Residuen)
  ) +
    geom_density(
      fill = "#EED5B7",
      color = "#8B6F47",
      alpha = 0.4,
      linewidth = 1
    ) +
    labs(
      title = stringr::str_wrap(
        "Dichteplot der Residuen: Faktorielle Mixed-ANOVA Größe × CVPA",
        width = 52),
      x = "Residuen",
      y = "Dichte"
    ) +
    theme_box_density_gross +
    theme(
      plot.title = element_text(hjust = 0.5),
      plot.subtitle = element_text(hjust = 0.5)
    ),
  "Dichteplot der Residuen"
)

```

Dichteplot der Residuen: Faktorielle Mixed-ANOVA Größe × CVPA



```
shapiro_auto_12 <- shapiro_residuen_cvpa(
  residuen_groesse_cvpa
)
shapiro_auto_12

levene_auto_16 <- levene_cvpa(
  inkonsistenz_groesse_cvpa, "Groesse", "klein"
)
levene_auto_16

levene_auto_17 <- levene_cvpa(
  inkonsistenz_groesse_cvpa, "Groesse", "gross"
)
levene_auto_17

if (shapiro_auto_12$p.value > 0.05 &&
    levene_auto_16$`Pr(>F)`[1] > 0.05 &&
    levene_auto_17$`Pr(>F)`[1] > 0.05) {
  anova_groesse_cvpa <- schaetze_cvpa_anova(
    inkonsistenz_groesse_cvpa,
    "Groesse"
  )
  anova_groesse_cvpa
} else {
  deskriptiv_groesse_cvpa_robust <- deskriptiv_cvpa_robust(
    inkonsistenz_groesse_cvpa,
    "Groesse"
  )
  deskriptiv_groesse_cvpa_robust

  robust_anova_groesse_cvpa <- bwtrim(
    Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Groesse,
    id = number,
    data = inkonsistenz_groesse_cvpa,
    tr = 0.20
  )
}
```

```

)
robust_anova_groesse_cvpa
}

```

Table 98: Deskriptive Kennwerte – Größe $\times$ CVPA - Teil 1 von 2

CVPA_Gruppe	Groesse	n	Median	IQR
unter Mittelwert	klein	63	2.000	0.833
unter Mittelwert	gross	63	3.000	1.500
über Mittelwert	klein	72	1.833	1.417
über Mittelwert	gross	72	3.000	1.083

Table 99: Deskriptive Kennwerte – Größe $\times$ CVPA - Teil 2 von 2

CVPA_Gruppe	Mittelwert	SD
unter Mittelwert	2.159	0.752
unter Mittelwert	3.164	0.894
über Mittelwert	1.944	0.832
über Mittelwert	3.046	0.937

Table 100: Shapiro-Wilk-Test der Residuen – Größe $\times$ CVPA

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.983
p-Wert	0.002271

Table 101: Levene-Test – KDS nach CVPA

Df	F value	Pr(>F)
1	1.436	0.233
133		

Table 102: Levene-Test – GDS nach CVPA

Df	F value	Pr(>F)
1	0.019	0.892
133		

Table 103: Robuste deskriptive Kennwerte – Größe × CVPA
- Teil 1 von 2

CVPA_Gruppe	Groesse	n	Getrimmter_Mittelwert	Median
unter Mittelwert	klein	63	2.111	2.000
unter Mittelwert	gross	63	3.197	3.000
über Mittelwert	klein	72	1.826	1.833
über Mittelwert	gross	72	3.045	3.000

Table 104: Robuste deskriptive Kennwerte – Größe × CVPA
- Teil 2 von 2

CVPA_Gruppe	IQR
unter Mittelwert	0.833
unter Mittelwert	1.500
über Mittelwert	1.417
über Mittelwert	1.083

Table 105: Robuste Mixed-ANOVA – Größe × CVPA

Ausgabe				
Call:				
bwtrim(formula = Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Groesse, id = number, data = inkonsistenz_groesse_cvpa, tr = 0.2)				
value	df1	df2	p.value	
CVPA_Gruppe	2.4508	1	74.2017	0.1217
Groesse	106.5921	1	75.9501	0.0000
CVPA_Gruppe:Groesse	0.3614	1	75.9501	0.5495

8.4 Stärke (2 Stufen) × CVPA

```
# 3. Stärke × CVPA
inkonsistenz_staerke_cvpa <- bereite_cvpa_daten_vor(
  conditions = c("Jacke-leicht-dezentral", "Jacke-stark-dezentral"),
  faktor = "Staerke",
  zuordnung = c(
    "Jacke-leicht-dezentral" = "leicht",
    "Jacke-stark-dezentral" = "stark"
  ),
  stufen = c("leicht", "stark")
)

table(inkonsistenz_staerke_cvpa$number)

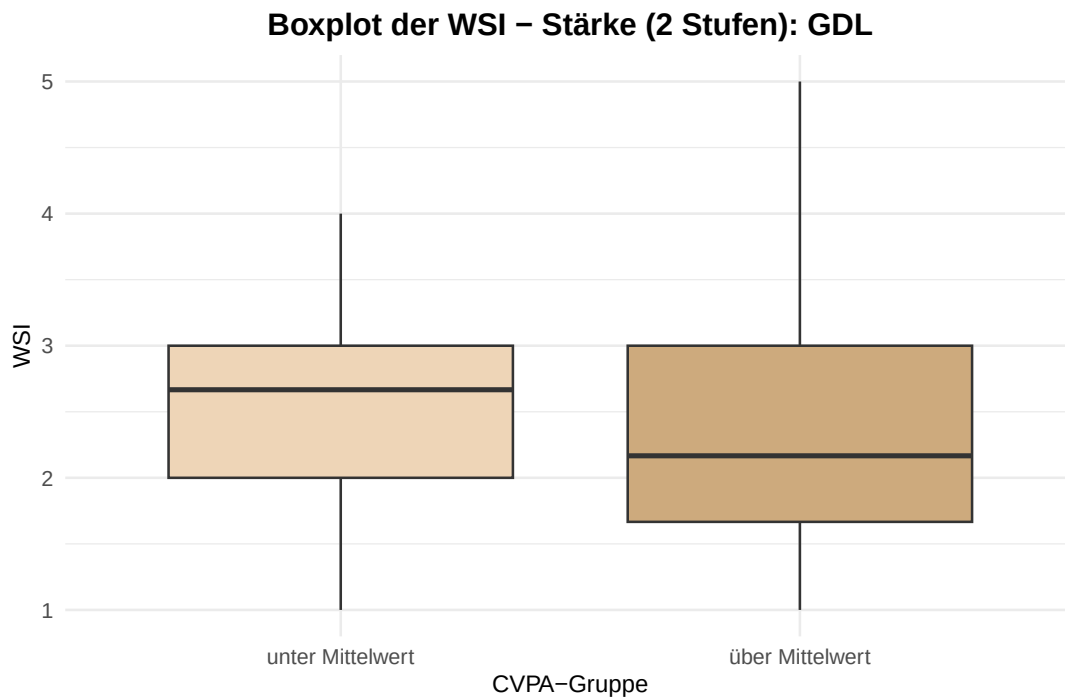
inkonsistenz_staerke_cvpa %>%
  distinct(number, CVPA_Gruppe) %>%
```

```

count(CVPA_Gruppe)

boxplot_staerke_leicht_cvpa <- erstelle_cvpa_boxplot(
  inkonsistenz_staerke_cvpa,
  "Staerke",
  "leicht",
  "Stärke (2 Stufen): GDL"
)
speichere_grafik(boxplot_staerke_leicht_cvpa, "WSI nach CVPA")
print(boxplot_staerke_leicht_cvpa)

```

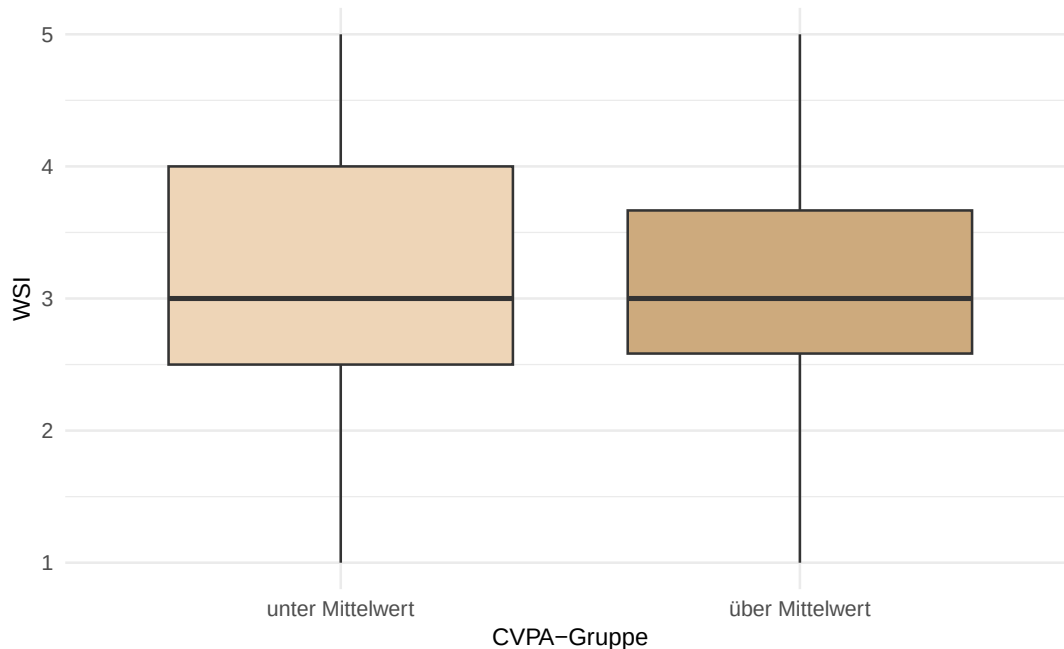


```

boxplot_staerke_stark_cvpa <- erstelle_cvpa_boxplot(
  inkonsistenz_staerke_cvpa,
  "Staerke",
  "stark",
  "Stärke (2 Stufen): GDS"
)
speichere_grafik(boxplot_staerke_stark_cvpa, "WSI nach CVPA")
print(boxplot_staerke_stark_cvpa)

```

Boxplot der WSI – Stärke (2 Stufen): GDS

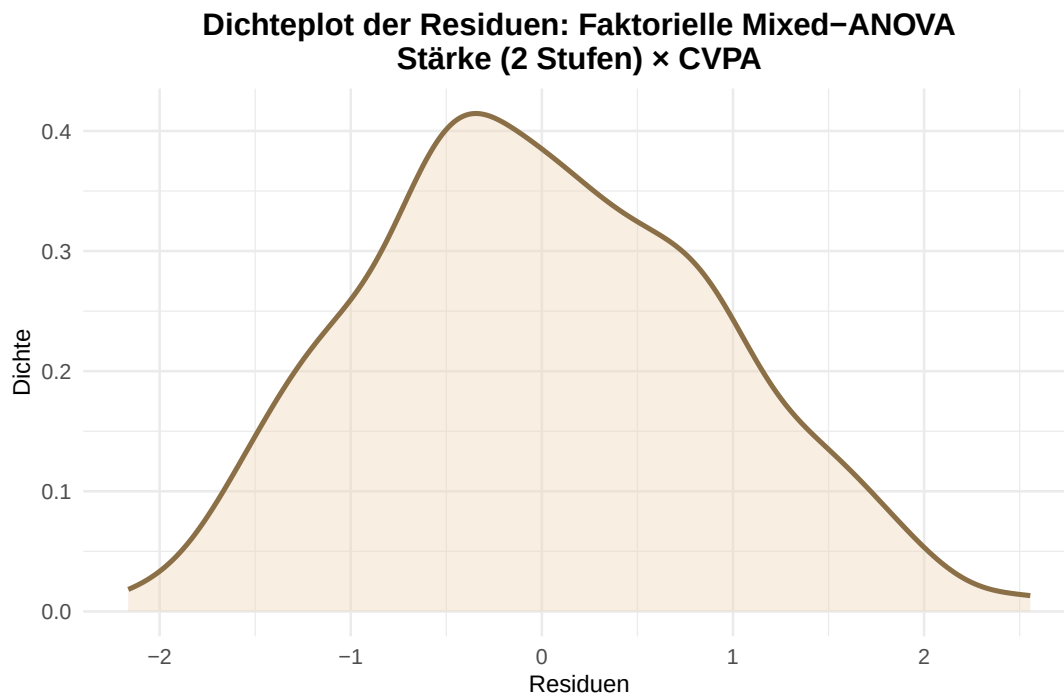


```
deskriptiv_staerke_cvpa <- deskriptiv_cvpa(
  inkonsistenz_staerke_cvpa,
  "Staerke"
)
deskriptiv_staerke_cvpa

anova_staerke_cvpa <- schaetze_cvpa_anova(
  inkonsistenz_staerke_cvpa,
  "Staerke"
)
residuen_staerke_cvpa <- residuals(anova_staerke_cvpa$lm)

# Density Plot
zeige_und_speichere_grafik(
  ggplot(
    data.frame(Residuen = as.numeric(residuen_staerke_cvpa)),
    aes(x = Residuen)
  ) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",
    alpha = 0.4,
    linewidth = 1
  ) +
  labs(
    title = stringr::str_wrap(
      "Dichteplot der Residuen: Faktorielle Mixed-ANOVA Stärke (2 Stufen) × CVPA",
      width = 52),
    x = "Residuen",
    y = "Dichte"
  ) +
  theme_box_density_gross +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  ),
  "Dichteplot der Residuen"
```

)



```
shapiro_auto_13 <- shapiro_residuen_cvpa(  
  residuen_staerke_cvpa  
)  
shapiro_auto_13  
  
levene_auto_18 <- levene_cvpa(  
  inkonsistenz_staerke_cvpa, "Staerke", "leicht"  
)  
levene_auto_18  
  
levene_auto_19 <- levene_cvpa(  
  inkonsistenz_staerke_cvpa, "Staerke", "stark"  
)  
levene_auto_19  
  
if (shapiro_auto_13$p.value > 0.05 &&  
    levene_auto_18$`Pr(>F)`[1] > 0.05 &&  
    levene_auto_19$`Pr(>F)`[1] > 0.05) {  
  anova_staerke_cvpa <- schaetze_cvpa_anova(  
    inkonsistenz_staerke_cvpa,  
    "Staerke"  
  )  
  anova_staerke_cvpa  
} else {  
  robust_anova_staerke_cvpa <- bwtrim(  
    Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Staerke,  
    id = number,  
    data = inkonsistenz_staerke_cvpa,  
    tr = 0.20  
  )  
  robust_anova_staerke_cvpa  
}
```

Table 106: Deskriptive Kennwerte – Stärke (2 Stufen) $\times$ CVPA - Teil 1 von 2

CVPA_Gruppe	Staerke	n	Median	IQR
unter Mittelwert	leicht	63	2.667	1.000
unter Mittelwert	stark	63	3.000	1.500
über Mittelwert	leicht	72	2.167	1.333
über Mittelwert	stark	72	3.000	1.083

Table 107: Deskriptive Kennwerte – Stärke (2 Stufen) $\times$ CVPA - Teil 2 von 2

CVPA_Gruppe	Mittelwert	SD
unter Mittelwert	2.593	0.815
unter Mittelwert	3.164	0.894
über Mittelwert	2.444	1.001
über Mittelwert	3.046	0.937

Table 108: Shapiro-Wilk-Test der Residuen – Stärke (2 Stufen) $\times$ CVPA

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.99
p-Wert	0.05839

Table 109: Levene-Test – GDL nach CVPA

Df	F value	Pr(>F)
1	2.312	0.131
133		

Table 110: Levene-Test – GDS nach CVPA

Df	F value	Pr(>F)
1	0.019	0.892
133		

Table 111: Faktorielle Mixed-ANOVA – Stärke (2 Stufen) × CVPA - Teil 1 von 2

Effect	df	MSE	F	ges
CVPA_Gruppe	1, 133	1.08	1.10	.005
Staerke	1, 133	0.60	38.39 ***	.094
CVPA_Gruppe:Staerke	1, 133	0.60	0.03	<.001

Table 112: Faktorielle Mixed-ANOVA – Stärke (2 Stufen) × CVPA - Teil 2 von 2

Effect	p.value
CVPA_Gruppe	.297
Staerke	<.001
CVPA_Gruppe:Staerke	.873

8.5 Stärke (3 Stufen) × CVPA

```
# 4. Stärke mit Baseline × CVPA
inkonsistenz_staerke3_cvpa <- bereite_cvpa_daten_vor(
  conditions = c(
    "Min-Baseline",
    "Jacke-leicht-dezentral",
    "Jacke-stark-dezentral"
  ),
  faktor = "Staerke",
  zuordnung = c(
    "Min-Baseline" = "baseline",
    "Jacke-leicht-dezentral" = "leicht",
    "Jacke-stark-dezentral" = "stark"
  ),
  stufen = c("baseline", "leicht", "stark")
)

table(inkonsistenz_staerke3_cvpa$number)

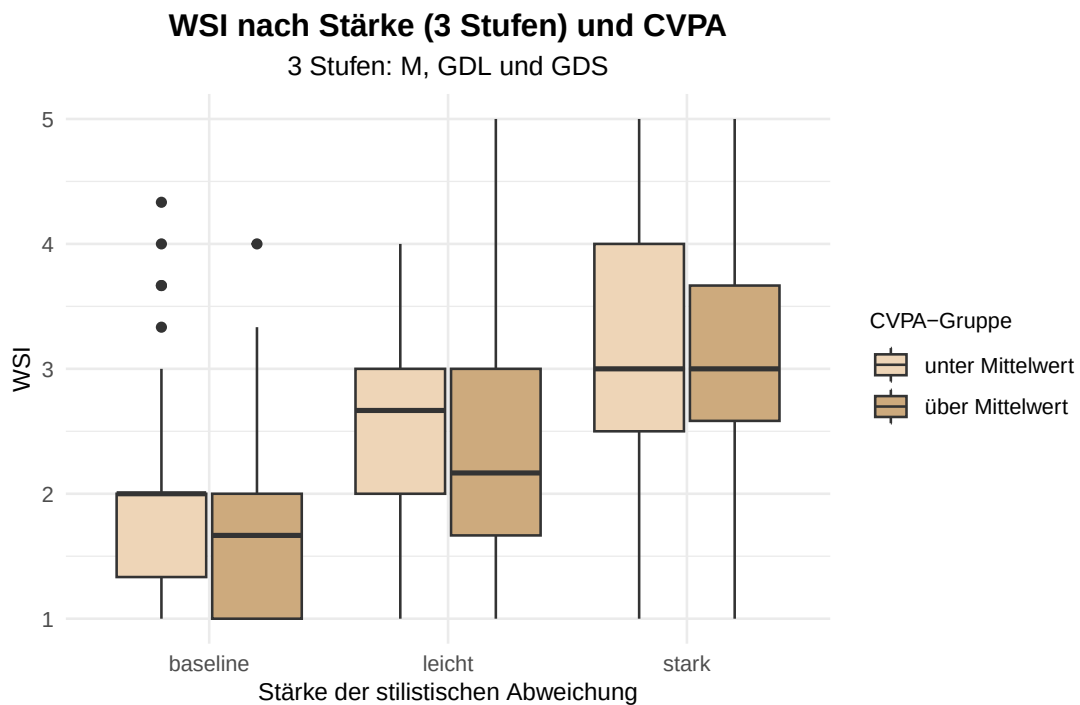
inkonsistenz_staerke3_cvpa %>%
  distinct(number, CVPA_Gruppe) %>%
  count(CVPA_Gruppe)

# Gemeinsamer Boxplot für alle drei Stärkestufen
zeige_und_speichere_grafik(
  ggplot(
    inkonsistenz_staerke3_cvpa,
    aes(
      x = Staerke,
      y = Mittelwert_Inkonsistenz,
      fill = CVPA_Gruppe
    )
  ) +
  geom_boxplot(position = position_dodge(width = 0.8)) +
  scale_fill_manual(
    values = c(
      "unter Mittelwert" = "#EED5B7",
```

```

    "über Mittelwert" = "#CDAA7D"
  )
) +
scale_y_continuous(breaks = 1:5, limits = c(1, 5)) +
labs(
  title = "WSI nach Stärke (3 Stufen) und CVPA",
  subtitle = "3 Stufen: M, GDL und GDS",
  x = "Stärke der stilistischen Abweichung",
  y = "WSI",
  fill = "CVPA-Gruppe"
) +
theme_box_density_gross,
"WSI nach Stärke und CVPA"
)

```



```

deskriptiv_staerke3_cvpa <- deskriptiv_cvpa(
  inkonsistenz_staerke3_cvpa,
  "Staerke"
)
deskriptiv_staerke3_cvpa

anova_staerke3_cvpa <- schaeetze_cvpa_anova(
  inkonsistenz_staerke3_cvpa,
  "Staerke"
)
residuen_staerke3_cvpa <- residuals(anova_staerke3_cvpa$lm)

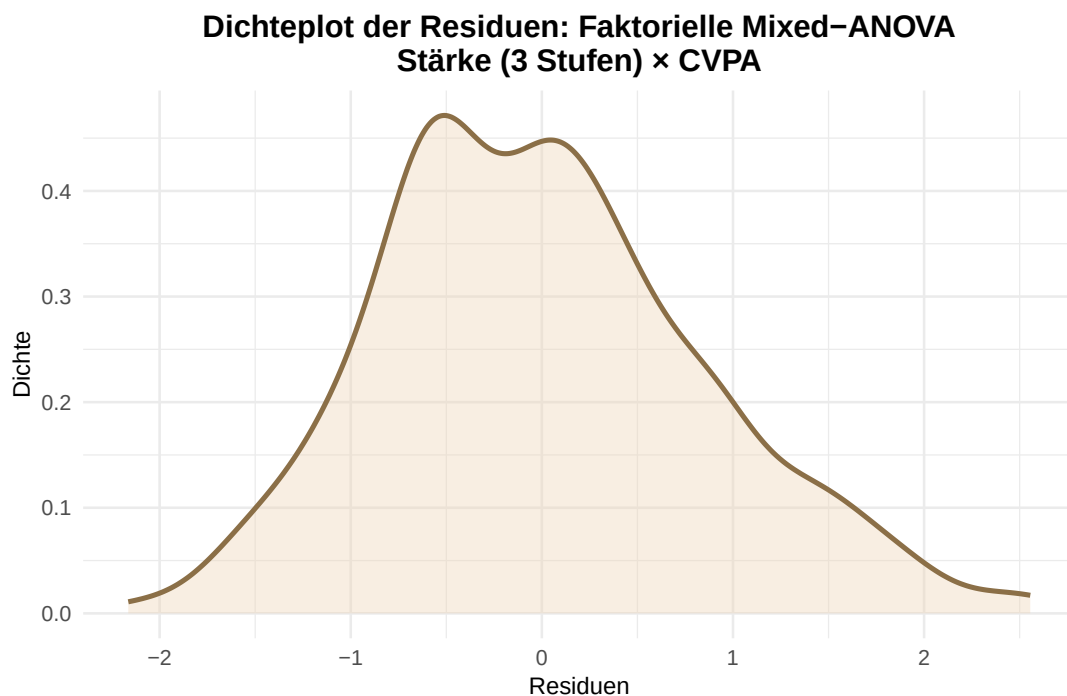
# Density Plot
zeige_und_speichere_grafik(
  ggplot(
    data.frame(Residuen = as.numeric(residuen_staerke3_cvpa)),
    aes(x = Residuen)
  ) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",

```

```

    alpha = 0.4,
    linewidth = 1
  ) +
  labs(
    title = stringr::str_wrap(
      "Dichteplot der Residuen: Faktorielle Mixed-ANOVA Stärke (3 Stufen) × CVPA",
      width = 52),
    x = "Residuen",
    y = "Dichte"
  ) +
  theme_box_density_gross +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  ),
  "Dichteplot der Residuen"
)

```



```

shapiro_auto_14 <- shapiro_residuen_cvpa(
  residuen_staerke3_cvpa
)
shapiro_auto_14

levene_auto_20 <- levene_cvpa(
  inkonsistenz_staerke3_cvpa, "Staerke", "baseline"
)
levene_auto_20

levene_auto_21 <- levene_cvpa(
  inkonsistenz_staerke3_cvpa, "Staerke", "leicht"
)
levene_auto_21

levene_auto_22 <- levene_cvpa(
  inkonsistenz_staerke3_cvpa, "Staerke", "stark"
)

```



```

)
levene_auto_22

# Sphärizität prüfen
summary_anova_staerke3_cvpa <- summary(anova_staerke3_cvpa)
summary_anova_staerke3_cvpa

if (shapiro_auto_14$p.value > 0.05 &&
    levene_auto_20$`Pr(>F)`[1] > 0.05 &&
    levene_auto_21$`Pr(>F)`[1] > 0.05 &&
    levene_auto_22$`Pr(>F)`[1] > 0.05) {
  anova_staerke3_cvpa
} else {
  robust_anova_staerke3_cvpa <- bwtrim(
    Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Staerke,
    id = number,
    data = inkonsistenz_staerke3_cvpa,
    tr = 0.20
  )
  robust_anova_staerke3_cvpa
}

```

Table 113: Deskriptive Kennwerte – Stärke (3 Stufen) × CVPA - Teil 1 von 2

CVPA_Gruppe	Staerke	n	Median	IQR
unter Mittelwert	baseline	63	2.000	0.667
unter Mittelwert	leicht	63	2.667	1.000
unter Mittelwert	stark	63	3.000	1.500
über Mittelwert	baseline	72	1.667	1.000
über Mittelwert	leicht	72	2.167	1.333
über Mittelwert	stark	72	3.000	1.083

Table 114: Deskriptive Kennwerte – Stärke (3 Stufen) × CVPA - Teil 2 von 2

CVPA_Gruppe	Mittelwert	SD
unter Mittelwert	1.884	0.755
unter Mittelwert	2.593	0.815
unter Mittelwert	3.164	0.894
über Mittelwert	1.616	0.687
über Mittelwert	2.444	1.001
über Mittelwert	3.046	0.937

Table 115: Shapiro-Wilk-Test der Residuen – Stärke (3 Stufen) $\times$ CVPA

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.985
p-Wert	0.000374

Table 116: Levene-Test – M nach CVPA

Df	F value	Pr(>F)
1	0.001	0.981
133		

Table 117: Levene-Test – GDL nach CVPA

Df	F value	Pr(>F)
1	2.312	0.131
133		

Table 118: Levene-Test – GDS nach CVPA

Df	F value	Pr(>F)
1	0.019	0.892
133		

Table 119: ANOVA-Zusammenfassung einschließlich Sphäritätsprüfung – CVPA

Ausgabe

Univariate Type III Repeated-Measures ANOVA Assuming Sphericity

Sum Sq	num Df	Error SS	den Df	F value	Pr(>F)
(Intercept)	2435.61	1	157.49	133	2056.8533 <2e-16 ***
CVPA_Gruppe	3.19	1	157.49	133	2.6944 0.1031
Staerke	124.21	2	135.50	266	121.9255 <2e-16 ***
CVPA_Gruppe:Staerke	0.42	2	135.50	266	0.4155 0.6604

—
Signif. codes: 0 ‘’ **0.001** ’’ 0.01 ’’ 0.05 ‘ 0.1 ’ ’ 1

Mauchly Tests for Sphericity

Ausgabe

Test statistic p-value

Staerke 0.96639 0.10476

CVPA_Gruppe:Staerke 0.96639 0.10476

Greenhouse-Geisser and Huynh-Feldt Corrections
for Departure from Sphericity

GG eps Pr(>F[GG])

Staerke 0.96749 <2e-16 ***

CVPA_Gruppe:Staerke 0.96749 0.6537

—
Signif. codes: 0 ‘’ **0.001** ’’ 0.01 ’’ 0.05 ‘’ 0.1 ’’ 1

HF eps Pr(>F[HF])

Staerke 0.9815223 1.190354e-37

CVPA_Gruppe:Staerke 0.9815223 6.566386e-01

Table 120: Robuste Mixed-ANOVA – Stärke (3 Stufen) ×
CVPA

Ausgabe

Call:

bwtrim(formula = Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Staerke,
id = number, data = inkonsistenz_staerke3_cvpa, tr = 0.2)

value df1 df2 p.value

CVPA_Gruppe 3.9657 1 75.8404 0.0500

Staerke 121.7395 2 64.2837 0.0000

CVPA_Gruppe:Staerke 0.5374 2 64.2837 0.5868

8.6 Schuh-Stärke (2 Stufen) × CVPA

```
# 5. Stärke Schuh × CVPA
inkonsistenz_staerke_schuh_cvpa <- bereite_cvpa_daten_vor(
  conditions = c("Min-Baseline", "Schuh-stark-dezentral"),
  faktor = "Staerke",
  zuordnung = c(
    "Min-Baseline" = "baseline",
    "Schuh-stark-dezentral" = "stark"
  ),
  stufen = c("baseline", "stark")
)

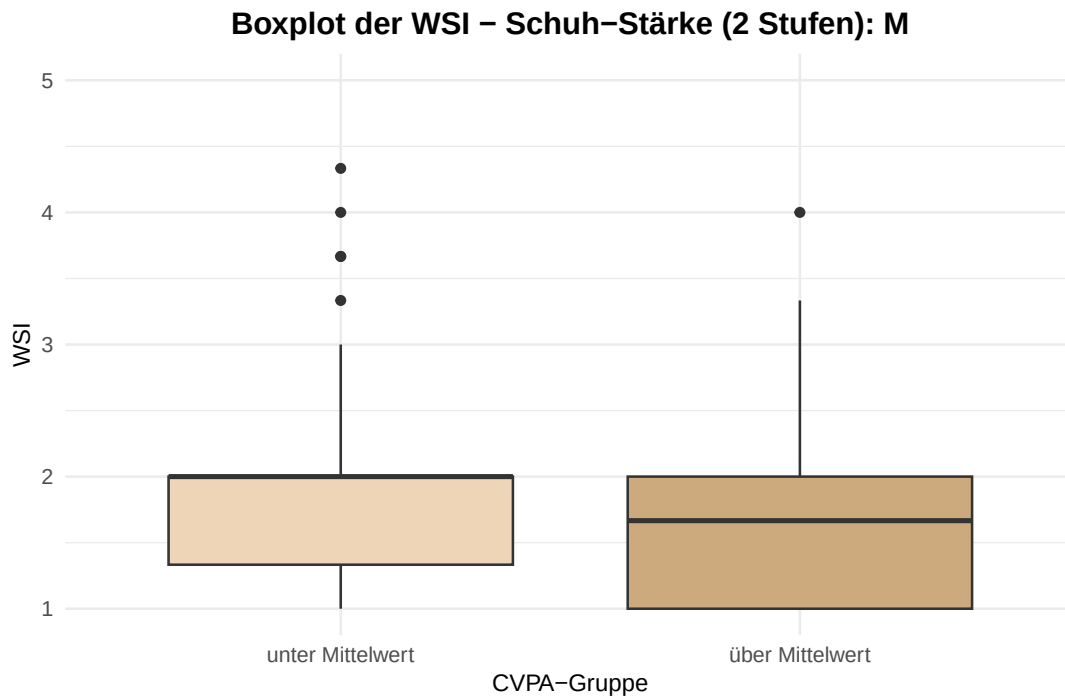
table(inkonsistenz_staerke_schuh_cvpa$number)
```

```

inkonsistenz_staerke_schuh_cvpa %>%
  distinct(number, CVPA_Gruppe) %>%
  count(CVPA_Gruppe)

boxplot_staerke_schuh_baseline_cvpa <- erstelle_cvpa_boxplot(
  inkonsistenz_staerke_schuh_cvpa,
  "Staerke",
  "baseline",
  "Schuh-Stärke (2 Stufen): M"
)
speichere_grafik(
  boxplot_staerke_schuh_baseline_cvpa,
  "WSI nach CVPA"
)
print(boxplot_staerke_schuh_baseline_cvpa)

```

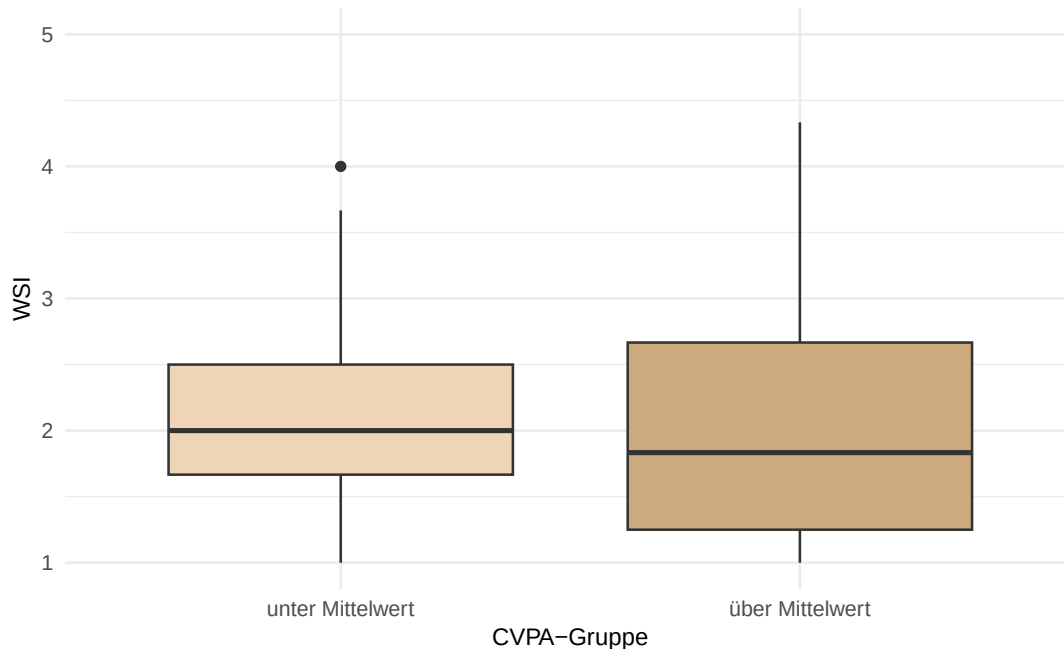


```

boxplot_staerke_schuh_stark_cvpa <- erstelle_cvpa_boxplot(
  inkonsistenz_staerke_schuh_cvpa,
  "Staerke",
  "stark",
  "Schuh-Stärke (2 Stufen): KDS"
)
speichere_grafik(
  boxplot_staerke_schuh_stark_cvpa,
  "WSI nach CVPA"
)
print(boxplot_staerke_schuh_stark_cvpa)

```

Boxplot der WSI – Schuh-Stärke (2 Stufen): KDS

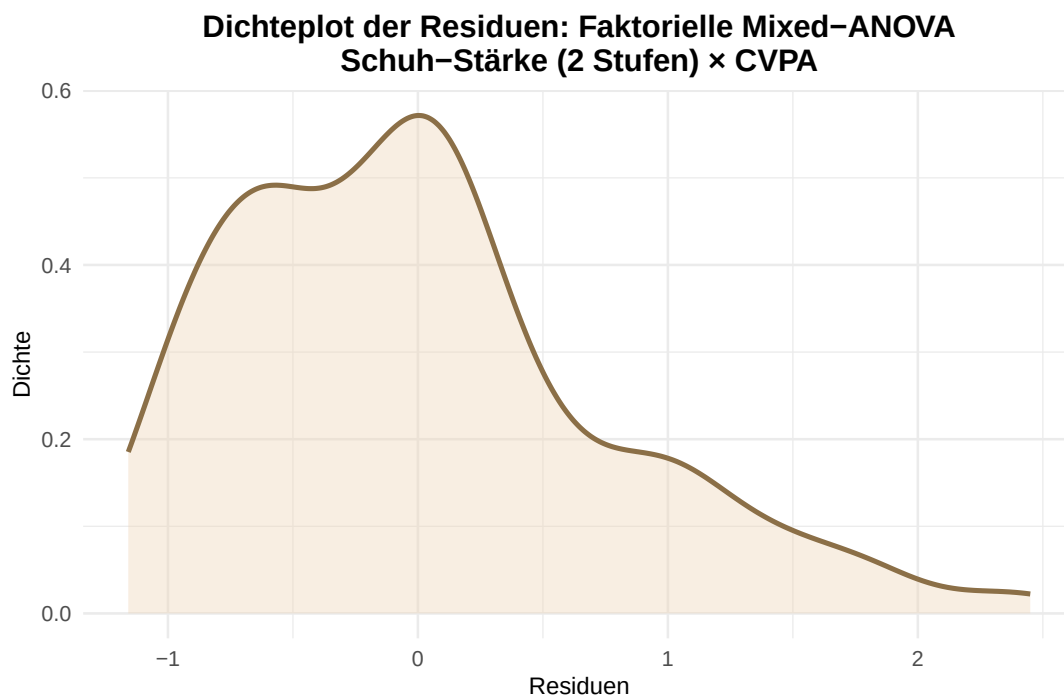


```
deskriptiv_staerke_schuh_cvpa <- deskriptiv_cvpa(
  inkonsistenz_staerke_schuh_cvpa,
  "Staerke"
)
deskriptiv_staerke_schuh_cvpa

anova_staerke_schuh_cvpa <- schaetze_cvpa_anova(
  inkonsistenz_staerke_schuh_cvpa,
  "Staerke"
)
residuen_staerke_schuh_cvpa <- residuals(anova_staerke_schuh_cvpa$lm)

# Density Plot
zeige_und_speichere_grafik(
  ggplot(
    data.frame(Residuen = as.numeric(residuen_staerke_schuh_cvpa)),
    aes(x = Residuen)
  ) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",
    alpha = 0.4,
    linewidth = 1
  ) +
  labs(
    title = stringr::str_wrap(
      "Dichteplot der Residuen: Faktorielle Mixed-ANOVA Schuh-Stärke (2 Stufen) × CVPA",
      width = 52),
    x = "Residuen",
    y = "Dichte"
  ) +
  theme_box_density_gross +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  ),
  "Dichteplot der Residuen"
```

)



```
shapiro_auto_15 <- shapiro_residuen_cvpa(  
  residuen_staerke_schuh_cvpa  
)  
shapiro_auto_15  
  
levene_auto_23 <- levene_cvpa(  
  inkonsistenz_staerke_schuh_cvpa, "Staerke", "baseline"  
)  
levene_auto_23  
  
levene_auto_24 <- levene_cvpa(  
  inkonsistenz_staerke_schuh_cvpa, "Staerke", "stark"  
)  
levene_auto_24  
  
if (shapiro_auto_15$p.value > 0.05 &&  
    levene_auto_23$`Pr(>F)`[1] > 0.05 &&  
    levene_auto_24$`Pr(>F)`[1] > 0.05) {  
  anova_staerke_schuh_cvpa <- schaeetze_cvpa_anova(  
    inkonsistenz_staerke_schuh_cvpa,  
    "Staerke"  
  )  
  anova_staerke_schuh_cvpa  
} else {  
  deskriptiv_staerke_schuh_cvpa_robust <- deskriptiv_cvpa_robust(  
    inkonsistenz_staerke_schuh_cvpa,  
    "Staerke"  
  )  
  deskriptiv_staerke_schuh_cvpa_robust  
  
  robust_anova_staerke_schuh_cvpa <- bwtrim(  
    Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Staerke,
```

```

    id = number,
    data = inkonsistenz_staerke_schuh_cvpa,
    tr = 0.20
)
robust_anova_staerke_schuh_cvpa
}

```

Table 121: Deskriptive Kennwerte – Schuh-Stärke (2 Stufen)
× CVPA - Teil 1 von 2

CVPA_Gruppe	Staerke	n	Median	IQR
unter Mittelwert	baseline	63	2.000	0.667
unter Mittelwert	stark	63	2.000	0.833
über Mittelwert	baseline	72	1.667	1.000
über Mittelwert	stark	72	1.833	1.417

Table 122: Deskriptive Kennwerte – Schuh-Stärke (2 Stufen)
× CVPA - Teil 2 von 2

CVPA_Gruppe	Mittelwert	SD
unter Mittelwert	1.884	0.755
unter Mittelwert	2.159	0.752
über Mittelwert	1.616	0.687
über Mittelwert	1.944	0.832

Table 123: Shapiro-Wilk-Test der Residuen – Schuh-Stärke
(2 Stufen) × CVPA

Kennwert	Wert
Methode	Shapiro-Wilk normality test
W	0.939
p-Wert	< 1e-04

Table 124: Levene-Test – M nach CVPA

Df	F value	Pr(>F)
1	0.001	0.981
133		

Table 125: Levene-Test – KDS nach CVPA

Df	F value	Pr(>F)
1	1.436	0.233

Df	F value	Pr(>F)
133		

Table 126: Robuste deskriptive Kennwerte – Schuh-Stärke (2 Stufen) $\times$ CVPA - Teil 1 von 2

CVPA_Gruppe	Staerke	n	Getrimmter_Mittelwert	Median
unter Mittelwert	baseline	63	1.803	2.000
unter Mittelwert	stark	63	2.111	2.000
über Mittelwert	baseline	72	1.462	1.667
über Mittelwert	stark	72	1.826	1.833

Table 127: Robuste deskriptive Kennwerte – Schuh-Stärke (2 Stufen) $\times$ CVPA - Teil 2 von 2

CVPA_Gruppe	IQR
unter Mittelwert	0.667
unter Mittelwert	0.833
über Mittelwert	1.000
über Mittelwert	1.417

Table 128: Robuste Mixed-ANOVA – Schuh-Stärke (2 Stufen) $\times$ CVPA

Ausgabe
Call: bwtrim(formula = Mittelwert_Inkonsistenz ~ CVPA_Gruppe * Staerke, id = number, data = inkonsistenz_staerke_schuh_cvpa, tr = 0.2)
value df1 df2 p.value
CVPA_Gruppe 6.2302 1 72.2374 0.0148
Staerke 13.9579 1 75.9883 0.0004
CVPA_Gruppe:Staerke 0.0969 1 75.9883 0.7564

9 Ermüdungseffekt und Reihenfolge

Geprüft wird, ob ein Outfit unterschiedlich bewertet wird, wenn es als erstes oder als letztes gezeigt wurde.

```
# Ermüdungseffekt
# Wird ein Outfit anders bewertet, wenn es als erstes vs. als letztes gezeigt wird?

# Gespeicherte Randomisierung der 10-Sekunden-Bilder prüfen
```



```

unique(salienz$Randomisierung_1)
unique(salienz$Randomisierung_2)
unique(salienz$Randomisierung_3)
unique(salienz$Randomisierung_4)
unique(salienz$Randomisierung_5)

# Unipark-IDs den Conditions zuordnen
randomisierung_zu_condition <- function(x) {
  x <- as.character(x)
  case_when(
    x == "7698430" ~ "Jacke-stark-dezentral",
    x == "7698432" ~ "Schuh-stark-dezentral",
    x == "7698426" ~ "Jacke-stark-zentral",
    x == "7698437" ~ "Jacke-leicht-dezentral",
    x == "7698434" ~ "Min-Baseline",
    TRUE ~ NA_character_
  )
}

# Erstes und letztes Outfit pro Person bestimmen
salienz_resaped <- salienz_resaped %>%
  mutate(
    Erstes_Outfit = randomisierung_zu_condition(Randomisierung_1),
    Letztes_Outfit = randomisierung_zu_condition(Randomisierung_5)
  )

# Zuordnung prüfen
salienz_resaped %>%
  distinct(number, Erstes_Outfit) %>%
  count(Erstes_Outfit)

salienz_resaped %>%
  distinct(number, Letztes_Outfit) %>%
  count(Letztes_Outfit)

# Erstes und letztes Outfit auswählen
ermuedung_plot <- salienz_resaped %>%
  mutate(
    Reihenfolge = case_when(
      Condition == Erstes_Outfit ~ "Als erstes gezeigt",
      Condition == Letztes_Outfit ~ "Als letztes gezeigt",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(Reihenfolge), !is.na(Mittelwert_Inkonsistenz)) %>%
  mutate(
    Reihenfolge = factor(
      Reihenfolge,
      levels = c("Als erstes gezeigt", "Als letztes gezeigt")
    )
  )

# Deskriptive Analyse
deskriptiv_ermuedung <- ermuedung_plot %>%
  group_by(Condition, Reihenfolge) %>%
  summarise(
    n = n(),
    Median = median(Mittelwert_Inkonsistenz, na.rm = TRUE),
    IQR = IQR(Mittelwert_Inkonsistenz, na.rm = TRUE),
    Mittelwert = mean(Mittelwert_Inkonsistenz, na.rm = TRUE),
    SD = sd(Mittelwert_Inkonsistenz, na.rm = TRUE),
  )

```

```

    .groups = "drop"
  ) %>%
  mutate(Condition = kurz_bedingung(Condition))

print(deskriptiv_ermuedung)

# Boxplots
outfit_namen <- c(
  "Min-Baseline" = "M",
  "Jacke-leicht-dezentral" = "GDL",
  "Jacke-stark-dezentral" = "GDS",
  "Jacke-stark-zentral" = "GZS",
  "Schuh-stark-dezentral" = "KDS"
)

boxplots_ermuedung <- list()

for (outfit in conditions) {

  plot_daten <- ermuedung_plot %>%
    filter(Condition == outfit)

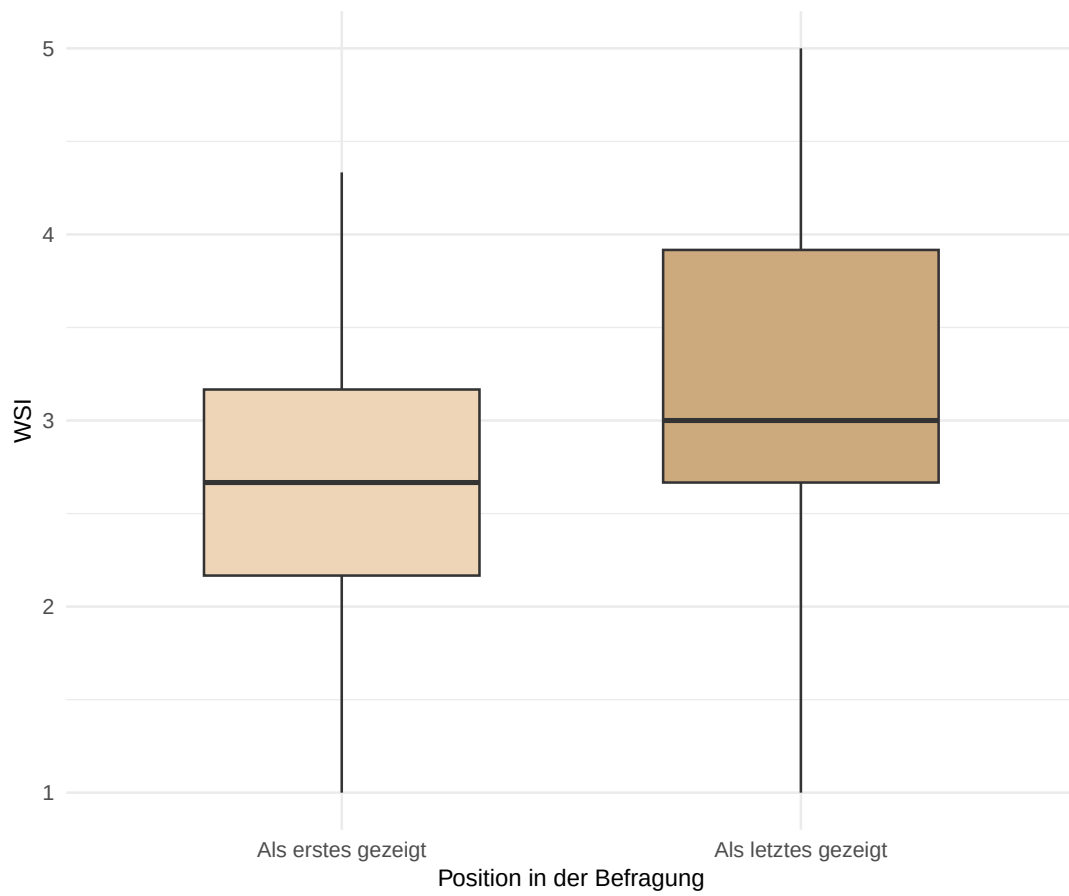
  plot <- ggplot(
    plot_daten,
    aes(
      x = Reihenfolge,
      y = Mittelwert_Inkonsistenz,
      fill = Reihenfolge
    )
  ) +
  geom_boxplot(width = 0.6) +
  scale_fill_manual(
    values = c(
      "Als erstes gezeigt" = "#EED5B7",
      "Als letztes gezeigt" = "#CDAA7D"
    )
  ) +
  scale_y_continuous(breaks = 1:5, limits = c(1, 5)) +
  labs(
    title = paste("WSI:", outfit_namen[outfit]),
    subtitle = "Als erstes vs. als letztes gezeigt",
    x = "Position in der Befragung",
    y = "WSI"
  ) +
  theme_box_density_gross +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  )

  speichere_grafik(plot, paste0("Ermuedung_", outfit))
  boxplots_ermuedung[[outfit]] <- plot
  print(plot)
}

```

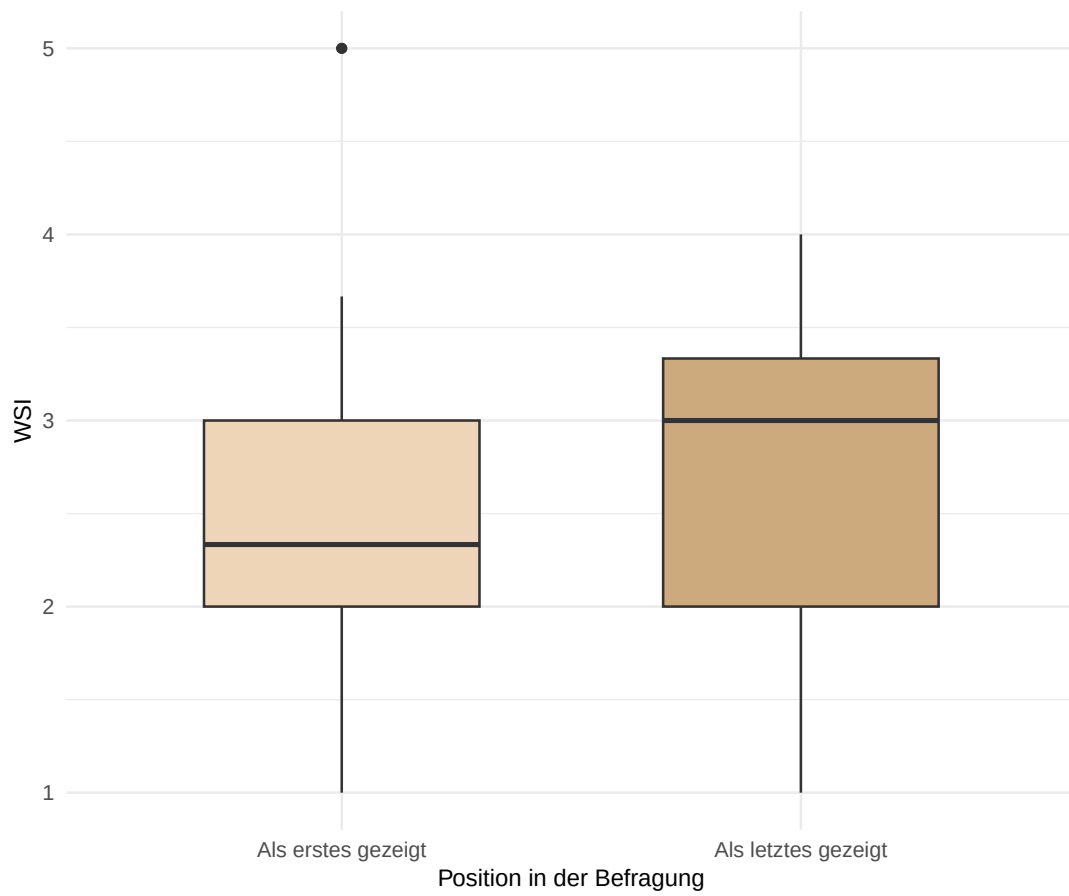
WSI: GDS

Als erstes vs. als letztes gezeigt



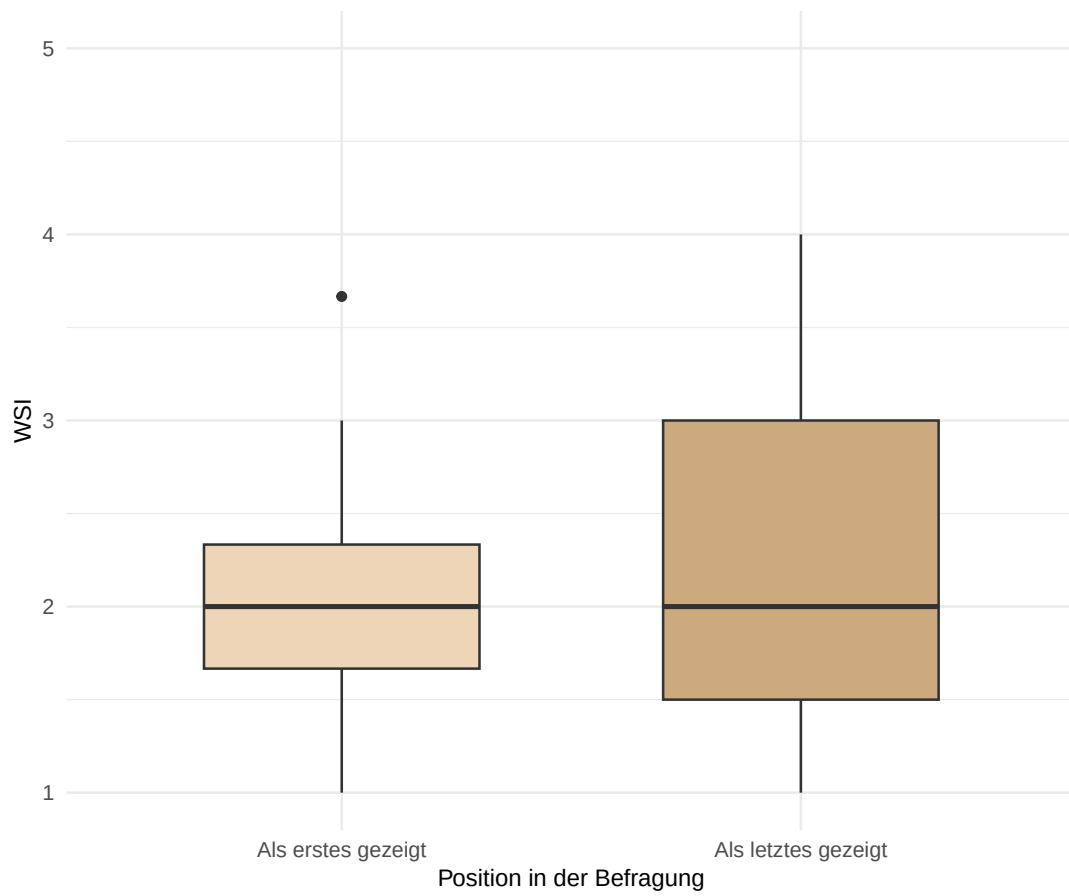
WSI: GDL

Als erstes vs. als letztes gezeigt



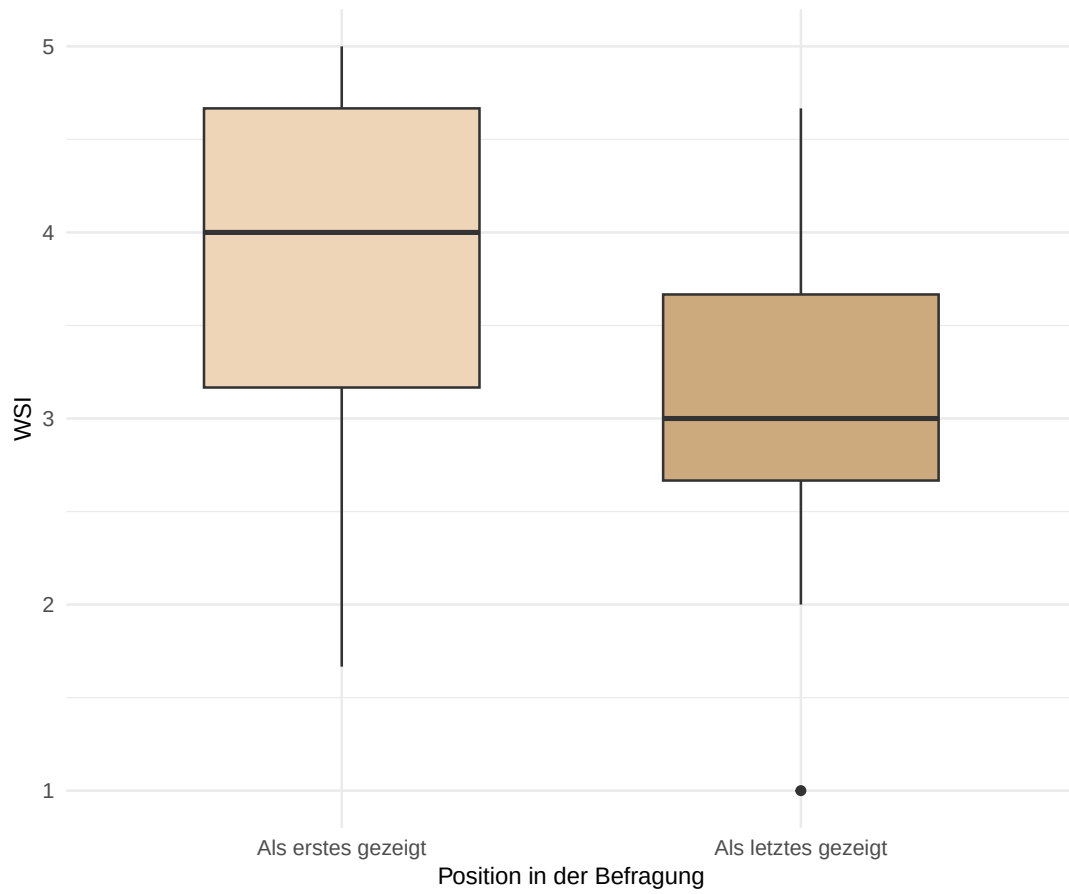
WSI: KDS

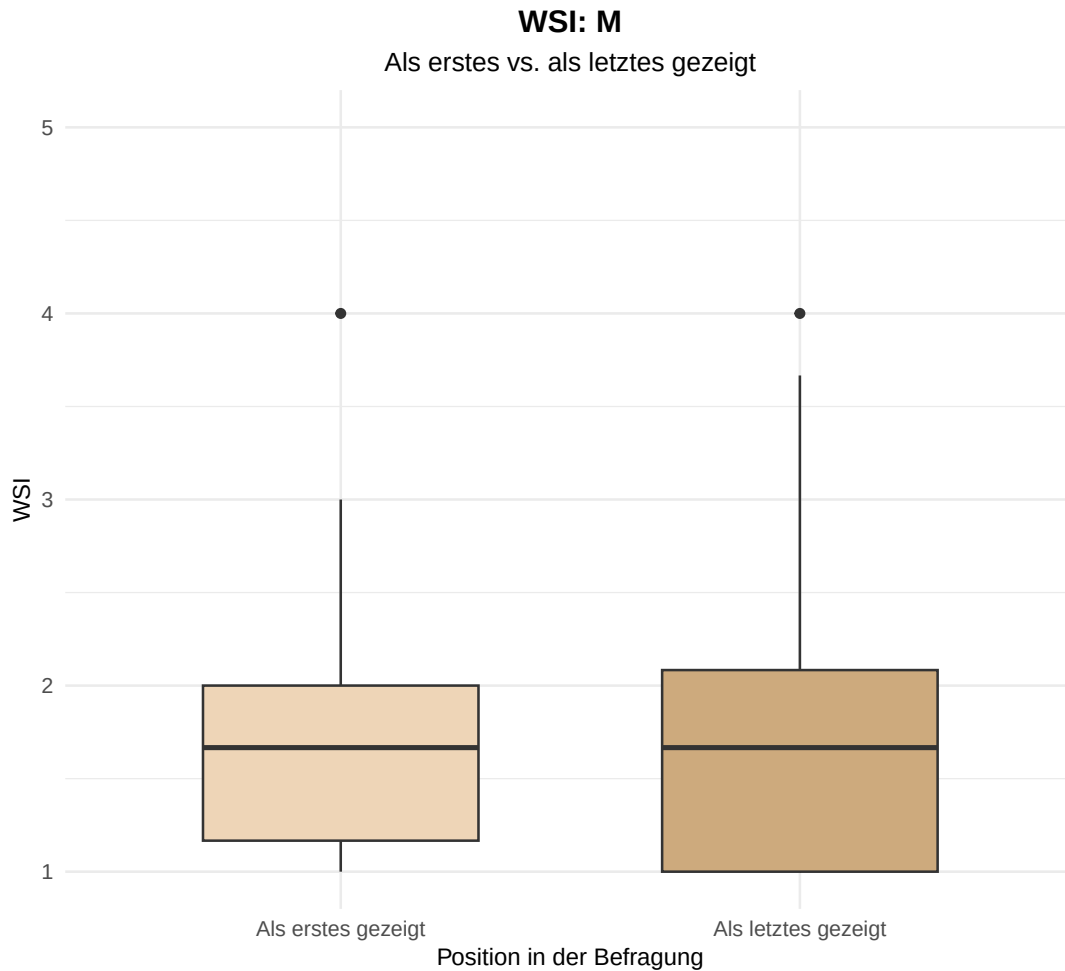
Als erstes vs. als letztes gezeigt



WSI: GZS

Als erstes vs. als letztes gezeigt





```
# Normalverteilung mit Shapiro-Wilk
shapiro_ermuedung <- ermuedung_plot %>%
  group_by(Condition, Reihenfolge) %>%
  shapiro_test(Mittelwert_Inkonsistenz)

print(shapiro_ermuedung)

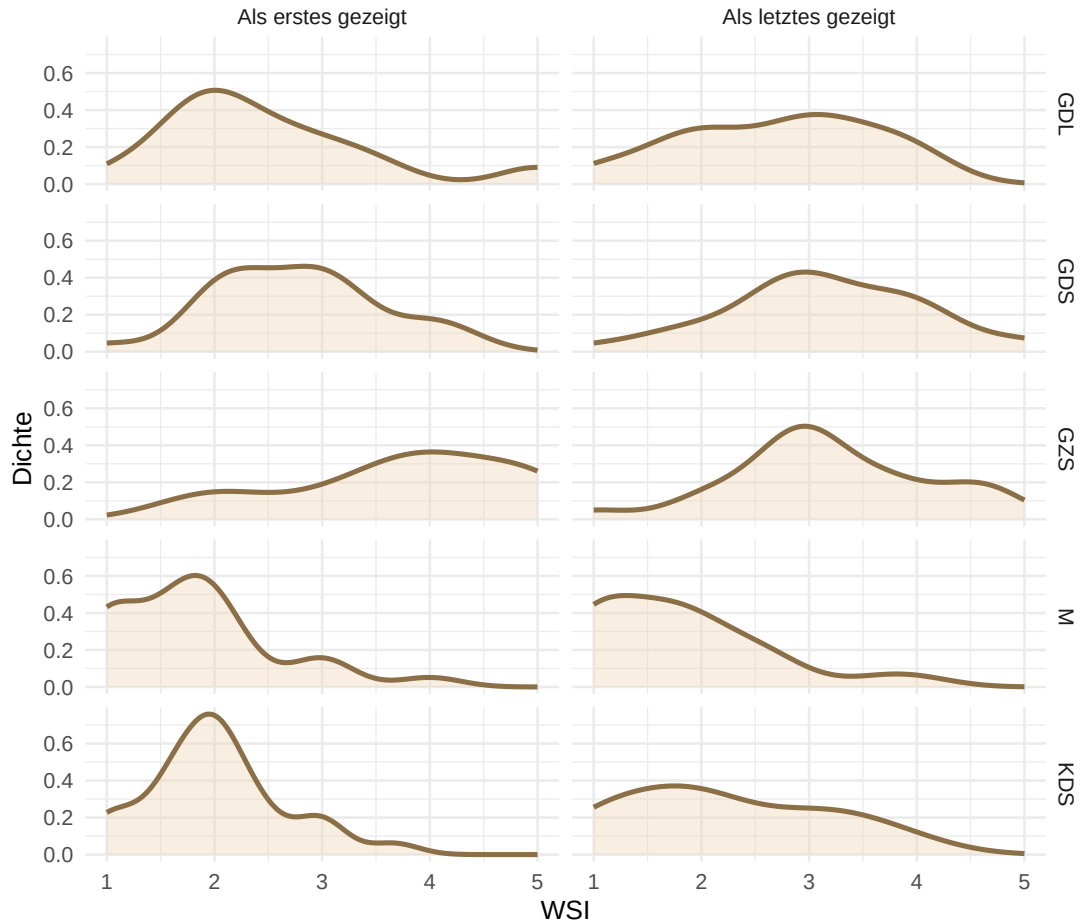
# Normalverteilung grafisch darstellen: Density Plot (nur deskriptiv; nicht für die
# Testwahl)
zeige_und_speichere_grafik(
  ggplot(ermuedung_plot, aes(x = Mittelwert_Inkonsistenz)) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",
    alpha = 0.4,
    linewidth = 1
  ) +
  facet_grid(Condition ~ Reihenfolge, labeller = labeller(Condition = as_labeller(bedingung_labels)))
  ↪ +
  labs(
    title = "Dichteplot der WSI: Erstes vs. letztes gezeigtes Outfit",
    x = "WSI",
    y = "Dichte"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
```

```

    plot.subtitle = element_text(hjust = 0.5)
  ),
  "Dichteplots der WSI"
)

```

Dichteplot der WSI: Erstes vs. letztes gezeigtes Outfit



```

# Levene-Test
levene_ermuedung <- ermuedung_plot %>%
  group_by(Condition) %>%
  levene_test(Mittelwert_Inkonsistenz ~ Reihenfolge)

print(levene_ermuedung)

# Hypothesentest für jedes Outfit
ergebnisse_ermuedung <- data.frame()

for (outfit in conditions) {

  # Daten des aktuellen Outfits
  daten_outfit <- ermuedung_plot %>%
    filter(Condition == outfit) %>%
    droplevels()

  # Gruppen getrennt speichern
  zuerst <- daten_outfit %>%
    filter(Reihenfolge == "Als erstes gezeigt") %>%
    pull(Mittelwert_Inkonsistenz)

```



```

zuletzt <- daten_outfit %>%
  filter(Reihenfolge == "Als letztes gezeigt") %>%
  pull(Mittelwert_Inkonsistenz)

# Normalverteilung prüfen
shapiro_zuerst <- shapiro.test(zuerst)

shapiro_zuletzt <- shapiro.test(zuletzt)

# Normalverteilung grafisch darstellen: Density Plot (nur deskriptiv; nicht für die
# Testwahl)
zeige_und_speichere_grafik(
  ggplot(
    daten_outfit,
    aes(x = Mittelwert_Inkonsistenz)
  ) +
  geom_density(
    fill = "#EED5B7",
    color = "#8B6F47",
    alpha = 0.4,
    linewidth = 1
  ) +
  facet_wrap(~ Reihenfolge) +
  labs(
    title = paste("Dichteplot der WSI:", outfit_namen[outfit], "- als erstes vs. als letztes
    ↪ gezeigt"),
    x = "WSI",
    y = "Dichte"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
  ),
  paste0("Density_Normalverteilung_Ermuedung_", outfit)
)

# Varianzhomogenität prüfen
levene_result <- car::leveneTest(
  Mittelwert_Inkonsistenz ~ Reihenfolge,
  data = daten_outfit
)
levene_p <- levene_result$`Pr(>F)`[1]

# Test ausschließlich anhand der p-Werte von Shapiro-Wilk und Levene auswählen
if (shapiro_zuerst$p.value > 0.05 &&
    shapiro_zuletzt$p.value > 0.05) {
  if (levene_p > 0.05) {
    test_result <- t.test(
      zuerst,
      zuletzt,
      paired = FALSE,
      var.equal = TRUE,
      alternative = "two.sided"
    )
    verwendeter_test <- "Unabhängiger t-Test"
  } else {
    test_result <- t.test(
      zuerst,
      zuletzt,

```

```

    paired = FALSE,
    var.equal = FALSE,
    alternative = "two.sided"
  )
  verwendeter_test <- "Welch-t-Test"
}

} else {
  test_result <- wilcox.test(
    zuerst,
    zuletzt,
    paired = FALSE,
    alternative = "two.sided",
    exact = FALSE
  )
  verwendeter_test <- "Mann-Whitney-U-Test"
}

# Ergebnisse speichern
ergebnisse_ermuedung <- rbind(
  ergebnisse_ermuedung,
  data.frame(
    Outfit = outfit,
    n_zuerst = length(zuerst),
    n_zuletzt = length(zuletzt),
    Mittelwert_zuerst = mean(zuerst, na.rm = TRUE),
    Mittelwert_zuletzt = mean(zuletzt, na.rm = TRUE),
    Differenz = mean(zuletzt, na.rm = TRUE) - mean(zuerst, na.rm = TRUE),
    Shapiro_p_zuerst = shapiro_zuerst$p.value,
    Shapiro_p_zuletzt = shapiro_zuletzt$p.value,
    Levene_p = levene_p,
    Test = verwendeter_test,
    Teststatistik = unname(test_result$statistic),
    p_Wert = test_result$p.value
  )
)

# Holm-Korrektur für fünf Tests
ergebnisse_ermuedung <- ergebnisse_ermuedung %>%
  mutate(
    p_Wert_Holm = p.adjust(p_Wert, method = "holm"),
    Signifikant_nach_Holm = ifelse(p_Wert_Holm < 0.05, "Ja", "Nein")
  )

# Für die sichtbare Ergebnistabelle die Outfit-Bezeichnungen abkürzen.
ergebnisse_ermuedung <- ergebnisse_ermuedung %>%
  mutate(Outfit = kurz_bedingung(Outfit))

print(ergebnisse_ermuedung)

```

Table 129: Deskriptive Kennwerte – erstes vs. letztes Outfit
- Teil 1 von 2

Condition	Reihenfolge	n	Median	IQR
GDL	Als erstes gezeigt	25	2.333	1.000
GDL	Als letztes gezeigt	31	3.000	1.333

Condition	Reihenfolge	n	Median	IQR
GDS	Als erstes gezeigt	27	2.667	1.000
GDS	Als letztes gezeigt	34	3.000	1.250
GZS	Als erstes gezeigt	27	4.000	1.500
GZS	Als letztes gezeigt	23	3.000	1.000
M	Als erstes gezeigt	27	1.667	0.833
M	Als letztes gezeigt	28	1.667	1.083
KDS	Als erstes gezeigt	29	2.000	0.667
KDS	Als letztes gezeigt	19	2.000	1.500

Table 130: Deskriptive Kennwerte – erstes vs. letztes Outfit
- Teil 2 von 2

Condition	Mittelwert	SD
GDL	2.493	0.996
GDL	2.710	0.897
GDS	2.765	0.778
GDS	3.147	0.911
GZS	3.741	1.039
GZS	3.174	0.920
M	1.815	0.753
M	1.786	0.807
KDS	2.011	0.633
KDS	2.228	0.943

Table 131: Shapiro-Wilk-Tests – Ermüdungsanalyse

Condition	Reihenfolge	variable	statistic	p
Jacke-leicht-dezentral	Als erstes gezeigt	Mittelwert_Inkonsistenz	0.888	0.010
Jacke-leicht-dezentral	Als letztes gezeigt	Mittelwert_Inkonsistenz	0.943	0.100
Jacke-stark-dezentral	Als erstes gezeigt	Mittelwert_Inkonsistenz	0.951	0.228
Jacke-stark-dezentral	Als letztes gezeigt	Mittelwert_Inkonsistenz	0.972	0.509
Jacke-stark-zentral	Als erstes gezeigt	Mittelwert_Inkonsistenz	0.908	0.020
Jacke-stark-zentral	Als letztes gezeigt	Mittelwert_Inkonsistenz	0.947	0.259
Min-Baseline	Als erstes gezeigt	Mittelwert_Inkonsistenz	0.869	0.003
Min-Baseline	Als letztes gezeigt	Mittelwert_Inkonsistenz	0.859	0.001
Schuh-stark-dezentral	Als erstes gezeigt	Mittelwert_Inkonsistenz	0.937	0.081
Schuh-stark-dezentral	Als letztes gezeigt	Mittelwert_Inkonsistenz	0.935	0.217

Table 132: Levene-Tests – Ermüdungsanalyse

Condition	df1	df2	statistic	p
Jacke-leicht-dezentral	1	54	0.017	0.897
Jacke-stark-dezentral	1	59	0.318	0.575

Condition	df1	df2	statistic	p
Jacke-stark-zentral	1	48	0.509	0.479
Min-Baseline	1	53	0.128	0.722
Schuh-stark-dezentral	1	46	4.272	0.044

Table 133: Hypothesentests zum Ermüdungseffekt mit Holm-Korrektur - Teil 1 von 4

Outfit	n_zuerst	n_zuletzt	Mittelwert_zuerst	Mittelwert_zuletzt
GDS	27	34	2.765	3.147
GDL	25	31	2.493	2.710
KDS	29	19	2.011	2.228
GZS	27	23	3.741	3.174
M	27	28	1.815	1.786

Table 134: Hypothesentests zum Ermüdungseffekt mit Holm-Korrektur - Teil 2 von 4

Outfit	n_zuerst	Differenz	Shapiro_p_zuerst	Shapiro_p_zuletzt
GDS	27	0.382	0.228	0.509
GDL	25	0.216	0.010	0.100
KDS	29	0.217	0.081	0.217
GZS	27	-0.567	0.020	0.259
M	27	-0.029	0.003	0.001

Table 135: Hypothesentests zum Ermüdungseffekt mit Holm-Korrektur - Teil 3 von 4

Outfit	n_zuerst	Levene_p	Test	Teststatistik
GDS	27	0.575	Unabhängiger t-Test	-1.732
GDL	25	0.897	Mann-Whitney-U-Test	311.000
KDS	29	0.044	Welch-t-Test	-0.880
GZS	27	0.479	Mann-Whitney-U-Test	420.000
M	27	0.722	Mann-Whitney-U-Test	396.000

Table 136: Hypothesentests zum Ermüdungseffekt mit Holm-Korrektur - Teil 4 von 4

Outfit	n_zuerst	p_Wert	p_Wert_Holm	Signifikant_nach_Holm
GDS	27	0.089	0.354	Nein
GDL	25	0.206	0.619	Nein
KDS	29	0.386	0.773	Nein
GZS	27	0.033	0.163	Nein

Outfit	n_zuerst	p_Wert	p_Wert_Holm	Signifikant_nach_Holm
M	27	0.763	0.773	Nein

10 Mediationsanalyse

Die Mediationsanalyse folgt dem im Skript definierten vorlesungsnahen Ansatz. Der Analysecode bleibt unverändert; die zusammengefassten Ergebnisse werden tabellarisch ausgegeben.

```
# Mediation - vorlesungs naher Ansatz:
# X -> WSI -> Bewertung
set.seed(123)

# Voraussetzung:
# salienz_reshaped existiert bereits und enthält:
# number
# Condition
# Mittelwert_Inkonsistenz
# Mittelwert_Interesse
# Mittelwert_Liking
# Mittelwert_Fluency
# Mittelwert_Kreativitaet
# Mittelwert_Authentizitaet
# Mittelwert_SophisticatedTaste

# 1. Zu untersuchende Kontraste
mediation_kontraste <- tibble::tribble(
  ~Kontrast, ~Bedingung_0, ~Bedingung_1,

  "Größe",
  "Schuh-stark-dezentral",
  "Jacke-stark-dezentral",

  "Stärke",
  "Jacke-leicht-dezentral",
  "Jacke-stark-dezentral",

  "Position",
  "Jacke-stark-dezentral",
  "Jacke-stark-zentral"
)

# 2. Zielvariablen
mediation_outcomes <- c(
  Fluency = "Mittelwert_Fluency"
)

# 3. Daten für einen Kontrast vorbereiten
bereite_mediationsdaten_vor <- function(
  data,
  bedingung_0,
  bedingung_1,
  outcome_variable) {

  # Nur die beiden relevanten Bedingungen
  daten <- data %>%
    filter(Condition %in% c(bedingung_0, bedingung_1)) %>%
    transmute(
```

```

    number = factor(number),
    Condition,
    M = as.numeric(Mittelwert_Inkonsistenz),
    Y = as.numeric(.data[[outcome_variable]])
  ) %>%
  drop_na()

# X codieren:
# 0 = Ausgangsbedingung
# 1 = experimentell stärkere Salienzbedingung
daten <- daten %>%
  mutate(
    X = ifelse(Condition == bedingung_1, 1, 0)
  )

# Personenmittelwert des Mediators
# trennt Within- und Between-Person-Anteil
daten <- daten %>%
  group_by(number) %>%
  mutate(
    M_Person = mean(M)
  ) %>%
  ungroup() %>%
  mutate(
    M_Person_zentriert =
      M_Person - mean(M_Person)
  )

daten
}

# 4. Eine Mediation schätzen
schaetze_mediation <- function(daten, sims = 5000) {

  # Pfad a:
  # UV -> Mediator
  pfad_a <- lme4::lmer(
    M ~ X + (1 | number),
    data = daten,
    REML = FALSE
  )

  # Pfad b und direkter Effekt c':
  # UV + Mediator -> AV
  pfad_b_c <- lme4::lmer(
    Y ~ X + M + M_Person_zentriert + (1 | number),
    data = daten,
    REML = FALSE
  )

  # Totaler Effekt c:
  pfad_c <- lme4::lmer(
    Y ~ X + (1 | number),
    data = daten,
    REML = FALSE
  )

  # Mediation
  mediation_result <- mediation::mediate(
    model.m = pfad_a,
    model.y = pfad_b_c,

```

```

    treat = "X",
    mediator = "M",
    control.value = 0,
    treat.value = 1,
    sims = sims,
    boot = FALSE,
    group.out = "number"
  )

  list(
    pfad_a = pfad_a,
    pfad_b_c = pfad_b_c,
    pfad_c = pfad_c,
    mediation = mediation_result
  )
}

# 5. Ergebnisse aus mediate() extrahieren
extrahiere_ergebnisse <- function(modell) {

  med <- modell$mediation

  # ACME = indirekter Effekt
  ACME <- med$d.avg
  ACME_CI <- med$d.avg.ci
  p_ACME <- med$d.avg.p

  # ADE = direkter Effekt
  ADE <- med$z.avg
  ADE_CI <- med$z.avg.ci
  p_ADE <- med$z.avg.p

  # Totaler Effekt
  Total <- med$tau.coef
  Total_CI <- med$tau.ci
  p_Total <- med$tau.p

  data.frame(
    Pfad_a = unname(lme4::fixef(modell$pfad_a)["X"]),
    Pfad_b = unname(lme4::fixef(modell$pfad_b_c)["M"]),

    ACME = ACME,
    ACME_CI_unten = ACME_CI[1],
    ACME_CI_oben = ACME_CI[2],
    p_ACME = p_ACME,

    ADE = ADE,
    ADE_CI_unten = ADE_CI[1],
    ADE_CI_oben = ADE_CI[2],
    p_ADE = p_ADE,

    Total_Effect = Total,
    Total_CI_unten = Total_CI[1],
    Total_CI_oben = Total_CI[2],
    p_Total = p_Total
  )
}

# 6. Alle 18 Mediationen durchführen
ergebnis_liste <- list()
zaehler <- 0

```

```

for (i in seq_len(nrow(mediation_kontraste))) {

  kontrast <- mediation_kontraste[i, ]

  for (j in seq_along(mediation_outcomes)) {

    outcome_name <- names(mediation_outcomes)[j]
    outcome_variable <- mediation_outcomes[j]

    # Daten vorbereiten
    daten <- bereite_mediationsdaten_vor(
      data = salienz_resaped,
      bedingung_0 = kontrast$Bedingung_0,
      bedingung_1 = kontrast$Bedingung_1,
      outcome_variable = outcome_variable
    )

    # Mediation rechnen
    modell <- schaeetze_mediation(
      daten = daten,
      sims = 5000
    )

    # Ergebnisse extrahieren
    ergebnis <- extrahiere_ergebnisse(modell)

    zaehler <- zaehler + 1

    ergebnis_liste[[zaehler]] <- data.frame(
      Kontrast = kontrast$Kontrast,
      Outcome = outcome_name,
      n = n_distinct(daten$number),
      ergebnis
    )
  }
}

# 7. Ergebnistabelle
mediation_ergebnisse <- bind_rows(ergebnis_liste) %>%
  group_by(Kontrast) %>%
  mutate(
    p_ACME_Holm = p.adjust(
      p_ACME,
      method = "holm"
    )
  ) %>%
  ungroup()

print(mediation_ergebnisse)

# 8. Ergebnisse speichern
write.csv2(
  mediation_ergebnisse,
  "Mediationsanalyse_Ergebnisse.csv",
  row.names = FALSE,
  fileEncoding = "UTF-8"
)

```


Table 137: Ergebnisse der Mediationsanalysen - Teil 1 von 6

Kontrast	Outcome	n	Pfad_a	Pfad_b
Größe	Fluency	135	1.057	-0.084
Stärke	Fluency	135	0.588	-0.182
Position	Fluency	135	0.430	0.020

Table 138: Ergebnisse der Mediationsanalysen - Teil 2 von 6

Kontrast	Outcome	ACME	ACME_CI_unten	ACME_CI_oben
Größe	Fluency	-0.088	-0.242	0.064
Stärke	Fluency	-0.106	-0.191	-0.032
Position	Fluency	0.008	-0.047	0.068

Table 139: Ergebnisse der Mediationsanalysen - Teil 3 von 6

Kontrast	Outcome	p_ACME	ADE	ADE_CI_unten
Größe	Fluency	0.251	-0.213	-0.425
Stärke	Fluency	0.006	-0.014	-0.161
Position	Fluency	0.778	0.054	-0.090

Table 140: Ergebnisse der Mediationsanalysen - Teil 4 von 6

Kontrast	Outcome	ADE_CI_oben	p_ADE	Total_Effect
Größe	Fluency	-0.003	0.046	-0.301
Stärke	Fluency	0.131	0.862	-0.120
Position	Fluency	0.202	0.465	0.063

Table 141: Ergebnisse der Mediationsanalysen - Teil 5 von 6

Kontrast	Outcome	Total_CI_unten	Total_CI_oben	p_Total
Größe	Fluency	-0.444	-0.154	0.000
Stärke	Fluency	-0.255	0.011	0.072
Position	Fluency	-0.073	0.200	0.369

Table 142: Ergebnisse der Mediationsanalysen - Teil 6 von 6

Kontrast	Outcome	p_ACME_Holm
Größe	Fluency	0.251
Stärke	Fluency	0.006
Position	Fluency	0.778

11 Ausblick: Wahrgenommene Stilzuordnung

```
# Ausblick

# Stilzuordnung prüfen
# Stilbewertungen ins Long-Format bringen
stilzuordnung_long <- salienz_reshaped %>%
  dplyr::select(
    number,
    Condition,
    Minimalismus,
    HipHopActiveWear,
    HippieBoho
  ) %>%
  pivot_longer(
    cols = c(
      Minimalismus,
      HipHopActiveWear,
      HippieBoho
    ),
    names_to = "Stil",
    values_to = "Bewertung"
  ) %>%
  mutate(
    Bewertung = as.numeric(Bewertung),

    Stil = factor(
      Stil,
      levels = c(
        "Minimalismus",
        "HipHopActiveWear",
        "HippieBoho"
      ),
      labels = c(
        "Minimalismus",
        "HipHopActiveWear",
        "HippieBoho"
      )
    ),

    Condition = factor(
      Condition,
      levels = c(
        "Jacke-stark-dezentral",
        "Jacke-leicht-dezentral",
        "Schuh-stark-dezentral",
        "Jacke-stark-zentral",
        "Min-Baseline"
      ),
      labels = c(
        "GDS",
        "GDL",
        "KDS",
        "GZS",
        "M"
      )
    )
  )

# Mittelwerte und Standardfehler berechnen
stilzuordnung_summary <- stilzuordnung_long %>%
```

```

group_by(Condition, Stil) %>%
summarise(
  Mittelwert = mean(Bewertung, na.rm = TRUE),
  SE = sd(Bewertung, na.rm = TRUE) / sqrt(sum(!is.na(Bewertung))),
  .groups = "drop"
)

# Liniengraph mit Fehlerbalken
zeige_und_speichere_grafik(
  ggplot(
    stilzuordnung_summary,
    aes(
      x = Condition,
      y = Mittelwert,
      color = Stil,
      group = Stil
    )
  ) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 2.5) +
  geom_errorbar(
    aes(
      ymin = Mittelwert - SE,
      ymax = Mittelwert + SE
    ),
    width = 0.08,
    linewidth = 0.7
  ) +
  scale_y_continuous(
    breaks = seq(1, 5, by = 0.5),
    limits = c(1, 5.2)
  ) +
  scale_color_manual(
    values = c(
      "Minimalismus" = "#8B6F47",
      "HipHopActiveWear" = "#CDAA7D",
      "HippieBoho" = "#EED5B7"
    ),
    labels = c(
      "Minimalismus",
      "Hip Hop / Active Wear",
      "Hippie / Boho"
    )
  ) +
  labs(
    title = "Überprüfung der wahrgenommenen Stilzuordnung",
    x = NULL,
    y = "Stilbewertung (trifft nicht zu 1 - 5 trifft zu)",
    color = NULL
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(
      hjust = 0.5
    ),
    legend.position = "top"
  ),
  "Manipulationscheck der wahrgenommenen Stilzuordnung"
)

```

Überprüfung der wahrgenommenen Stilzuordnung

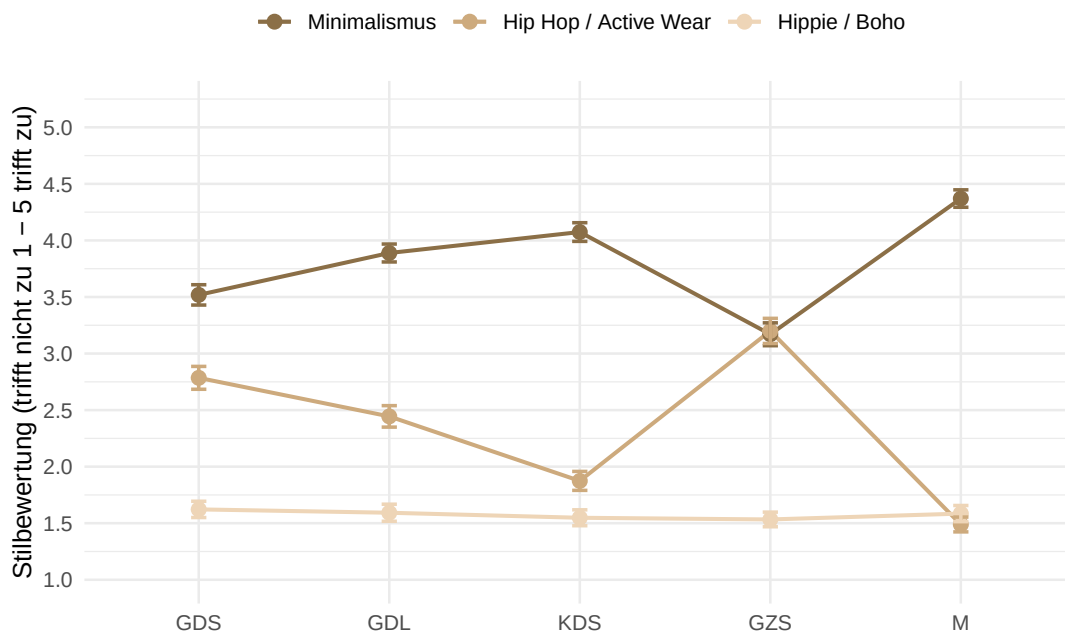


Table 143: Mittelwerte und Standardfehler der wahrgenommenen Stilzuordnung

Condition	Stil	Mittelwert	SE
GDS	Minimalismus	3.519	0.090
GDS	HipHopActiveWear	2.785	0.101
GDS	HippieBoho	1.622	0.072
GDL	Minimalismus	3.889	0.079
GDL	HipHopActiveWear	2.444	0.095
GDL	HippieBoho	1.593	0.075
KDS	Minimalismus	4.074	0.083
KDS	HipHopActiveWear	1.874	0.084
KDS	HippieBoho	1.548	0.070
GZS	Minimalismus	3.170	0.101
GZS	HipHopActiveWear	3.200	0.111
GZS	HippieBoho	1.533	0.065
M	Minimalismus	4.370	0.077
M	HipHopActiveWear	1.489	0.066
M	HippieBoho	1.585	0.072